

Metaheuristic Optimization: State of the Art

Thomas Weise (汤卫思)

tweise@hfu.edu.cn · <https://thomasweise.github.io>

School of Artificial Intelligence and Big Data
Hefei University
Hefei, Anhui, China

人工智能与大数据学院
合肥大学
中国安徽省合肥市

Outline



1. Introduction
2. Metaheuristics
3. Discrete Optimization
4. (Multi-)Permutation Problems
5. Continuous Optimization
6. Multi-Objective Optimization / Noisy Optimization
7. Tree-based Search Spaces
8. Summary
9. Advertisement





Introduction



Introduction



- Today, we will discuss the research frontier on metaheuristic optimization.



Introduction



- Today, we will discuss the research frontier on metaheuristic optimization.
- Therefore, we will first introduce what metaheuristic optimization is and why we need.

Introduction



- Today, we will discuss the research frontier on metaheuristic optimization.
- Therefore, we will first introduce what metaheuristic optimization is and why we need.
- Then we will discuss the current SOTA on several of the many, many, many domains where metaheuristics are applied.

Metaheuristics



- Metaheuristics are a class of algorithms used for **optimization**.

What is Optimization?



- There are **three** ways to approach this topic.

What is Optimization?



- There are **three** ways to approach this topic
 1. Optimization means searching for **superlatives**.

What is Optimization?



- There are **three** ways to approach this topic
 1. Optimization means searching for **superlatives**.
 2. Optimization is the art of solving **hard** problems.

What is Optimization?



- There are **three** ways to approach this topic
 1. Optimization means searching for **superlatives**.
 2. Optimization is the art of solving **hard** problems.
 3. Optimization allows us to find acceptable solutions for problems where our tools from high school maths *and* traditional algorithms from computer science are no longer enough, problems where exact solutions **cannot always be reached**.

Views on Optimization, Part 1



- Let's start with the first view: **Superlatives**.

Optimization is the search for superlatives



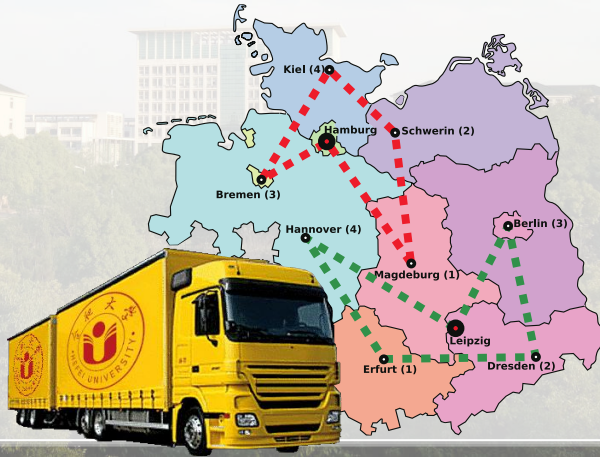
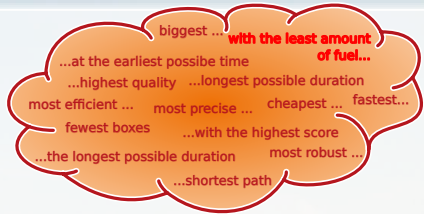
- Optimization means finding superlatives.

biggest ... with the least amount
of fuel...
...at the earliest possible time
...highest quality ...longest possible duration
most efficient ... most precise ... cheapest ... fastest...
fewest boxes ...with the highest score
...the longest possible duration most robust ...
...shortest path

The *superlative* form of an adjective is used to show that something has a quality to the greatest or least degree.

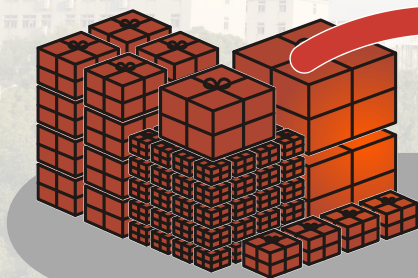
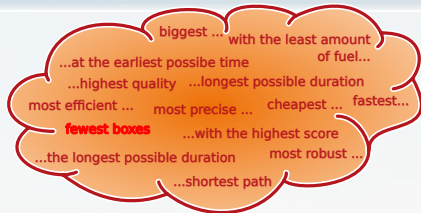
Optimization is the search for superlatives

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{107,133-135}.



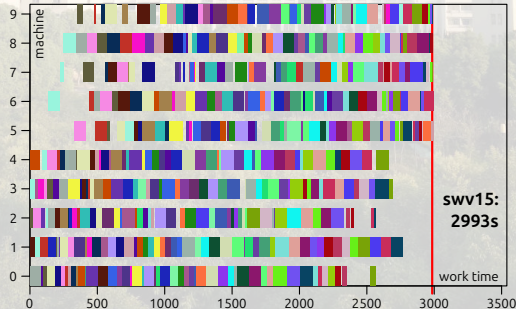
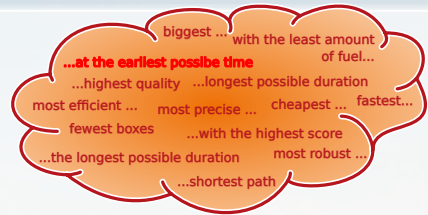
Optimization is the search for superlatives

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{107,133–135}.
- Pack a set of things into the **fewest** boxes^{41,117,148,149}.



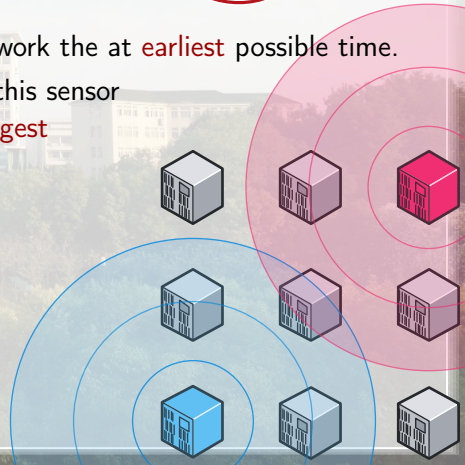
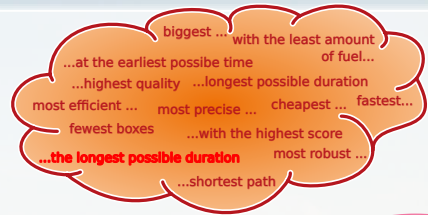
Optimization is the search for superlatives

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{107,133–135}.
- Pack a set of things into the **fewest** boxes^{41,117,148,149}.
- Assign tasks to machines such that we can finish our work the at **earliest** possible time.



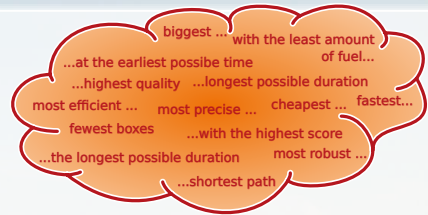
Optimization is the search for superlatives

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{107,133–135}.
- Pack a set of things into the **fewest** boxes^{41,117,148,149}.
- Assign tasks to machines such that we can finish our work the at **earliest** possible time.
- Find a strategy to manage the power of the nodes in this sensor network so that full coverage is guaranteed for the **longest** possible duration.



Optimization is the search for superlatives

- Optimization means finding superlatives.
- Pick up and deliver packages from different places to customers using the **least** amount of fuel^{107,133-135}.
- Pack a set of things into the **fewest** boxes^{41,117,148,149}.
- Assign tasks to machines such that we can finish our work the at **earliest** possible time.
- Find a strategy to manage the power of the nodes in this sensor network so that full coverage is guaranteed for the **longest** possible duration.
- And so on.



Views on Optimization, Part 2



- Now let us consider the second view: Solving hard problems.

Views on Optimization, Part 2



- Now let us consider the second view: Solving hard problems.
- Later, we learn how to quantify *hard*.

Views on Optimization, Part 2



- Now let us consider the second view: Solving hard problems.
- Later, we learn how to quantify *hard*.
- For now, let's first think about how to define *problem*?

What are Optimization Problems?

- There are two ways to define optimization problems.



What are Optimization Problems?



- The economic view.

Optimization Problem

An optimization problem is a situation which requires deciding for one choice from a set of possible alternatives in order to reach a predefined or required benefit at minimal costs.

What are Optimization Problems?



- The mathematical view.

Optimization Problem

An optimization problem is a situation which requires deciding for one choice from a set of possible alternatives in order to reach a predefined or required benefit at minimal costs.

Solving an optimization problem requires finding an input element x^* within a set $\mathbb{X}$ of allowed elements for which a mathematical function $f: \mathbb{X} \mapsto \mathbb{R}$ takes on the smallest possible value.

Optimization in Business

delivery

- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits



Optimization in Business



delivery

production

- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits
- production planning and scheduling
- optimized assignment of jobs/orders to machines
- optimization of production processes
- optimization of stock-keeping
- optimization of intra-enterprise logistics
- optimization of supply chains
- optimization of factory layouts and intra-factory logistics

Optimization in Business



delivery

production

management

- optimized logistics (business-to-customer)
 - planning and scheduling of maintenance visits
 - planning and scheduling of supply visits
- production planning and scheduling
 - optimized assignment of jobs/orders to machines
 - optimization of production processes
 - optimization of stock-keeping
 - optimization of intra-enterprise logistics
 - optimization of supply chains
 - optimization of factory layouts and intra-factory logistics
- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
(based current or predicted future customers)

Optimization in Business



delivery

production

management

products/
services

- optimized logistics (business-to-customer)
- planning and scheduling of maintenance visits
- planning and scheduling of supply visits
- production planning and scheduling
- optimized assignment of jobs/orders to machines
- optimization of production processes
- optimization of stock-keeping
- optimization of intra-enterprise logistics
- optimization of supply chains
- optimization of factory layouts and intra-factory logistics
- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
(based current or predicted future customers)
- optimization of product design
- optimization of product feature configuration
- optimization of service offers
- improved tailoring of products/services to customers

Optimization in Business



Optimization in Business



optimization
operations research
artificial intelligence (AI)
computational intelligence
machine learning
data mining

delivery

production

management

products/
services

sales

- optimized logistics (business-to-customer)
 - planning and scheduling of maintenance visits
 - planning and scheduling of supply visits
- production planning and scheduling
 - optimized assignment of jobs/orders to machines
 - optimization of production processes
 - optimization of stock-keeping
 - optimization of intra-enterprise logistics
 - optimization of supply chains
 - optimization of factory layouts and intra-factory logistics
- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
(based current or predicted future customers)
- optimization of product design
- optimization of product feature configuration
- optimization of service offers
- improved tailoring of products/services to customers
- optimization of pricing and offers
- mining of customer data for targeted offers

Optimization in Business



optimization
operations research
artificial intelligence (AI)
computational intelligence
machine learning
data mining

delivery

production

management

products/
services

sales

- optimized logistics (business-to-customer)
 - planning and scheduling of maintenance visits
 - planning and scheduling of supply visits
- production planning and scheduling
 - optimized assignment of jobs/orders to machines
 - optimization of production processes
 - optimization of stock-keeping
 - optimization of intra-enterprise logistics
 - optimization of supply chains
 - optimization of factory layouts and intra-factory logistics
- scheduling of employee work
- optimal assignment of employees to tasks or customers
- optimized locations for new branch offices
(based current or predicted future customers)
- optimization of product design
- optimization of product feature configuration
- optimization of service offers
- improved tailoring of products/services to customers
- optimization of pricing and offers
- mining of customer data for targeted offers

Views on Optimization, Part 3



- This brings us to the third view of optimization: We need to solve problems where equations are “not enough” and for which we cannot always find the best solution.

Views on Optimization, Part 3

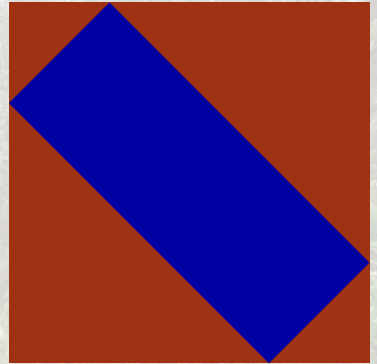


- This brings us to the third view of optimization: We need to solve problems where equations are “not enough” and for which we cannot always find the best solution.
- Let’s start with high school Maths, move to undergraduate computer science, and then approach this field of problems.

A Problem that we can solve with high school Maths equations



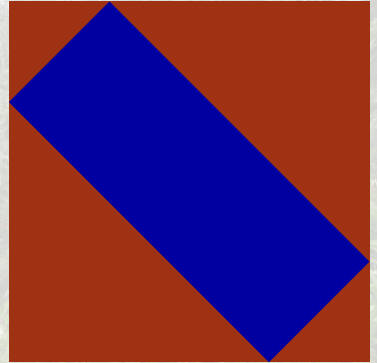
- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.



A Problem that we can solve with high school Maths equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?

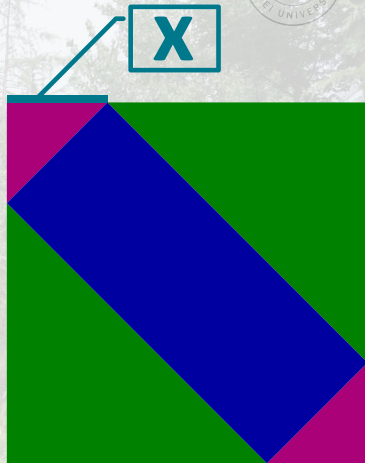


A Problem that we can solve with high school Maths equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?
- We can solve this by realizing that

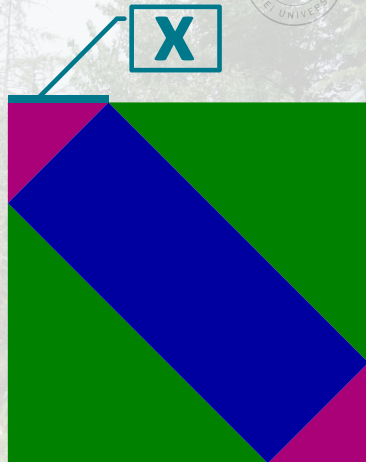
$$A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right).$$



A Problem that we can solve with high school Maths equations



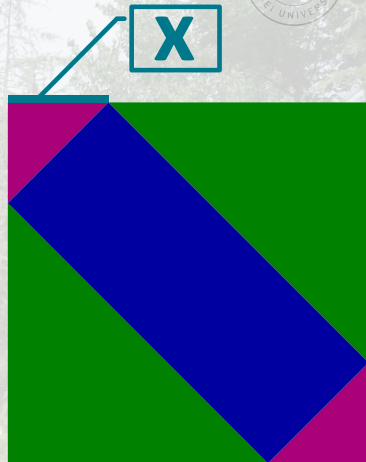
- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?
- We can solve this by realizing that
 $A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$ and then finding the maximum by setting
 $0 = A'_{\blacksquare}(x) = -4x + 26\text{cm}.$



A Problem that we can solve with high school Maths equations



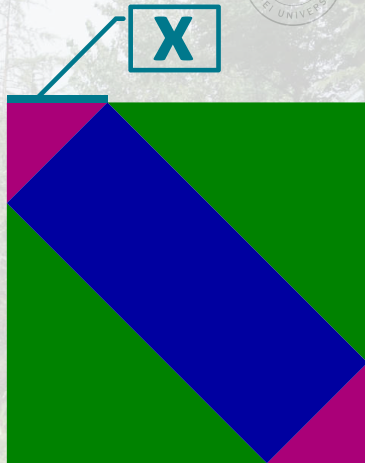
- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?
- We can solve this by realizing that
 $A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$ and then finding the maximum by setting
 $0 = A'_{\blacksquare}(x) = -4x + 26\text{cm}.$
- We get $\hat{x} = 6.5\text{cm}.$



A Problem that we can solve with high school Maths equations



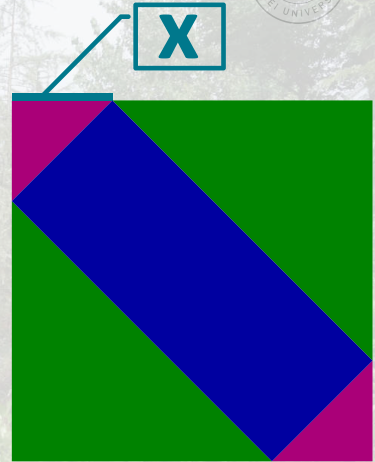
- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?
- We can solve this by realizing that
 $A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$ and then finding the maximum by setting
 $0 = A'_{\blacksquare}(x) = -4x + 26\text{cm}.$
- We get $\hat{x} = 6.5\text{cm}$ and, hence, $\widehat{A}_{\blacksquare} = 84.5\text{cm}^2.$



A Problem that we can solve with high school Maths equations



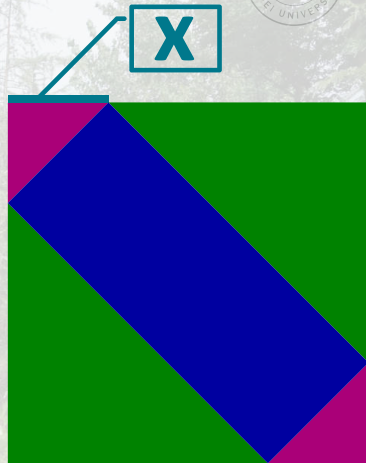
- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?
- We can solve this by realizing that $A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$ and then finding the maximum by setting $0 = A'_{\blacksquare}(x) = -4x + 26\text{cm}$.
- We get $\hat{x} = 6.5\text{cm}$ and, hence, $\widehat{A}_{\blacksquare} = 84.5\text{cm}^2$.
- Maxima and minima are superlatives = optima.



A Problem that we can solve with high school Maths equations



- A **rectangle** should be placed inside a **square** with side length 13cm as sketched.
- What area has the *largest* such **rectangle**?
- We can solve this by realizing that $A_{\blacksquare}(x) = (13\text{cm}^2) - 2\left(\frac{1}{2}x^2\right) - 2\left(\frac{1}{2}(13\text{cm} - x)^2\right)$ and then finding the maximum by setting $0 = A'_{\blacksquare}(x) = -4x + 26\text{cm}$.
- We get $\hat{x} = 6.5\text{cm}$ and, hence, $\widehat{A}_{\blacksquare} = 84.5\text{cm}^2$.
- Maxima and minima are superlatives = optima.
- Sometimes we can simply compute them and do not need fancy algorithms.



Problems that algorithms can solve efficiently



Problems that algorithms can solve efficiently



- I am at the starting point 合肥大学南二区北大门.



Problems that algorithms can solve efficiently



- I am at the **starting point** 合肥大学南二区北大门.
- I want to go to the **goal**: 食堂.



Problems that algorithms can solve efficiently



- I am at the **starting point** 合肥大学南二区北大门.
- I want to go to the **goal**: 食堂.
- How do I get there the **fastest**?

Problems that algorithms can solve efficiently

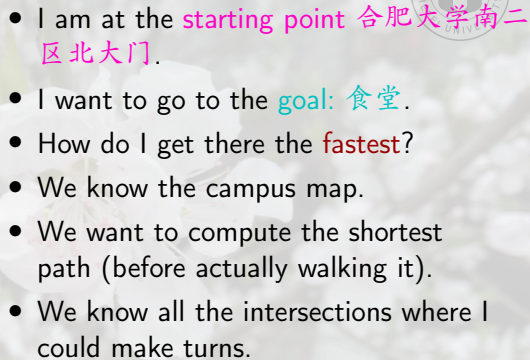


- I am at the starting point 合肥大学南二区北大门.
- I want to go to the goal: 食堂.
- How do I get there the fastest?
- We know the campus map.

Problems that algorithms can solve efficiently

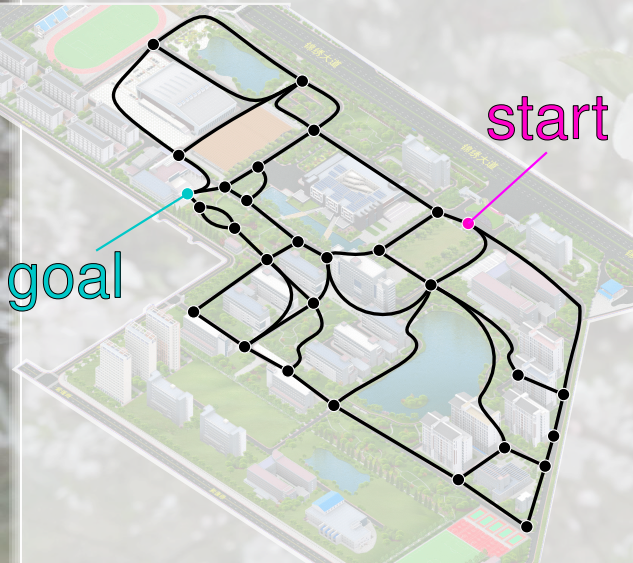


- I am at the **starting point** 合肥大学南二区北大门.
- I want to go to the **goal**: 食堂.
- How do I get there the **fastest**?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).



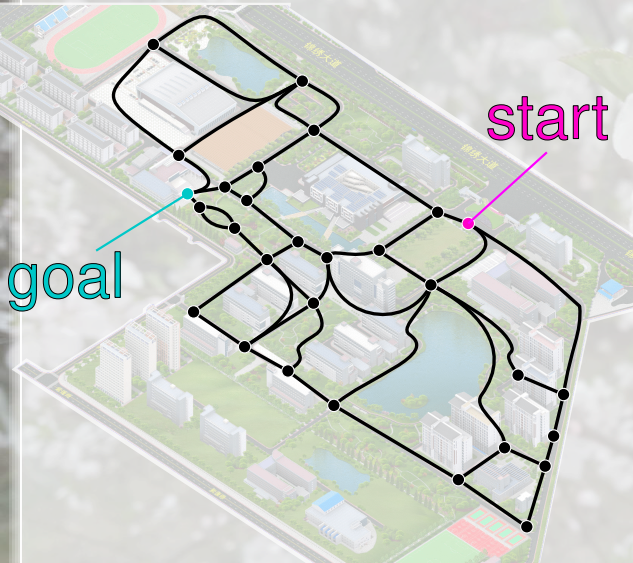
- I am at the **starting point** 合肥工业大学南区北大门.
- I want to go to the **goal**: 食堂.
- How do I get there the **fastest**?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).
- We know all the intersections where I could make turns.

Problems that algorithms can solve efficiently



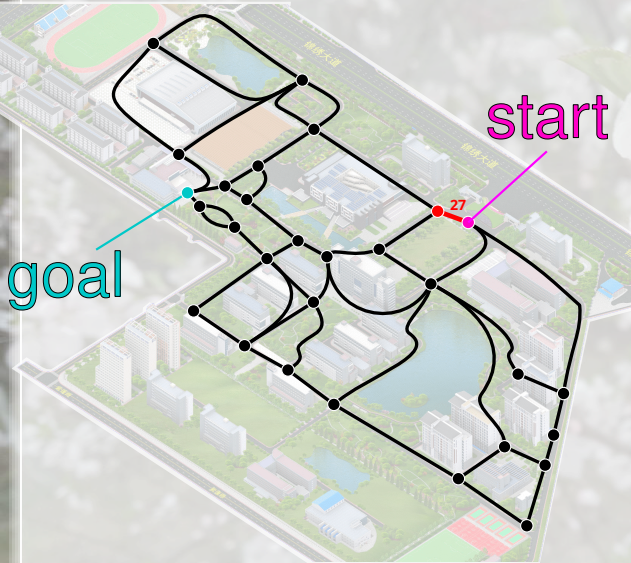
- I am at the **starting point** 合肥大学南二区北大门.
- I want to go to the **goal**: 食堂.
- How do I get there the **fastest**?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).
- We know all the intersections where I could make turns.
- There is no mathematical equation that we can use to solve this.

Problems that algorithms can solve efficiently



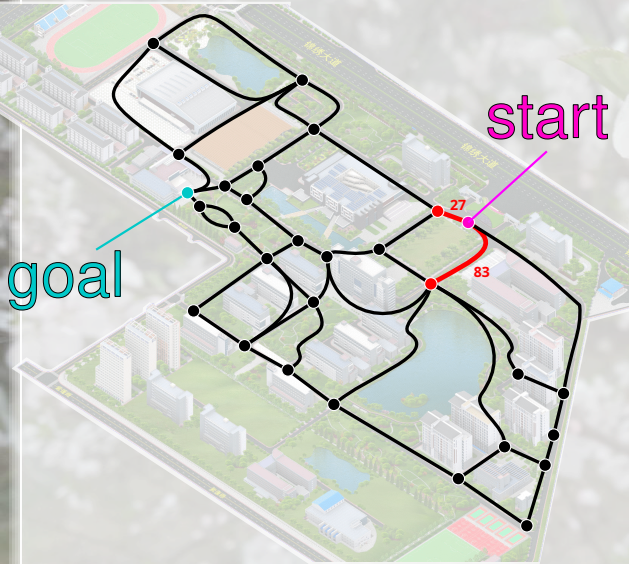
- I am at the **starting point** 合肥大学南二区北大门.
- I want to go to the **goal**: 食堂.
- How do I get there the **fastest**?
- We know the campus map.
- We want to compute the shortest path (before actually walking it).
- We know all the intersections where I could make turns.
- There is no mathematical equation that we can use to solve this.
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.

Problems that algorithms can solve efficiently



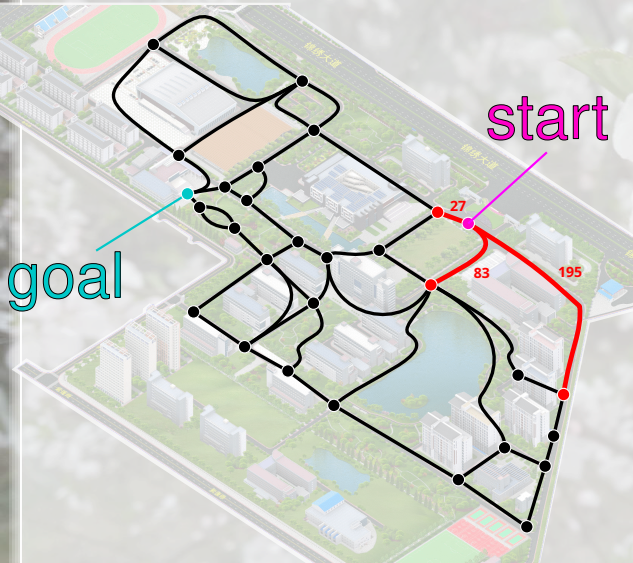
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- For example, I could walk for **27s** to this intersection.

Problems that algorithms can solve efficiently



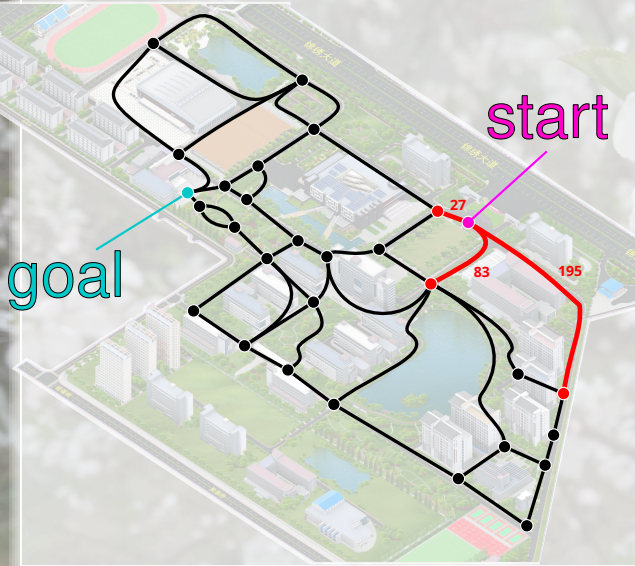
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- For example, I could walk for **27s** to this intersection.
- Or for **83s** to that one.

Problems that algorithms can solve efficiently



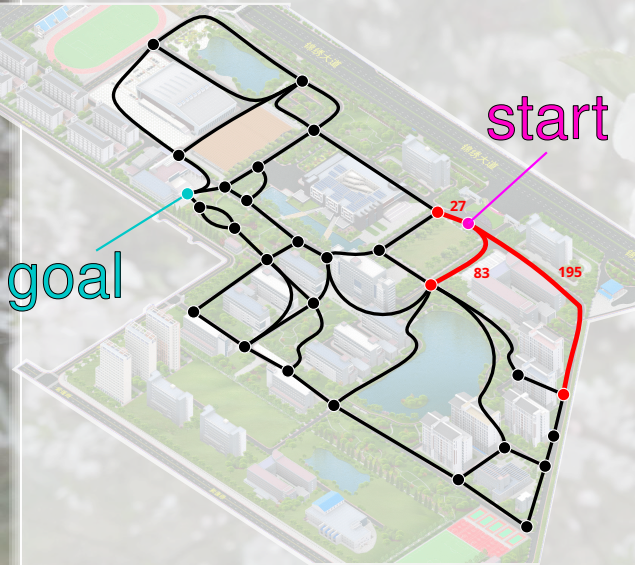
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- For example, I could walk for **27s** to this intersection.
- Or for **83s** to that one.
- Or for **195s** to that one.

Problems that algorithms can solve efficiently



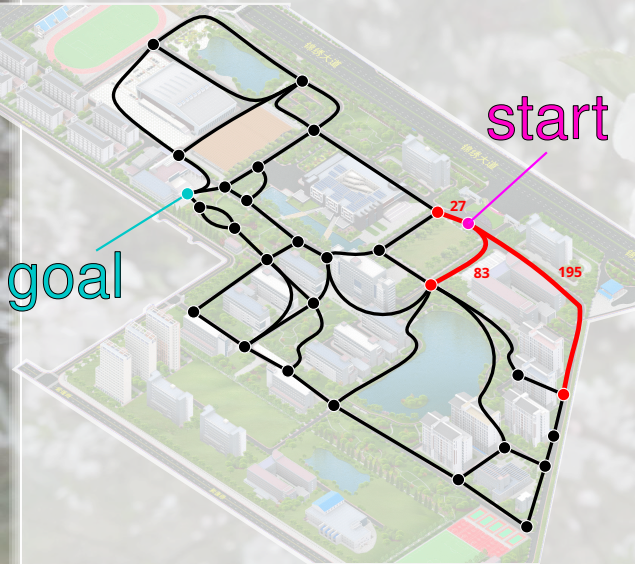
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- For example, I could walk for **27s** to this intersection.
- Or for **83s** to that one.
- Or for **195s** to that one.
- We have 3 choices: (a) **27s**, (b) **83s**, (c) **195s**.

Problems that algorithms can solve efficiently



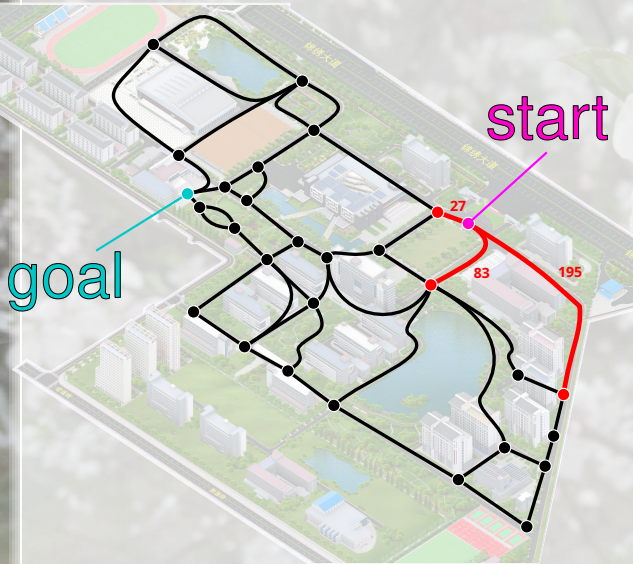
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- For example, I could walk for **27s** to this intersection.
- Or for **83s** to that one.
- Or for **195s** to that one.
- We have 3 choices: (a) **27s**, (b) **83s**, (c) **195s**.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.

Problems that algorithms can solve efficiently



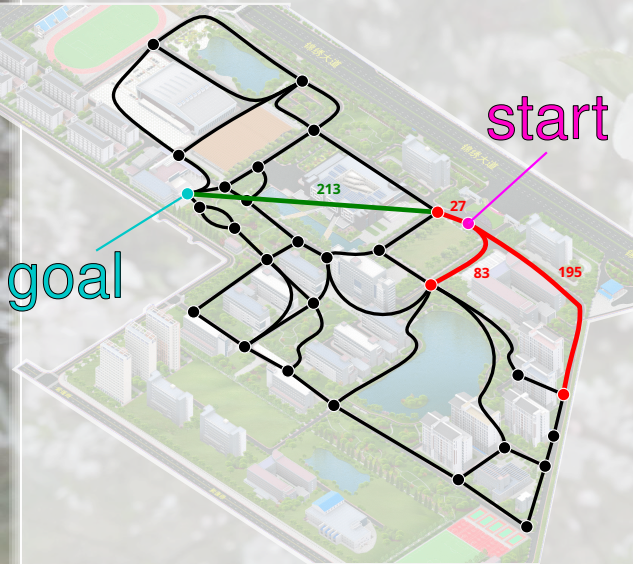
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- For example, I could walk for **27s** to this intersection.
- Or for **83s** to that one.
- Or for **195s** to that one.
- We have 3 choices: (a) **27s**, (b) **83s**, (c) **195s**.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.
- The actual walking distance can never be shorter than that.

Problems that algorithms can solve efficiently



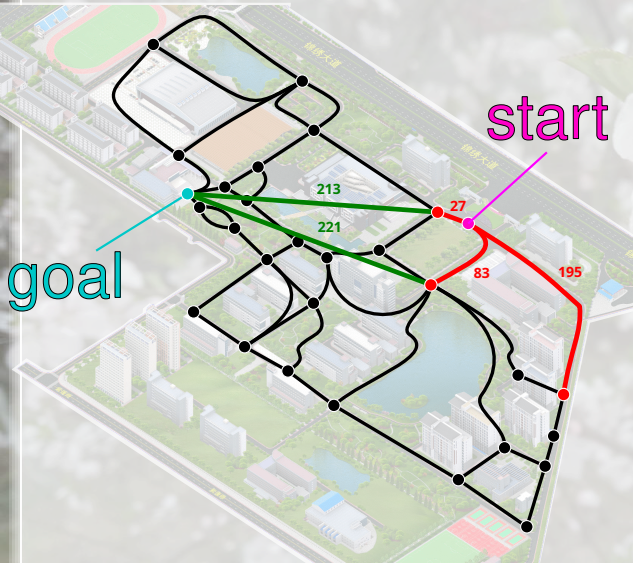
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We have 3 choices: (a) **27s**, (b) **83s**, (c) **195s**.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.
- The actual walking distance can never be shorter than that.
- **Adding** the two distances for each node gives us a good impression of how useful going there would be.

Problems that algorithms can solve efficiently



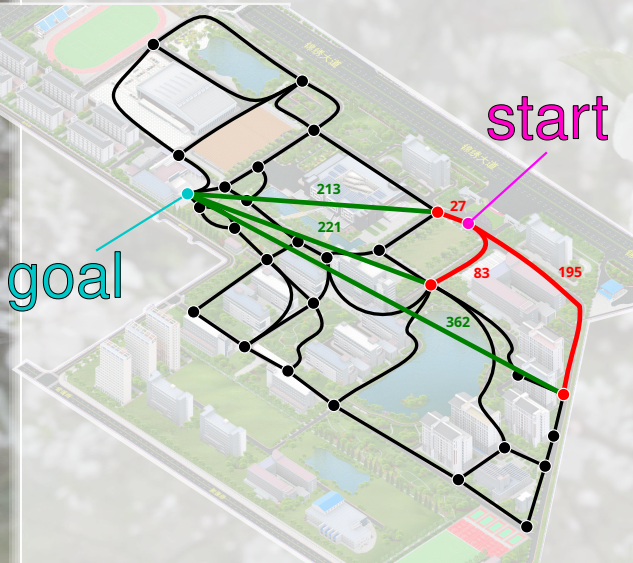
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We have 3 choices: (a) $27s + 213s$, (b) $83s$, (c) $195s$.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.
- The actual walking distance can never be shorter than that.
- **Adding** the two distances for each node gives us a good impression of how useful going there would be.

Problems that algorithms can solve efficiently



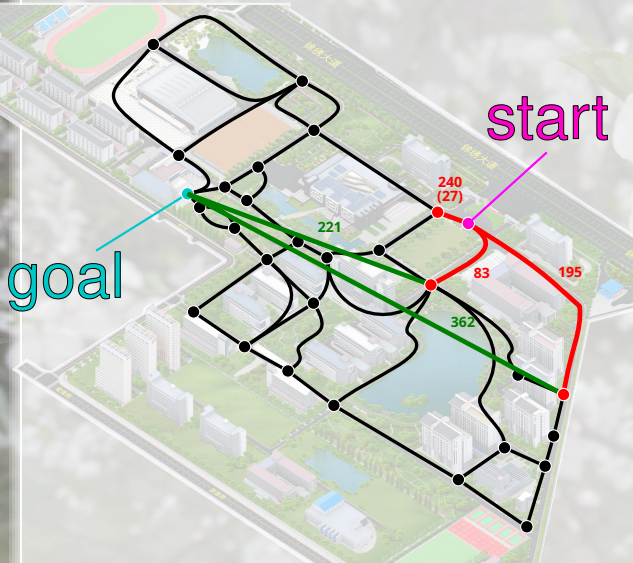
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s$.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.
- The actual walking distance can never be shorter than that.
- **Adding** the two distances for each node gives us a good impression of how useful going there would be.

Problems that algorithms can solve efficiently



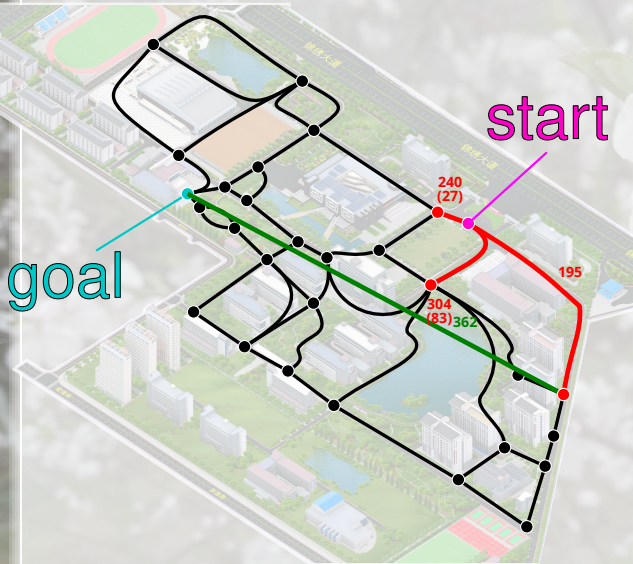
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s + 362s$.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.
- The actual walking distance can never be shorter than that.
- **Adding** the two distances for each node gives us a good impression of how useful going there would be.

Problems that algorithms can solve efficiently



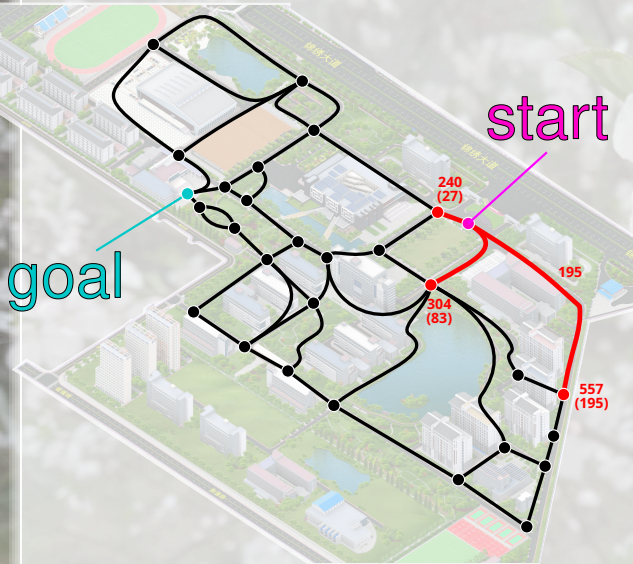
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s + 362s$.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.
- The actual walking distance can never be shorter than that.
- **Adding** the two distances for each node gives us a good impression of how useful going there would be.

Problems that algorithms can solve efficiently



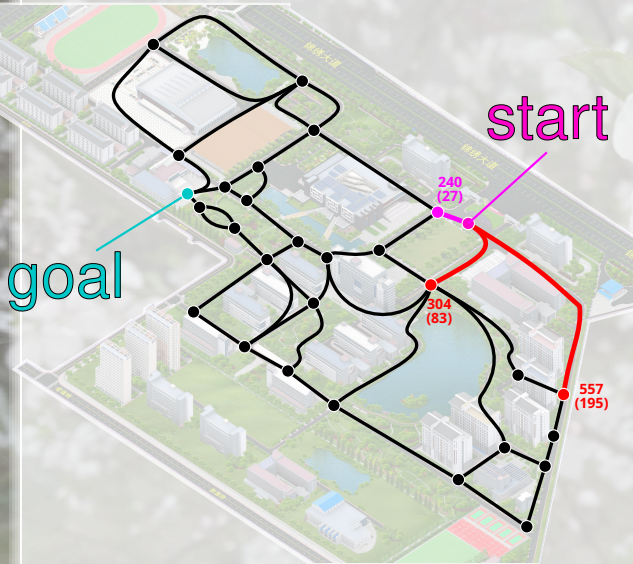
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s + 362s$.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.
- The actual walking distance can never be shorter than that.
- **Adding** the two distances for each node gives us a good impression of how useful going there would be.

Problems that algorithms can solve efficiently



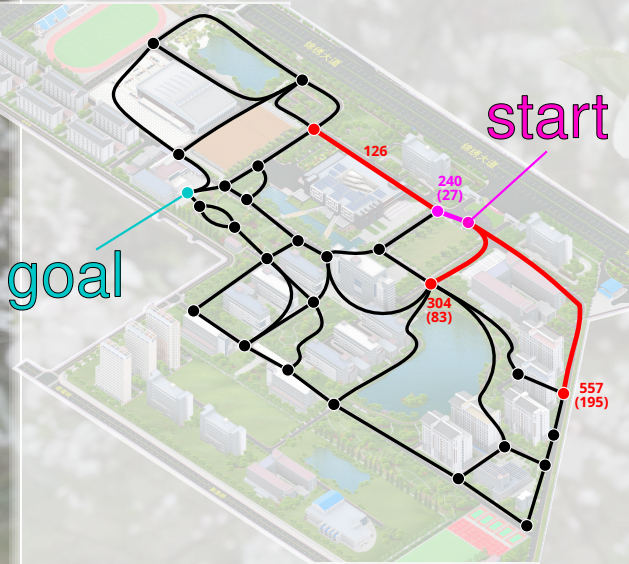
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s + 362s$.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.
- The actual walking distance can never be shorter than that.
- **Adding** the two distances for each node gives us a good impression of how useful going there would be.

Problems that algorithms can solve efficiently



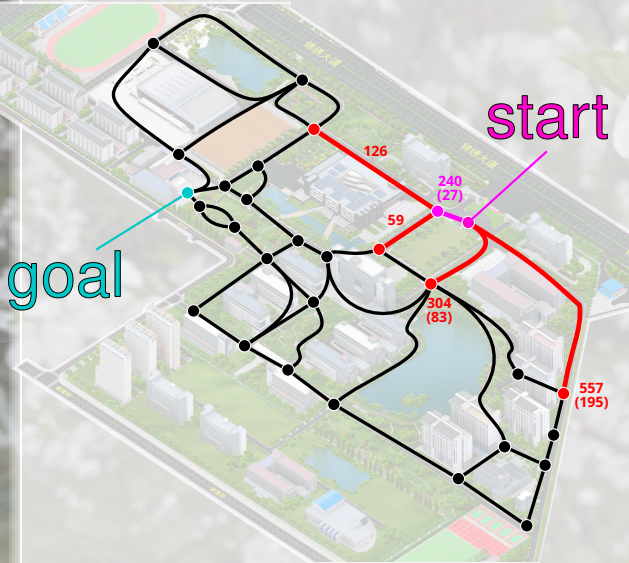
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We have 3 choices: (a) $27s + 213s$, (b) $83s + 221s$, (c) $195s + 362s$.
- For each intersection, we can compute the airline distance (**as the crow flies**) to the **食堂**.
- The actual walking distance can never be shorter than that.
- **Adding** the two distances for each node gives us a good impression of how useful going there would be.
- (a) it is...

Problems that algorithms can solve efficiently



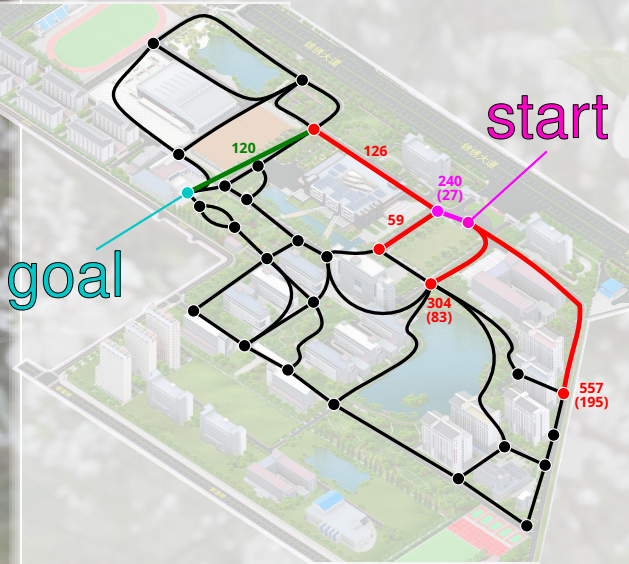
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



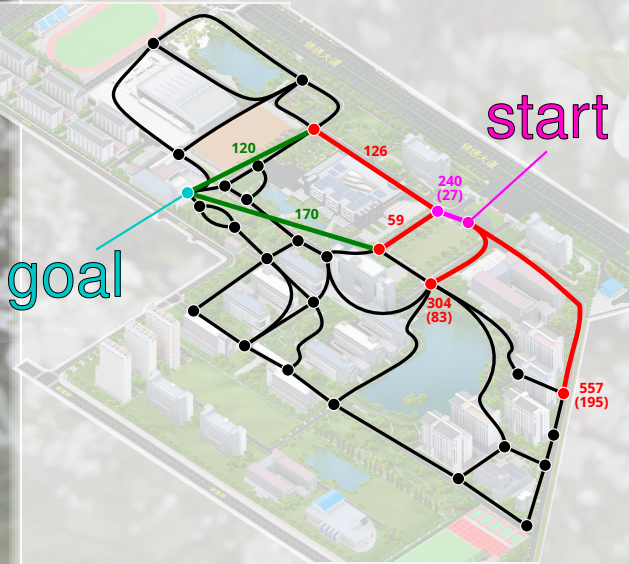
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



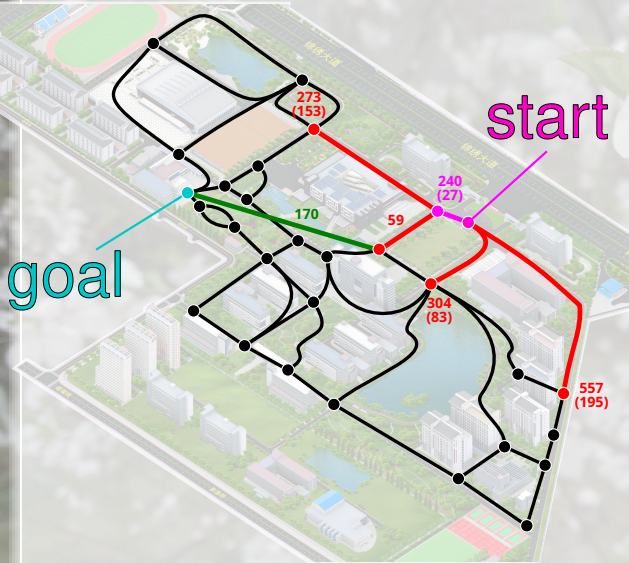
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



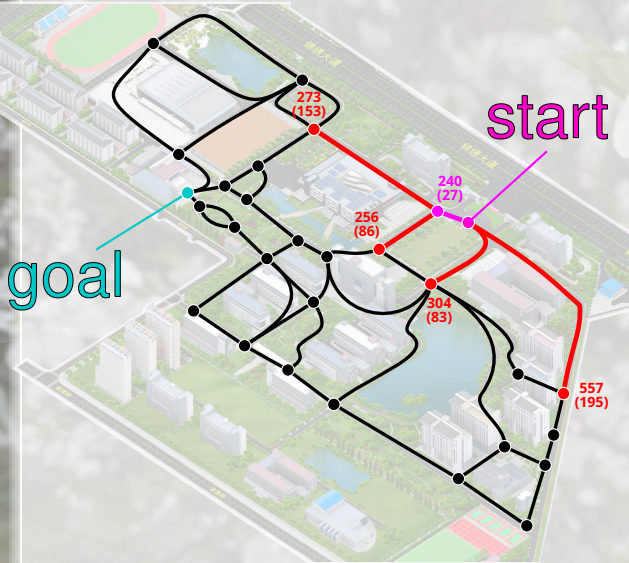
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



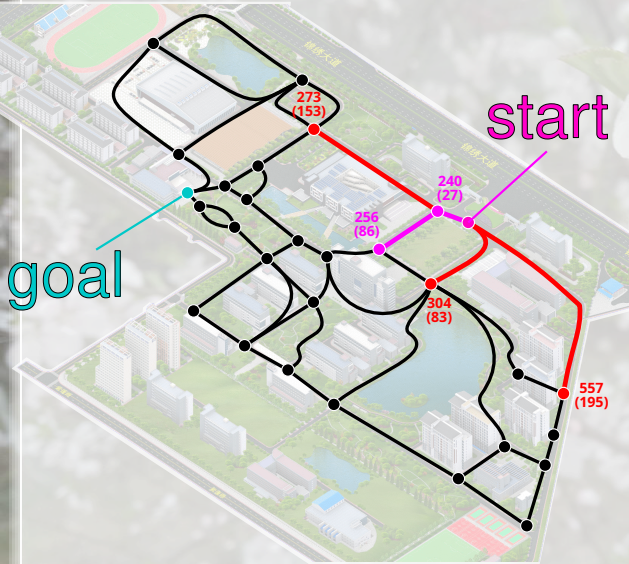
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



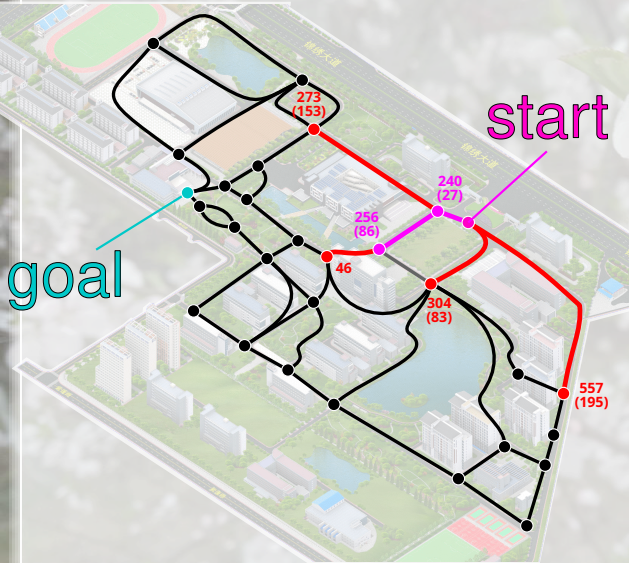
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a heuristic to a cost function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



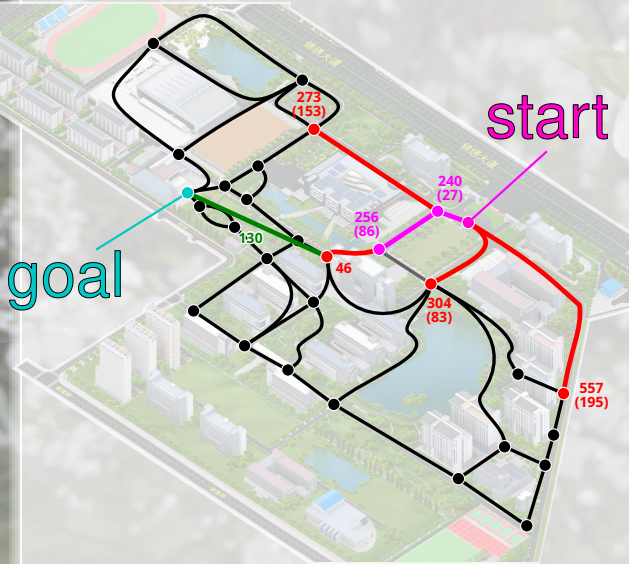
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



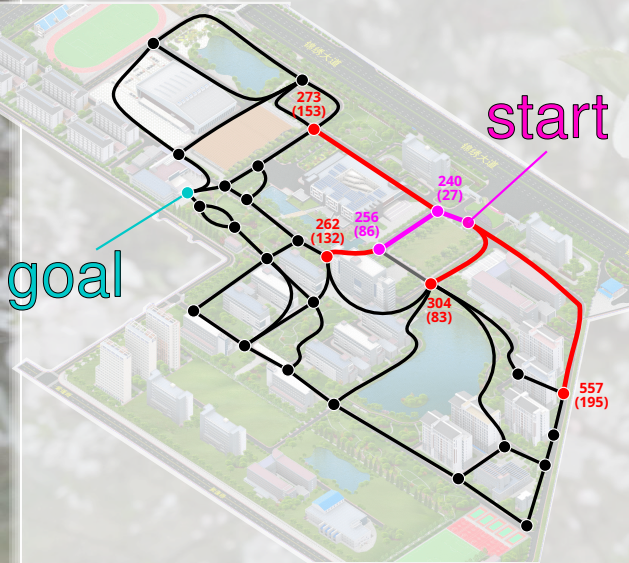
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



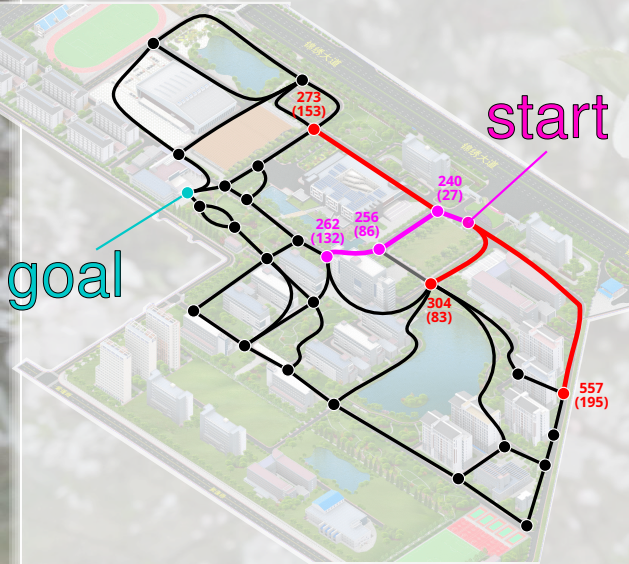
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



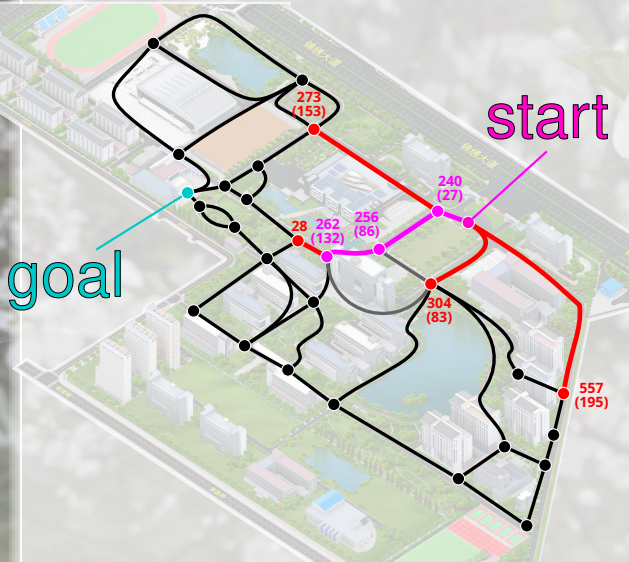
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



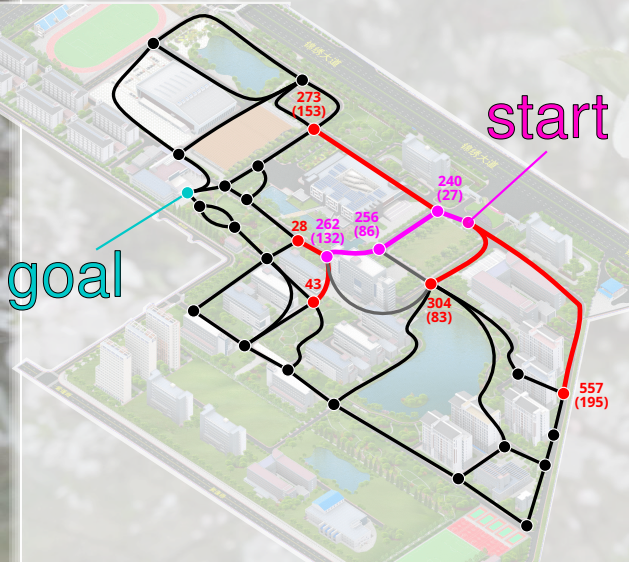
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



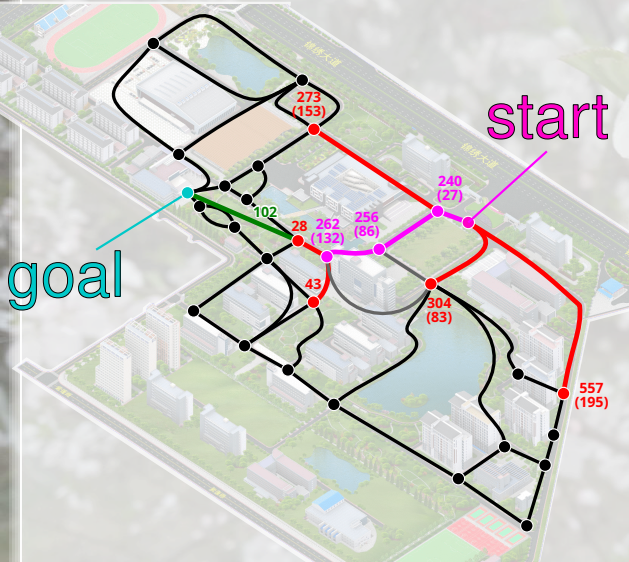
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



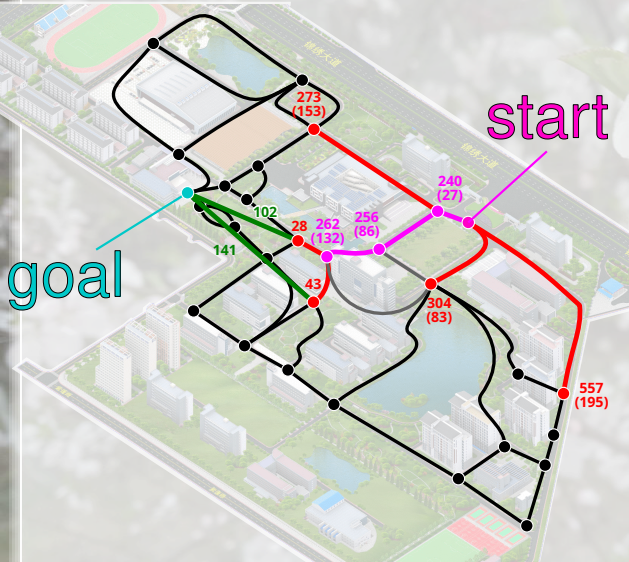
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



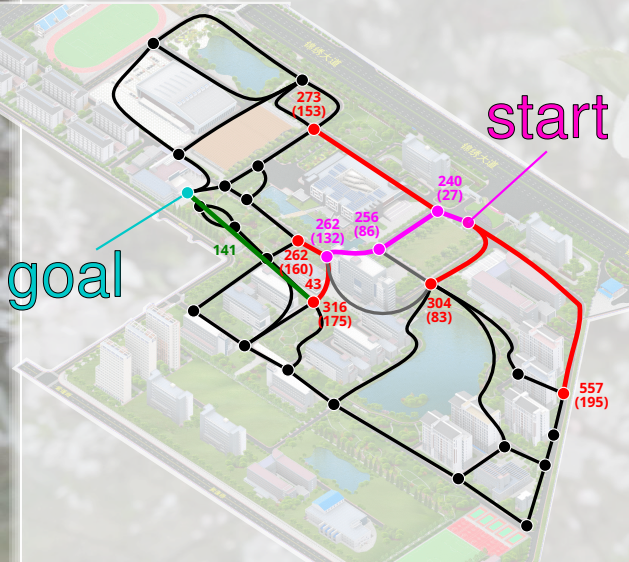
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



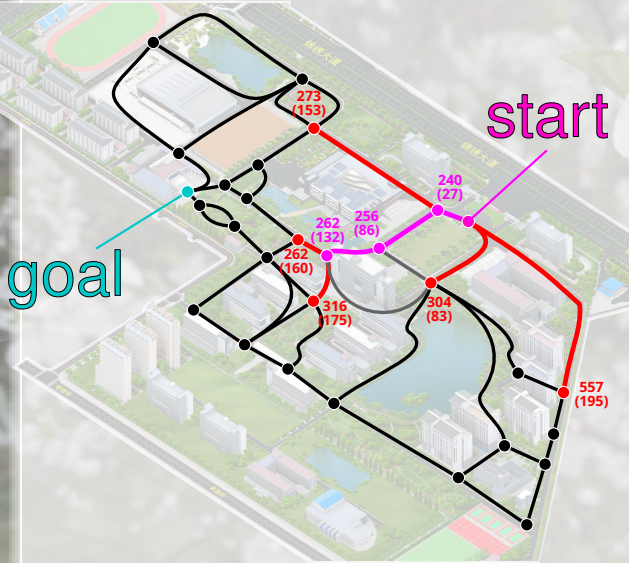
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



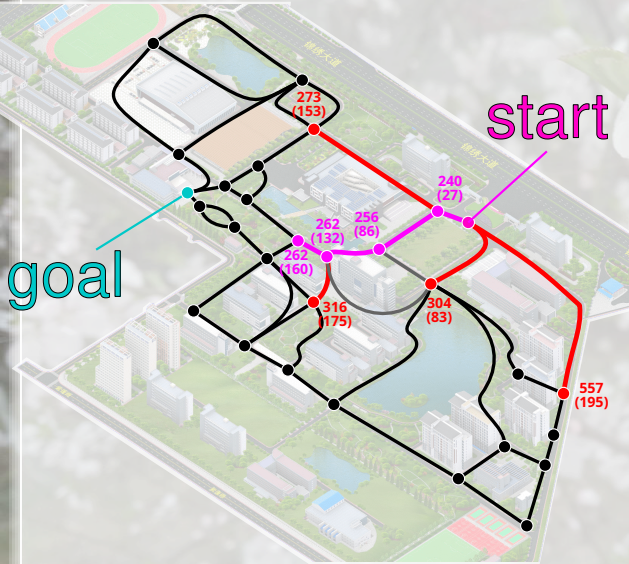
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



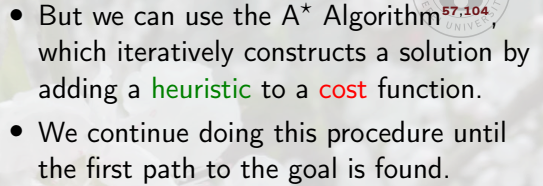
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



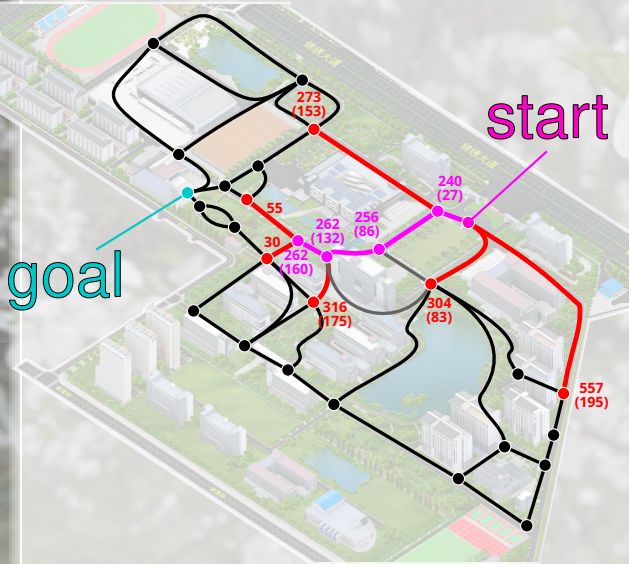
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

hm 57,104



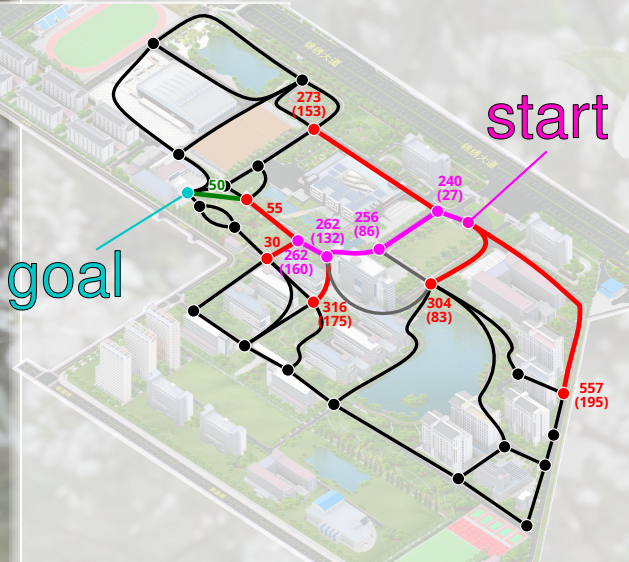
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a heuristic to a cost function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



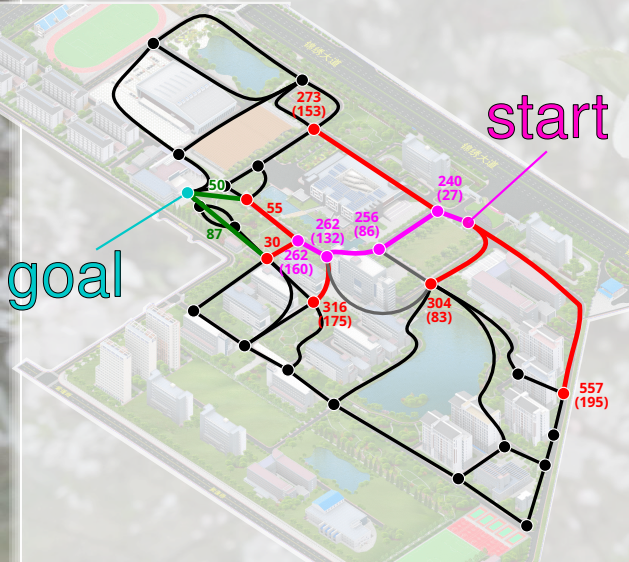
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



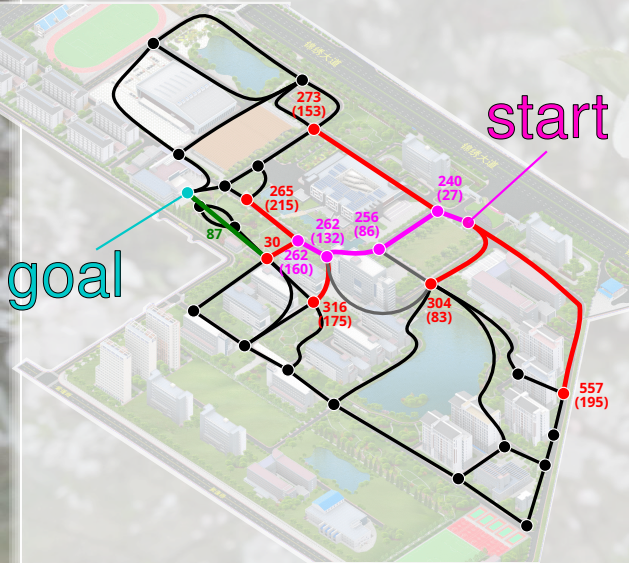
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



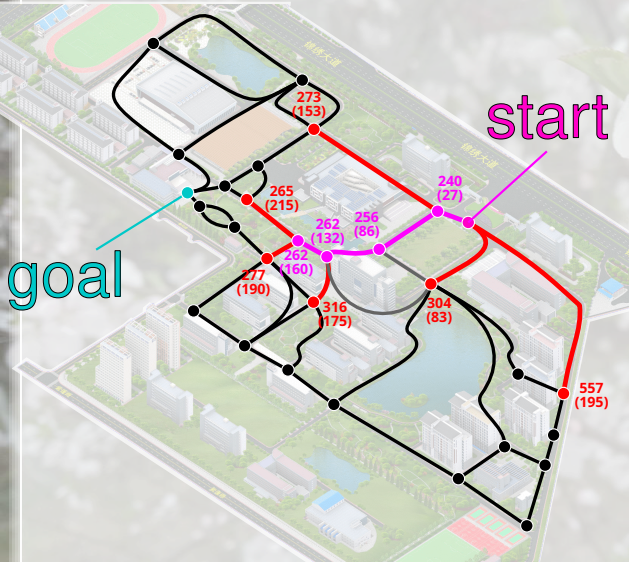
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



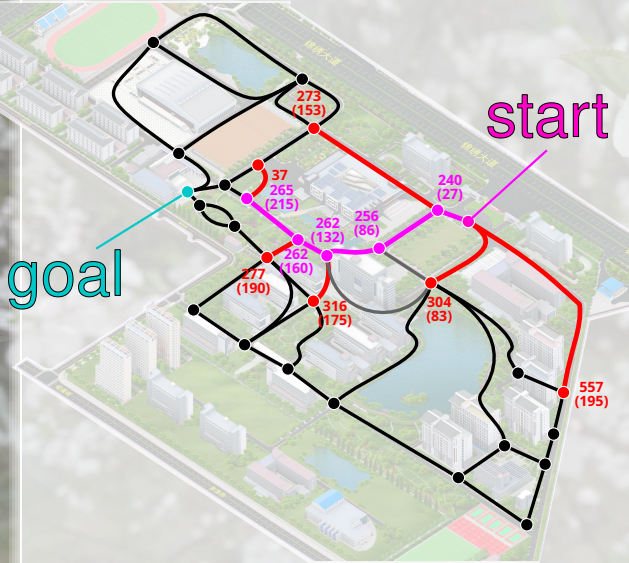
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



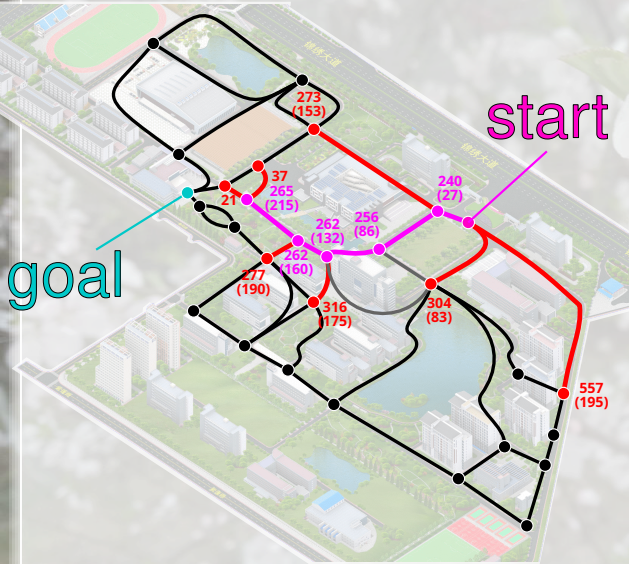
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



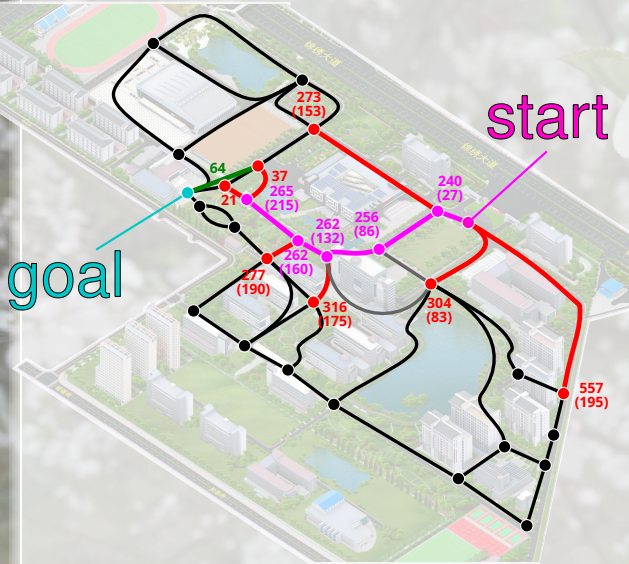
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



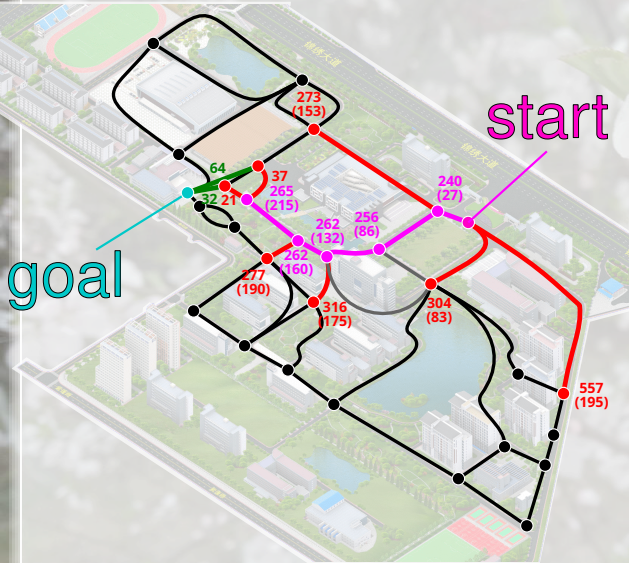
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



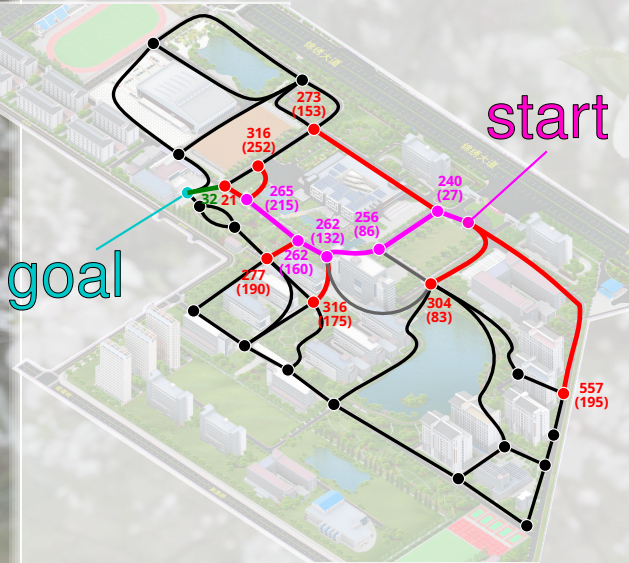
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



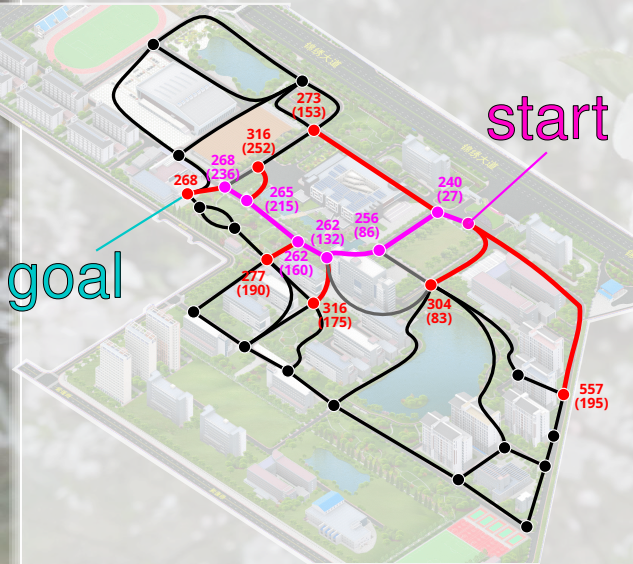
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.

Problems that algorithms can solve efficiently



- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.
- We found a first complete path from **start** to **goal**.

Problems that algorithms can solve efficiently



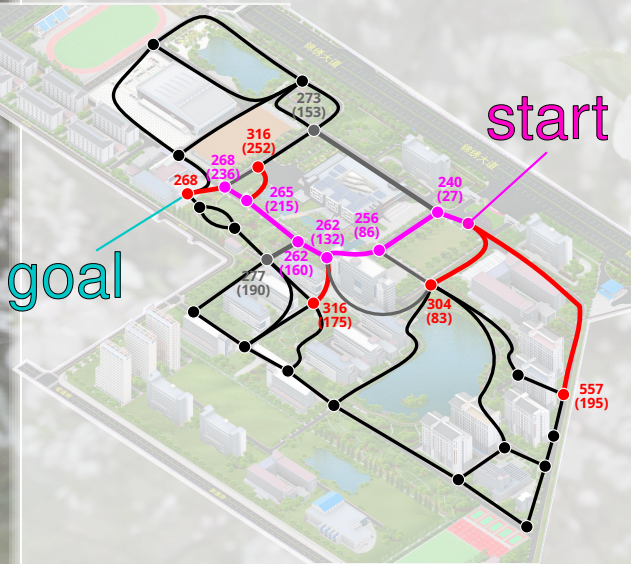
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.
- We found a first complete path from **start** to **goal**.
- It has the total length **268s**.

Problems that algorithms can solve efficiently



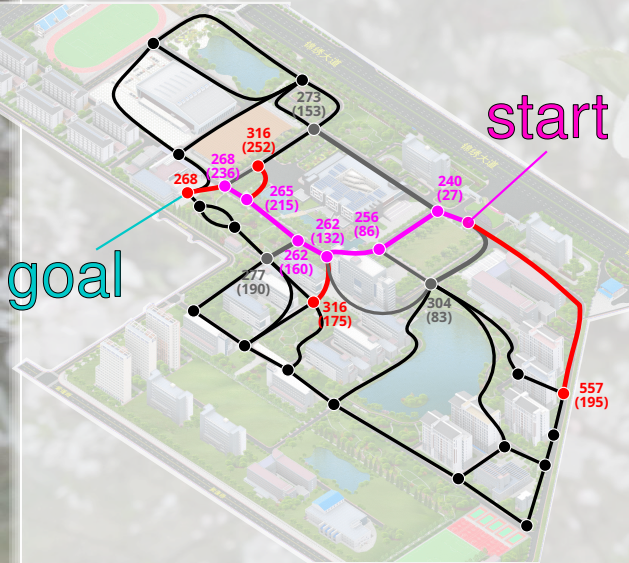
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.
- We found a first complete path from **start** to **goal**.
- It has the total length **268s**.
- Now we need to only check the remaining paths that might be shorter, those that are at least as long can be ignored.

Problems that algorithms can solve efficiently



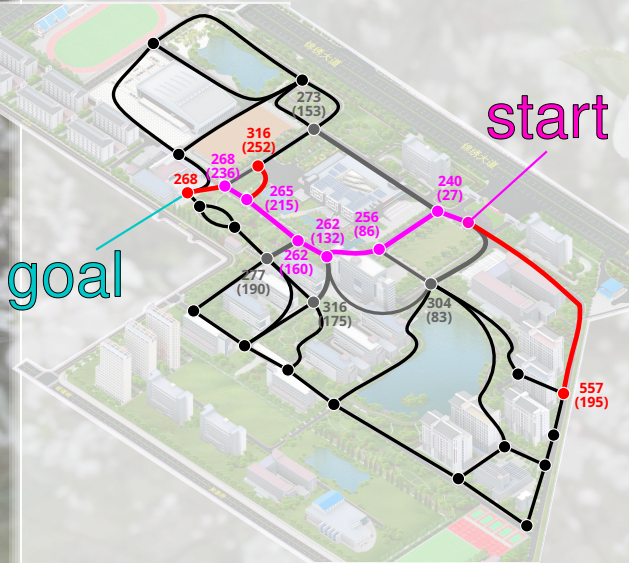
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.
- We found a first complete path from **start** to **goal**.
- It has the total length **268s**.
- Now we need to only check the remaining paths that might be shorter, those that are at least as long can be ignored.

Problems that algorithms can solve efficiently



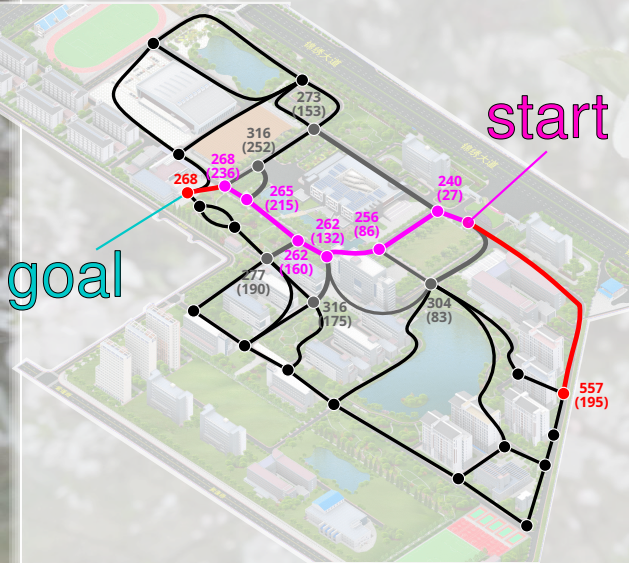
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.
- We found a first complete path from **start** to **goal**.
- It has the total length **268s**.
- Now we need to only check the remaining paths that might be shorter, those that are at least as long can be ignored.

Problems that algorithms can solve efficiently



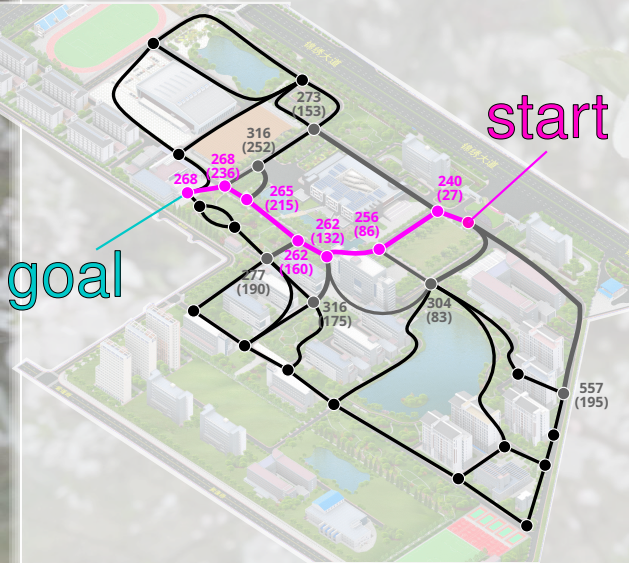
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.
- We found a first complete path from **start** to **goal**.
- It has the total length **268s**.
- Now we need to only check the remaining paths that might be shorter, those that are at least as long can be ignored.

Problems that algorithms can solve efficiently



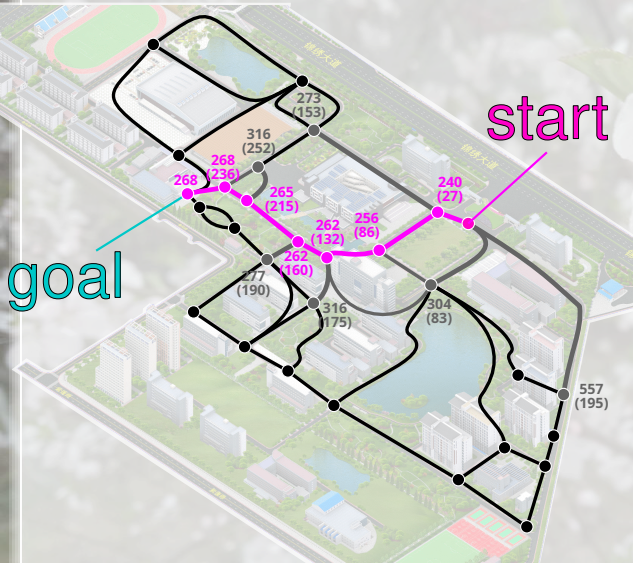
- But we can use the A* Algorithm^{57,104}, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- We continue doing this procedure until the first path to the goal is found.
- We found a first complete path from **start** to **goal**.
- It has the total length **268s**.
- Now we need to only check the remaining paths that might be shorter, those that are at least as long can be ignored.

Problems that algorithms can solve efficiently



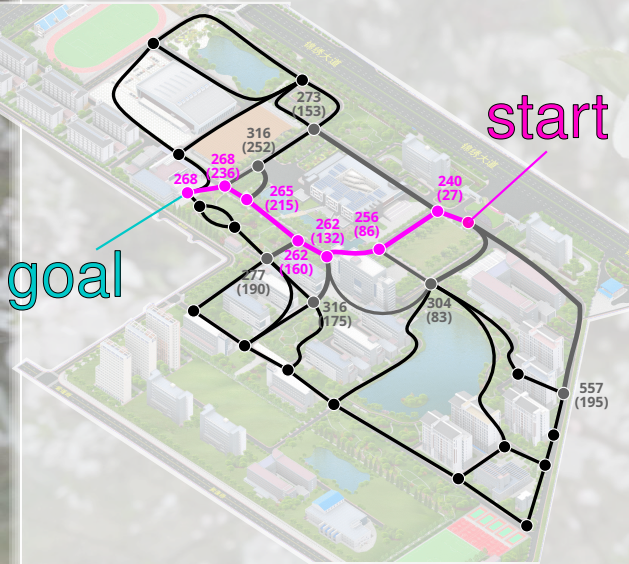
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a heuristic to a cost function.
- We continue doing this procedure until the first path to the goal is found.
- We found a first complete path from start to goal.
- It has the total length 268s.
- Now we need to only check the remaining paths that might be shorter, those that are at least as long can be ignored.

Problems that algorithms can solve efficiently



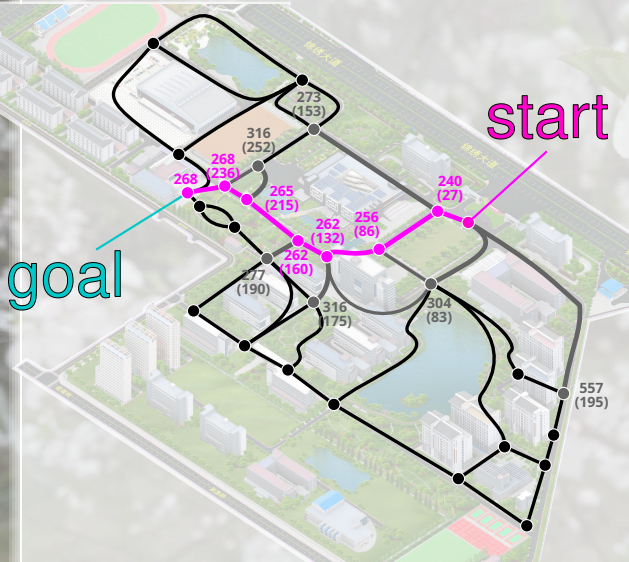
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a heuristic to a cost function.
- We continue doing this procedure until the first path to the goal is found.
- We found a first complete path from start to goal.
- It has the total length 268s.
- Now we need to only check the remaining paths that might be shorter, those that are at least as long can be ignored.
- We confirmed the shortest possible path and it takes 268s.

Problems that algorithms can solve efficiently



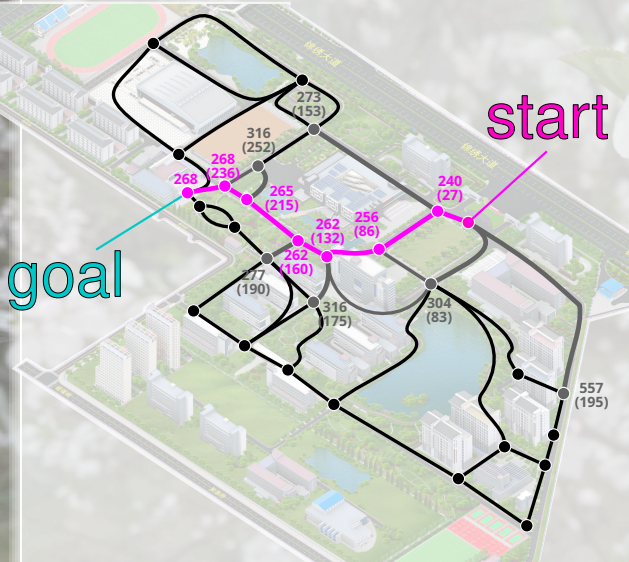
- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- In the worst case, the A* Algorithm needs to “look” at all intersections (nodes) and streets (edges) once.

Problems that algorithms can solve efficiently



- But we can use the A* Algorithm, which iteratively constructs a solution by adding a **heuristic** to a **cost** function.
- In the worst case, the A* Algorithm needs to “look” at all intersections (nodes) and streets (edges) once.
- Usually, it is quite efficient to find shortest paths on maps.

Problems that algorithms can solve efficiently

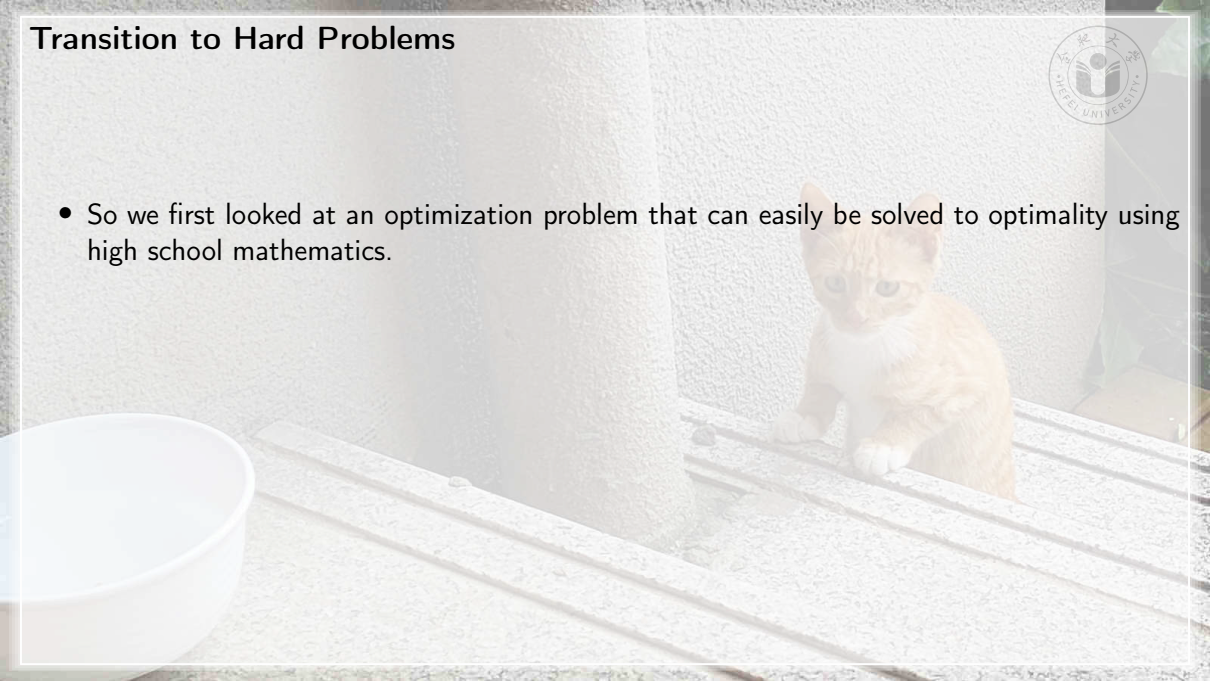


- But we can use the A* Algorithm, which iteratively constructs a solution by adding a heuristic to a cost function.
- In the worst case, the A* Algorithm needs to “look” at all intersections (nodes) and streets (edges) once.
- Usually, it is quite efficient to find shortest paths on maps.
- If we look for shortest paths that do not visit any node twice, where the edge distances are not negative, where the number of choices per node is limited, and where the graph and path are not too big/long – then this algorithm is very efficient.

Transition to Hard Problems



- So we first looked at an optimization problem that can easily be solved to optimality using high school mathematics.



Transition to Hard Problems



- So we first looked at an optimization problem that can easily be solved to optimality using high school mathematics.
- Then we looked at a problem that can be solved to optimality using an algorithm that you should have learned during your Bachelor's studies.

Transition to Hard Problems



- So we first looked at an optimization problem that can easily be solved to optimality using high school mathematics.
- Then we looked at a problem that can be solved to optimality using an algorithm that you should have learned during your Bachelor's studies.
- What about *hard* problems?

Transition to Hard Problems



- So we first looked at an optimization problem that can easily be solved to optimality using high school mathematics.
- Then we looked at a problem that can be solved to optimality using an algorithm that you should have learned during your Bachelor's studies.
- What about *hard* problems?
- What about problems where traditional Maths and algorithms do not really help us much?

Transition to Hard Problems



- So we first looked at an optimization problem that can easily be solved to optimality using high school mathematics.
- Then we looked at a problem that can be solved to optimality using an algorithm that you should have learned during your Bachelor's studies.
- What about *hard* problems?
- What about problems where traditional Maths and algorithms do not really help us much?
- Here we go.

Hard Problems (1)



- There exists a group of problems that are *hard*.

Hard Problems (1)



- There exists a group of problems that are *hard*.
- Actually, most of the problems I mention at the beginning of this talk belong to this group of hard problems.

Hard Problems (1)



- There exists a group of problems that are *hard*.
- Actually, most of the problems I mention at the beginning of this talk belong to this group of hard problems.
- A hard problem cannot be solved both exactly and efficiently.

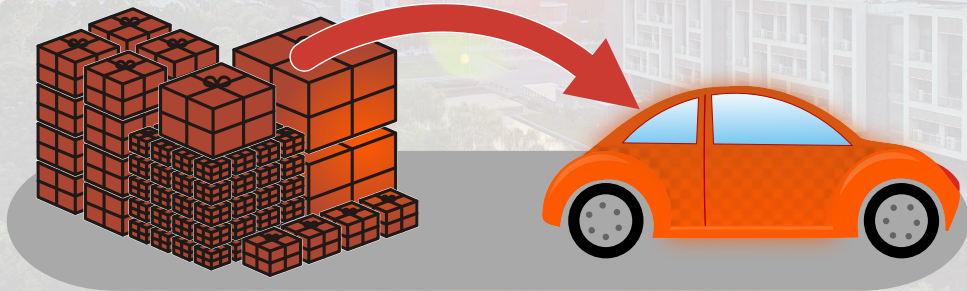
Hard Problems (1)



- There exists a group of problems that are *hard*.
- Actually, most of the problems I mention at the beginning of this talk belong to this group of hard problems.
- A hard problem cannot be solved both exactly and efficiently.
- Let's look at two quick examples.

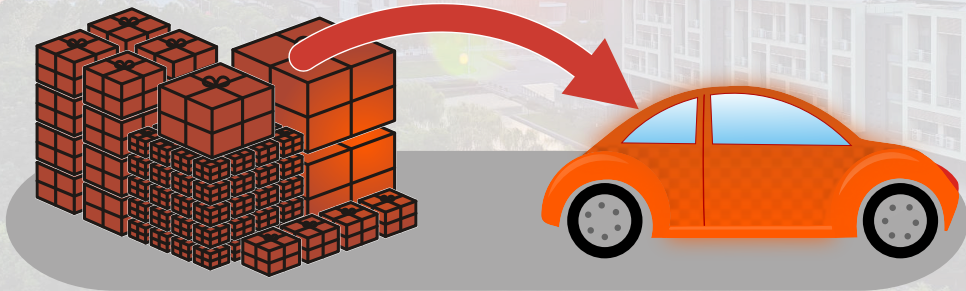
Bin Packing Problem

- My car can carry T kg of weight.



Bin Packing Problem

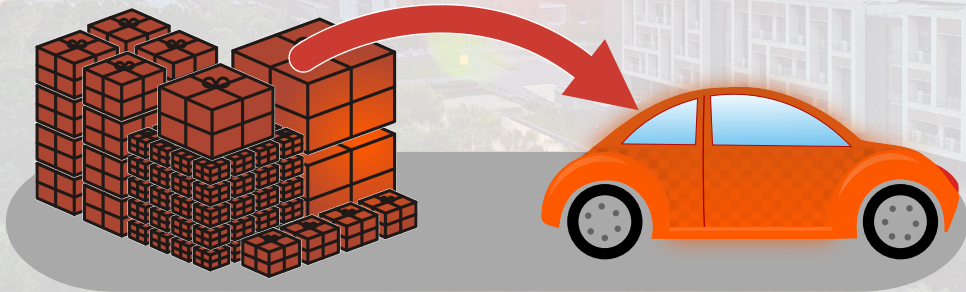
- My car can carry T kg of weight.
- I have n objects, each with weight w_i with i in 1 to n .



Bin Packing Problem



- My car can carry T kg of weight.
- I have n objects, each with weight w_i with i in 1 to n .
- How can I pack my car so that I can carry them from A to B with the fewest possible hauls?^{148,149}



Traveling Salesperson Problem



- Find the shortest path that visits n locations and returns back to its origin. ^{3,78}



Hard Problems (2)

- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.



Hard Problems (2)

- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?



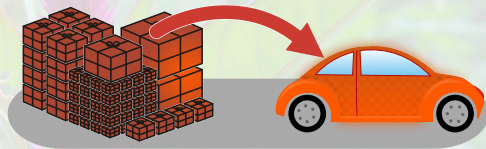
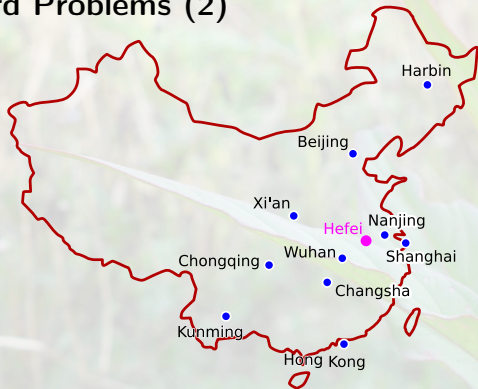
Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?

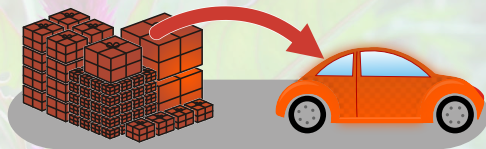
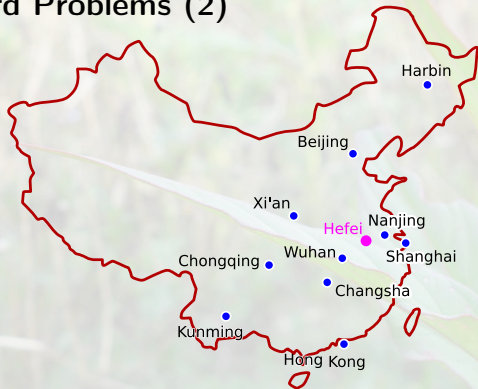
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)



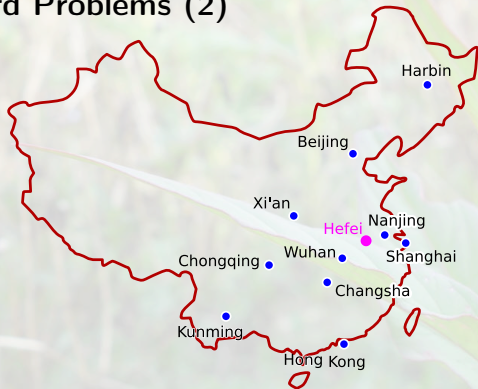
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)

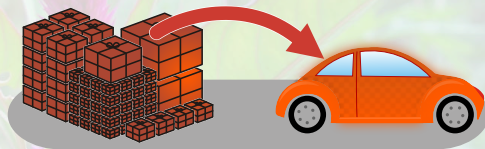


If an algorithm guarantees to always find the **best-possible** solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)



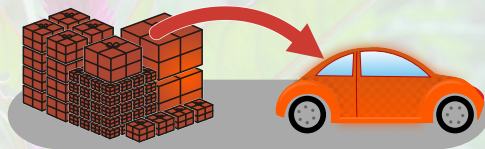
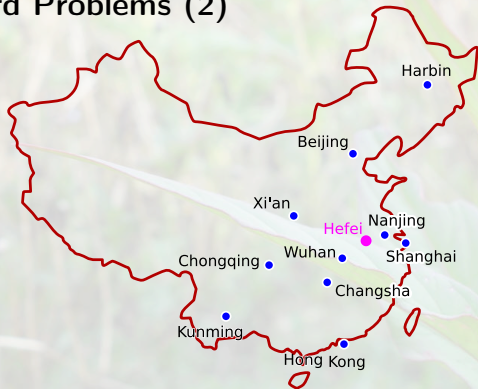
shortest round-trip tour



packing with fewest hauls

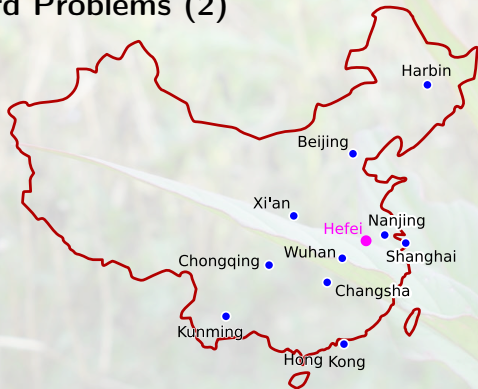
If an algorithm guarantees to always find the **best-possible** solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)

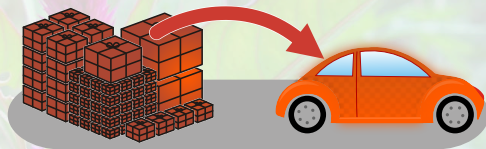


If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the **input size** n .

Hard Problems (2)



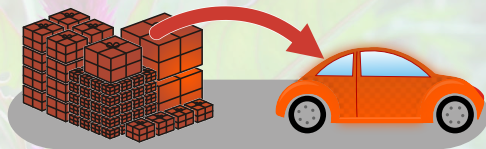
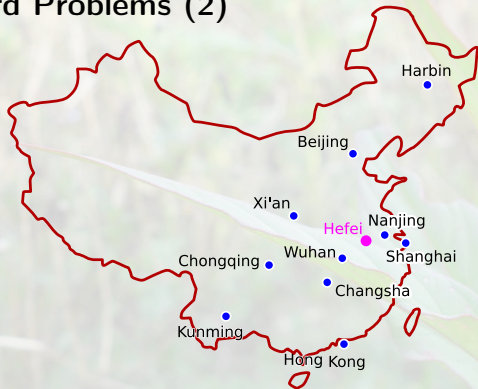
number n of cities to visit



number n of items to pack

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the **input size** n .

Hard Problems (2)

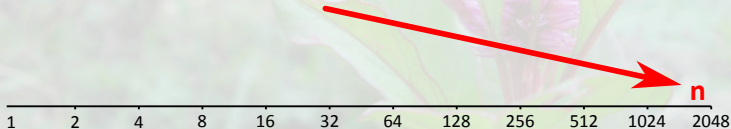


If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)

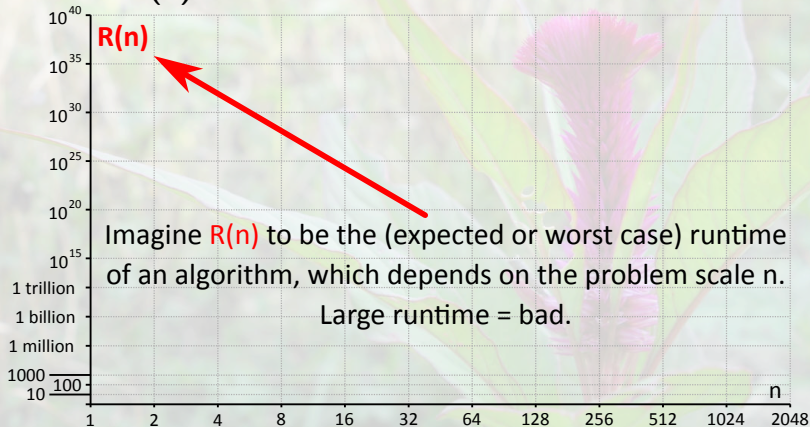


The problem scale n usually corresponding to the number of decision variables.



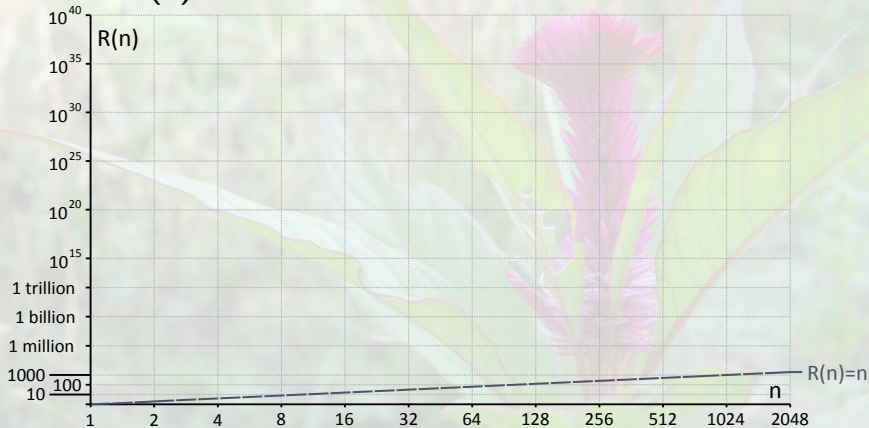
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



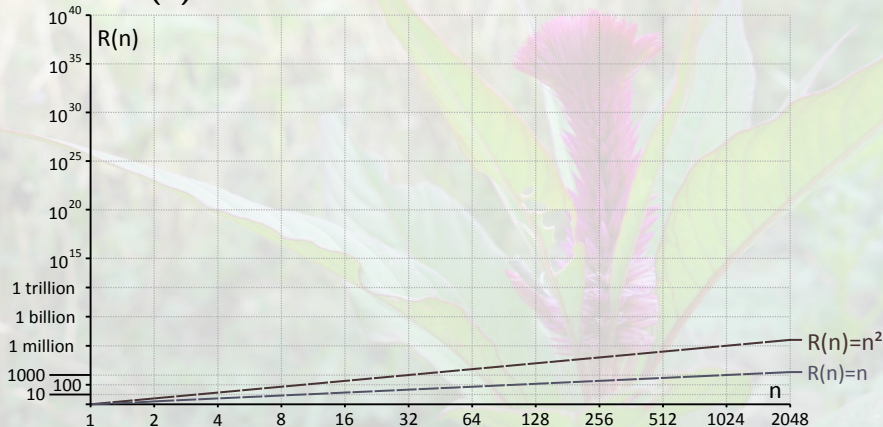
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



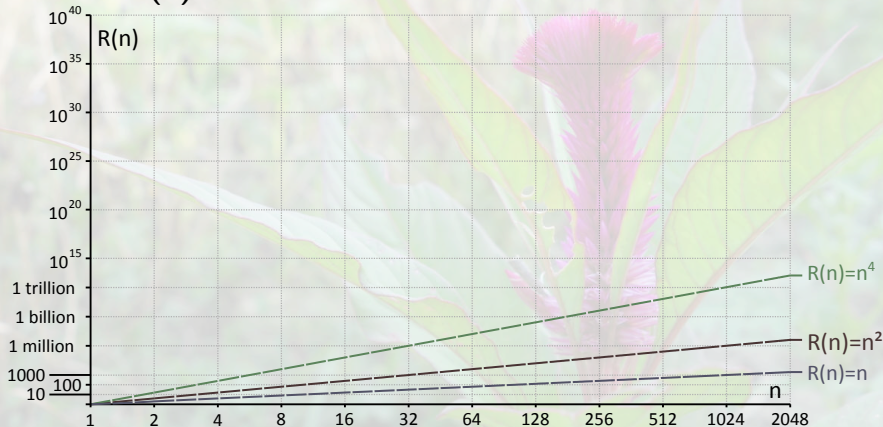
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



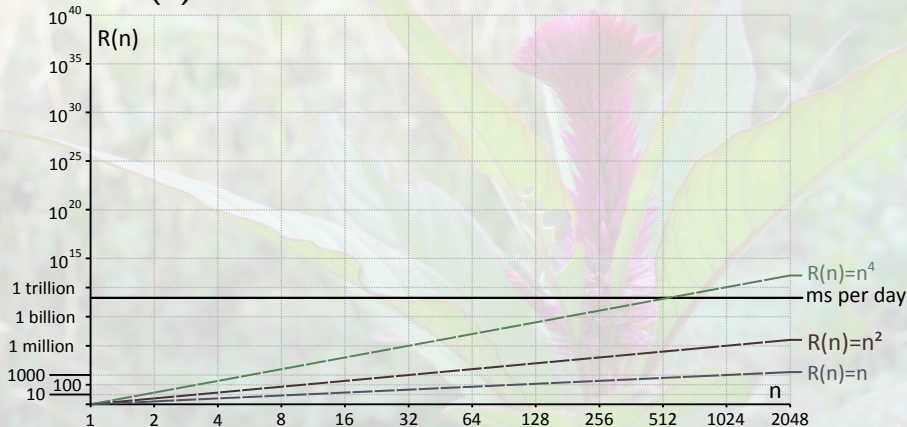
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



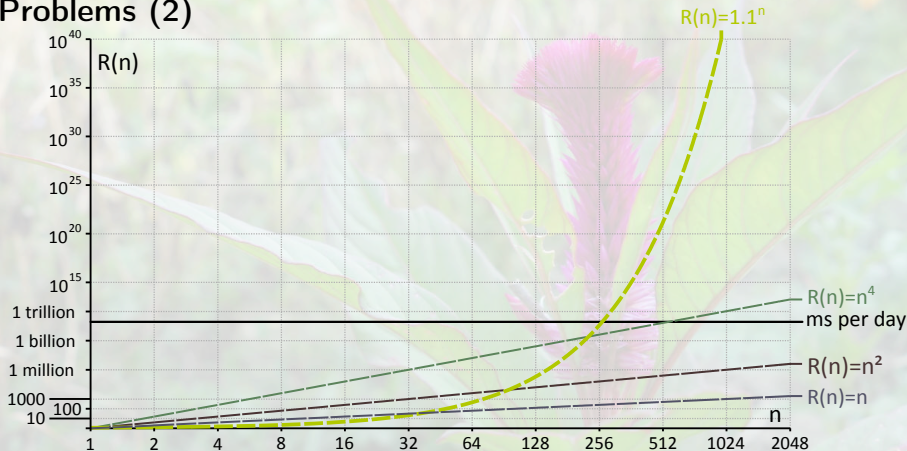
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



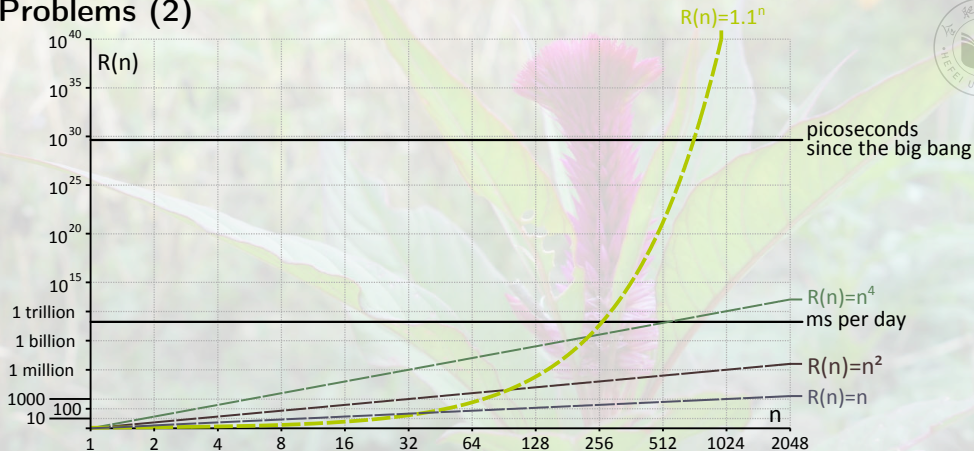
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



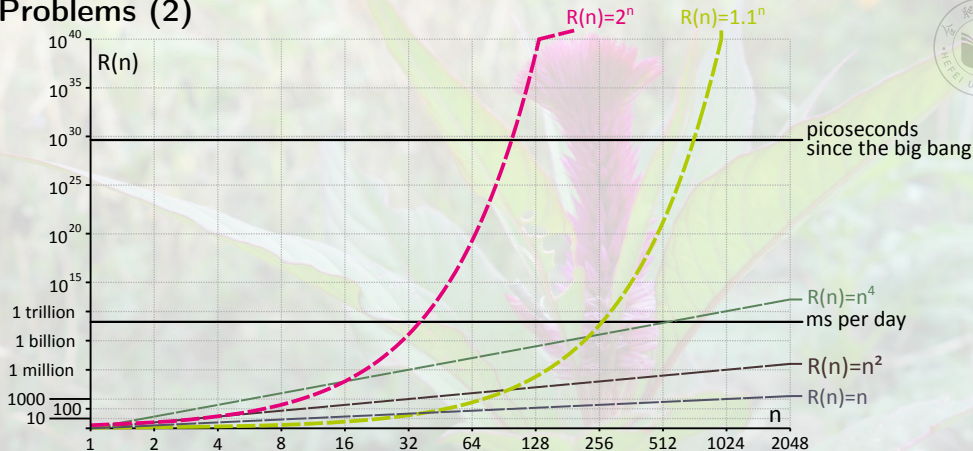
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



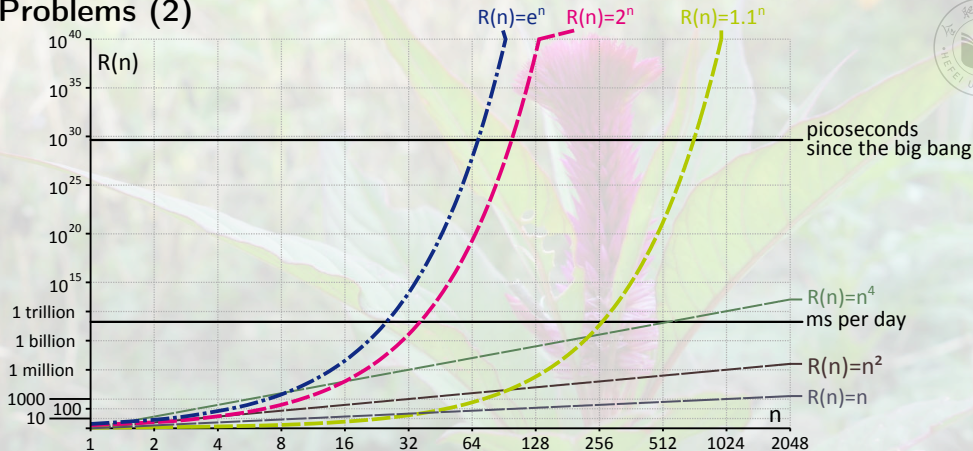
If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .



If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will **grow exponential** with the input size n .

Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)



- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?

**If we want to get the optimal solution,
it will take too long.**

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its worst-case runtime will grow exponential with the input size n .

Hard Problems (2)



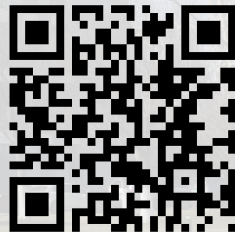
- These are just two examples from a huge family of problems called $\mathcal{NP}$ -hard.
- What does that mean?

If we want to get the optimal solution,
it will **sometimes/often/always** take too long.

If an algorithm guarantees to always find the best-possible solution for an $\mathcal{NP}$ -hard problem, then its **worst-case** runtime will grow exponential with the input size n .



Metaheuristics



Metaheuristics



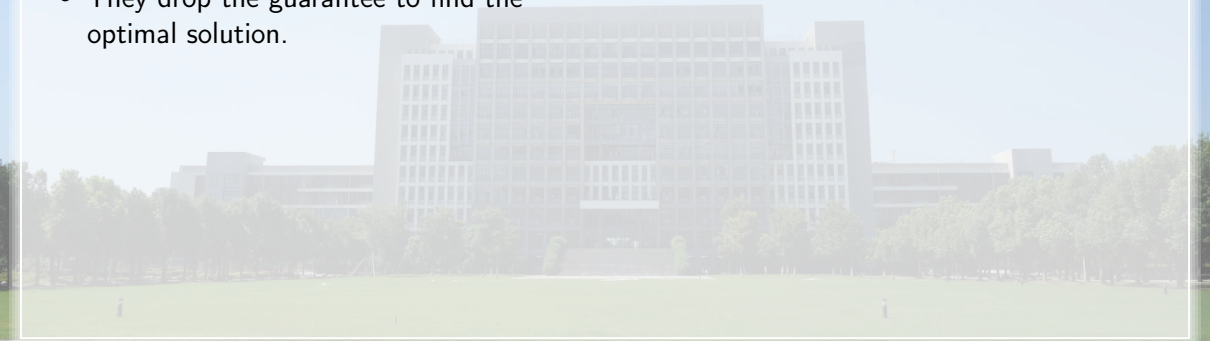
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.



Metaheuristics



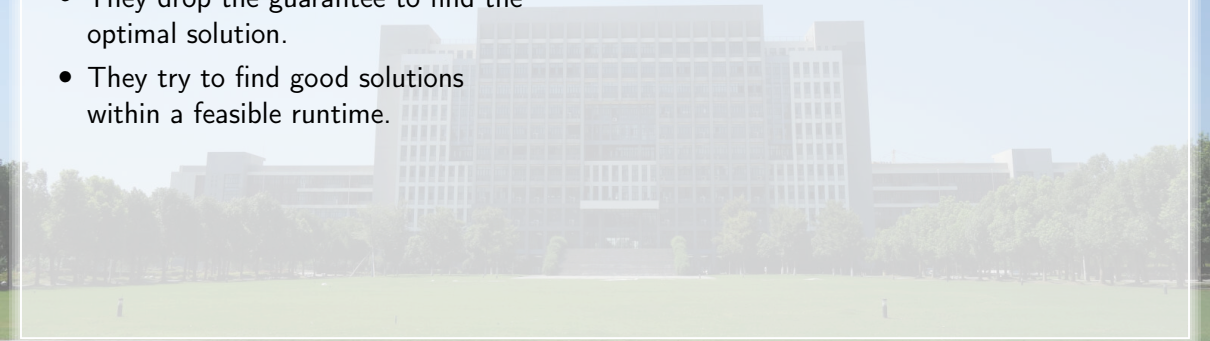
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.



Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.



Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.

A large orange circular arrow pointing clockwise, with a smaller orange arrow pointing upwards from the bottom of the circle towards the text box.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.

Begin with a set $S_0 \subset \mathbb{X}$ of one or multiple randomly sampled solutions

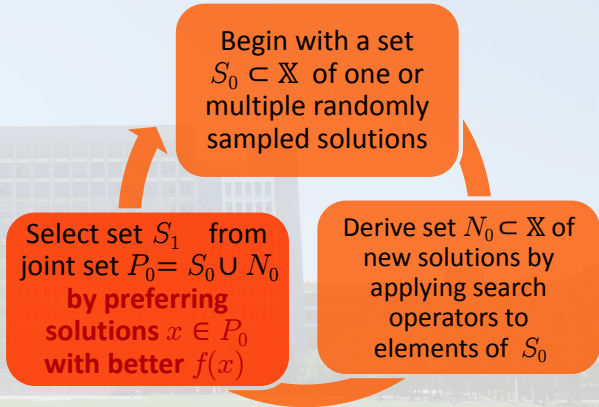
Derive set $N_0 \subset \mathbb{X}$ of new solutions by applying search operators to elements of S_0



Metaheuristics



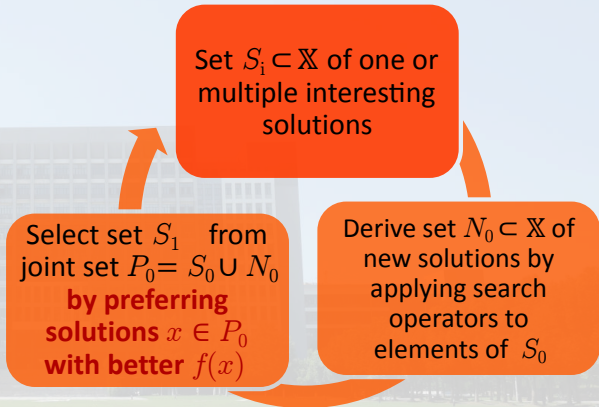
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Metaheuristics



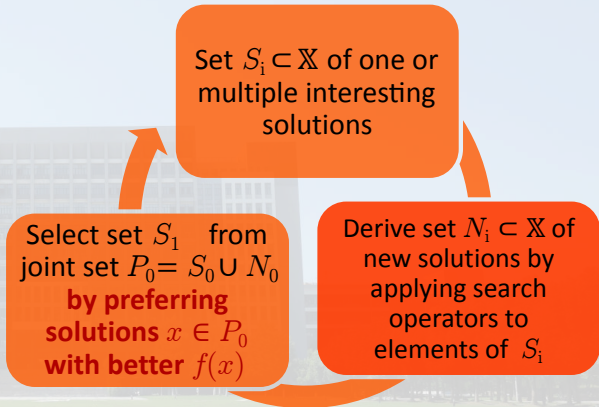
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Metaheuristics



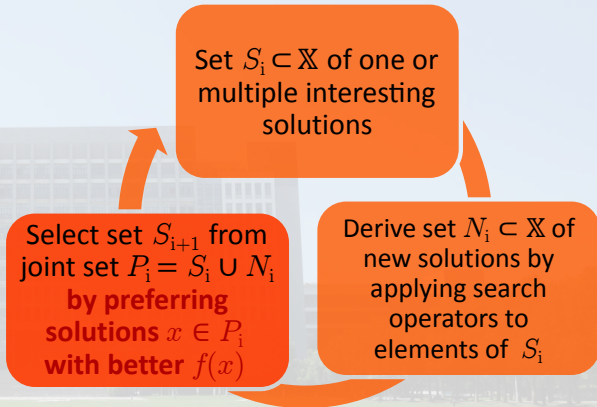
- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Metaheuristics



- Because of this hardness of optimization, metaheuristic algorithms that follow the trial-and-error idea of iterative improvement have emerged.
- They drop the guarantee to find the optimal solution.
- They try to find good solutions within a feasible runtime.
- They (usually) start with random solutions.
- And then roughly follow this cycle.



Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .



Components needed for Metaheuristic Optimization



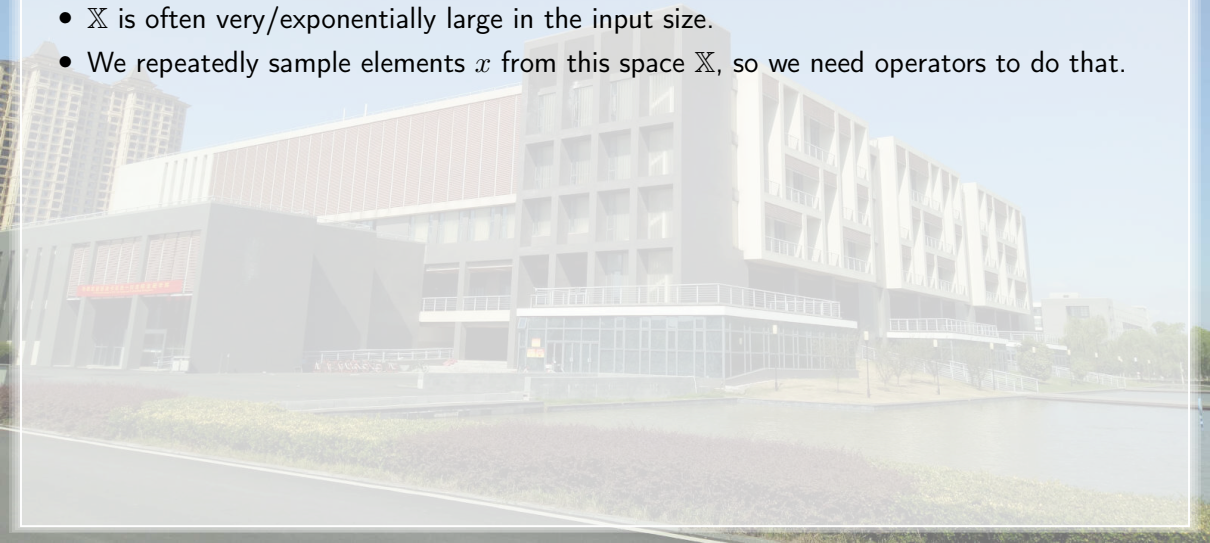
- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.



Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.
- We repeatedly sample elements x from this space $\mathbb{X}$, so we need operators to do that.



Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.
- We repeatedly sample elements x from this space $\mathbb{X}$, so we need operators to do that:
 - a *nullary* operator usually returns a randomly chosen point from $\mathbb{X}$.

Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.
- We repeatedly sample elements x from this space $\mathbb{X}$, so we need operators to do that:
 - a *nullary* operator usually returns a randomly chosen point from $\mathbb{X}$. This is often used to get starting points for the search.

Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.
- We repeatedly sample elements x from this space $\mathbb{X}$, so we need operators to do that:
 - a *nullary* operator usually returns a randomly chosen point from $\mathbb{X}$. This is often used to get starting points for the search.
 - a *unary* operator receives an existing point as input and creates a slightly modified copy.

Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.
- We repeatedly sample elements x from this space $\mathbb{X}$, so we need operators to do that:
 - a *nullary* operator usually returns a randomly chosen point from $\mathbb{X}$. This is often used to get starting points for the search.
 - a *unary* operator receives an existing point as input and creates a slightly modified copy. This is often used to explore the neighborhood of good solutions.

Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.
- We repeatedly sample elements x from this space $\mathbb{X}$, so we need operators to do that:
 - a *nullary* operator usually returns a randomly chosen point from $\mathbb{X}$. This is often used to get starting points for the search.
 - a *unary* operator receives an existing point as input and creates a slightly modified copy. This is often used to explore the neighborhood of good solutions.
 - a *binary* operator receives a two existing points as input and creates a new point with elements from both of these “parents”.

Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.
- We repeatedly sample elements x from this space $\mathbb{X}$, so we need operators to do that:
 - a *nullary* operator usually returns a randomly chosen point from $\mathbb{X}$. This is often used to get starting points for the search.
 - a *unary* operator receives an existing point as input and creates a slightly modified copy. This is often used to explore the neighborhood of good solutions.
 - a *binary* operator receives a two existing points as input and creates a new point with elements from both of these “parents”. This is often used to combine traits from good solutions.

Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.
- We repeatedly sample elements x from this space $\mathbb{X}$, so we need operators to do that:
 - a *nullary* operator usually returns a randomly chosen point from $\mathbb{X}$. This is often used to get starting points for the search.
 - a *unary* operator receives an existing point as input and creates a slightly modified copy. This is often used to explore the neighborhood of good solutions.
 - a *binary* operator receives a two existing points as input and creates a new point with elements from both of these “parents”. This is often used to combine traits from good solutions.
- We evaluate the quality of solutions using the objective function $f : \mathbb{X} \mapsto \mathbb{R}$, which usually is subject to minimization (smaller values are better).

Components needed for Metaheuristic Optimization



- We have a space $\mathbb{X}$ in which we search for potential solutions x .
- $\mathbb{X}$ is often very/exponentially large in the input size.
- We repeatedly sample elements x from this space $\mathbb{X}$, so we need operators to do that:
 - a *nullary* operator usually returns a randomly chosen point from $\mathbb{X}$. This is often used to get starting points for the search.
 - a *unary* operator receives an existing point as input and creates a slightly modified copy. This is often used to explore the neighborhood of good solutions.
 - a *binary* operator receives a two existing points as input and creates a new point with elements from both of these “parents”. This is often used to combine traits from good solutions.
- We evaluate the quality of solutions using the objective function $f : \mathbb{X} \mapsto \mathbb{R}$, which usually is subject to minimization (smaller values are better).
- These are the usual main ingredients of metaheuristic optimization.

Example: RLS and the $(1 + 1)$ EA



- stochastic local search (SLS) with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.



Example: RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)

Example: RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.

procedure $(1 + 1)$ EA($f : \mathbb{X} \mapsto \mathbb{R}$)
 randomly sample x_c from $\mathbb{X}$;

Example: RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
```

Example: RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do
```

Example: RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;
```


Example: RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```

Example: RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then
```

Example: RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;
```

Example: RLS and the $(1 + 1)$ EA



- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

Example: RLS and the $(1 + 1)$ EA

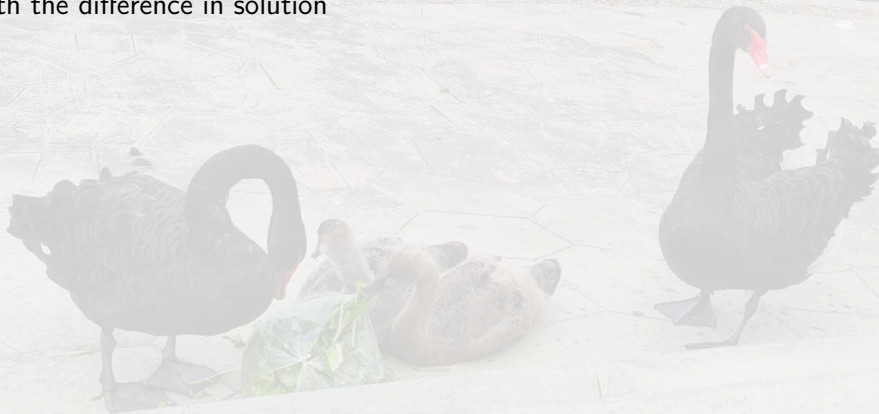


- SLS with $|S_i| = |N_i| = 1$ is the simplest realization of the metaheuristic idea.
- Randomized local search (RLS) and the $(1 + 1)$ evolutionary algorithm ($(1 + 1)$ EA) work according to this pattern^{17,39}.
- They accept the new solution if it is better or equally good compared to the current solution.

```
procedure  $(1 + 1)$  EA( $f : \mathbb{X} \mapsto \mathbb{R}$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
    if  $y_n \leq y_c$  then  $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
  return  $x_c, y_c$ 
```


Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.





Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- Let's start with RLS/(1 + 1) EA and modify it to become SA.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
    randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
    while  $\neg$  terminate do  
         $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
        if  $y_n \leq y_c$  then  
             $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```



Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
```

```
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```

```
    if  $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then  
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```

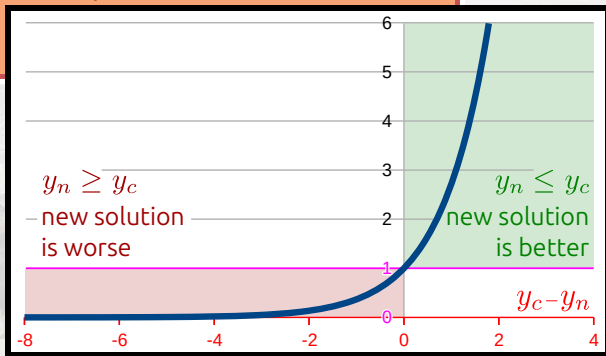
Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
```

```
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
```

```
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then
```





Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.

procedure SA($f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$)

randomly sample x_c from $\mathbb{X}$; $y_c \leftarrow f(x_c)$;

while $\neg$ terminate **do**

$x_n \leftarrow \text{move}(x_c)$; $y_n \leftarrow f(x_n)$;

if $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$ **then** $\triangleright$ always true if $y_n \leq y_c$
 $x_c \leftarrow x_n$; $y_c \leftarrow y_n$;



Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.

procedure SA($f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$)

randomly sample x_c from $\mathbb{X}$; $y_c \leftarrow f(x_c)$;

while $\neg$ terminate **do**

$x_n \leftarrow \text{move}(x_c)$; $y_n \leftarrow f(x_n)$;

if $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$ **then** $\triangleright$ always true if $y_n \leq y_c$
 $x_c \leftarrow x_n$; $y_c \leftarrow y_n$;



Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameter T_0 .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
    randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
    while  $\neg$  terminate do  
         $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
         $T \leftarrow T_0$  ;  
        if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then  $\triangleright$  always true if  $y_n \leq y_c$   
             $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```



Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameter T_0 .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$   
  
     $T \leftarrow T_0$  ;  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ 
```



Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameter T_0 .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0$  ;  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```



Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathcal{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$   
  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathcal{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```




Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .
- SA also remembers best-so-far solution x_B and its objective value y_B , because it could get lost.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ; ▷ preserve best!  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
```



Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .
- SA also remembers best-so-far solution x_B and its objective value y_B , because it could get lost.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )  
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;  
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ; ▷ preserve best!  
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter  
  while  $\neg$  terminate do  
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;  
     $\tau \leftarrow \tau + 1$ ;  
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time  
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$   
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;  
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;
```



Example: Simulated Annealing

- Simulated Annealing (SA)⁷⁰ is a local search that accepts also worsening moves, but with a probability that decreases over time and with the difference in solution quality.
- $\mathfrak{R}_0^1$ be a random number uniformly distributed in $[0, 1)$.
- High temperatures T increase the acceptance probability P of worse solutions, low T reduce it.
- P is regulated by temperature schedule with parameters T_0 and ε .
- SA also remembers best-so-far solution x_B and its objective value y_B , because it could get lost.

```
procedure SA( $f : \mathbb{X} \mapsto \mathbb{R}, T_0, \varepsilon$ )
  randomly sample  $x_c$  from  $\mathbb{X}$ ;  $y_c \leftarrow f(x_c)$ ;
   $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ; ▷ preserve best!
   $\tau \leftarrow 0$ ; ▷  $\tau$  is iteration counter
  while  $\neg$  terminate do
     $x_n \leftarrow \text{move}(x_c)$ ;  $y_n \leftarrow f(x_n)$ ;
     $\tau \leftarrow \tau + 1$ ;
     $T \leftarrow T_0(1 - \varepsilon)^{\tau-1}$ ; ▷  $T$  decreases over time
    if  $\mathfrak{R}_0^1 < e^{\frac{y_c - y_n}{T}}$  then ▷ always true if  $y_n \leq y_c$ 
       $x_c \leftarrow x_n$ ;  $y_c \leftarrow y_n$ ;
      if  $y_c < y_B$  then  $x_B \leftarrow x_c$ ;  $y_B \leftarrow y_c$ ;
  return  $x_B, y_B$ 
```

Types of Metaheuristics

- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).



Types of Metaheuristics

- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search



Types of Metaheuristics

- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.



Types of Metaheuristics

- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
 - **Advantage:** Fast (and sometimes simple)



Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
 - **Advantage:** Fast (and sometimes simple)
 - **Disadvantage:** May get stuck at a local optimum

Types of Metaheuristics

- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods



Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
 - **Advantage:** Less likely to get stuck at local optimum.

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
 - **Advantage:** Less likely to get stuck at local optimum.
 - **Disadvantage:** Slow.

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
- Model-based Methods

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
- Model-based Methods:
 - Maintain a stochastic model M (usually a multivariate probability distribution) of what good solutions could look like, sample set N of $|N| = \lambda \geq 1$ new solutions based on M in each step, then update M .

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
- Model-based Methods:
 - Maintain a stochastic model M (usually a multivariate probability distribution) of what good solutions could look like, sample set N of $|N| = \lambda \geq 1$ new solutions based on M in each step, then update M .
 - **Advantage:** Sometimes really good performance.

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
- Model-based Methods:
 - Maintain a stochastic model M (usually a multivariate probability distribution) of what good solutions could look like, sample set N of $|N| = \lambda \geq 1$ new solutions based on M in each step, then update M .
 - **Advantage:** Sometimes really good performance.
 - **Disadvantage:** Complicated, sampling and model update may be slow.

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
- Model-based Methods:
 - Maintain a stochastic model M (usually a multivariate probability distribution) of what good solutions could look like, sample set N of $|N| = \lambda \geq 1$ new solutions based on M in each step, then update M .
- Hybrid Algorithms

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
- Model-based Methods:
 - Maintain a stochastic model M (usually a multivariate probability distribution) of what good solutions could look like, sample set N of $|N| = \lambda \geq 1$ new solutions based on M in each step, then update M .
- Hybrid Algorithms:
 - Usually combine model-based or population-based approach with local search: Each new solution from model/population is refined with local search.

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS) Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
- Model-based Methods:
 - Maintain a stochastic model M (usually a multivariate probability distribution) of what good solutions could look like, sample set N of $|N| = \lambda \geq 1$ new solutions based on M in each step, then update M .
- Hybrid Algorithms:
 - Usually combine model-based or population-based approach with local search: Each new solution from model/population is refined with local search.
 - **Advantage:** Fast (because local search), good global search (because of population/model)

Types of Metaheuristics



- We can build many interesting algorithms just using such components (and sometimes a bit more problem-specific information).
- stochastic local search (SLS)Local Search:
 - Keep one solution x_c , sample at least one new solution x_n from its neighborhood in each step, move to better solutions as discovered.
- Population-based Methods:
 - Maintain population S of $|S| = \mu \geq 1$ solutions, sample set N of $|N| = \lambda \geq 1$ new solutions in each step by creating modified copies of the elements in S , then update S .
- Model-based Methods:
 - Maintain a stochastic model M (usually a multivariate probability distribution) of what good solutions could look like, sample set N of $|N| = \lambda \geq 1$ new solutions based on M in each step, then update M .
- Hybrid Algorithms:
 - Usually combine model-based or population-based approach with local search: Each new solution from model/population is refined with local search.
 - **Advantage:** Fast (because local search), good global search (because of population/model)
 - **Disadvantage:** usually few.



Discrete Optimization



Black-Box Discrete Optimization

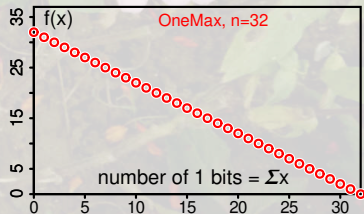


- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96} (total number of 1 bits, very easy, single optimum)



Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, **Linear Harmonic Function (LinHarm)**^{37,39} like , but bits have linearly increasing weights)

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, **Binary Integer (BinInt)**¹²³ (like , but bits have exponentially increasing weights)

Black-Box Discrete Optimization

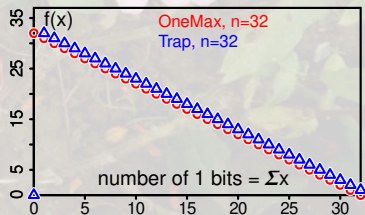


- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142} (number of leading 1 bits: contribution of variable x_i depends on all variables x_j with $j < i$)

Black-Box Discrete Optimization



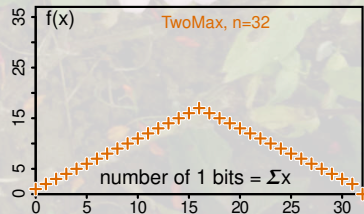
- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100} (, but optimum and worse solution swapped – extremely hard, local search needs exponential runtime)



Black-Box Discrete Optimization



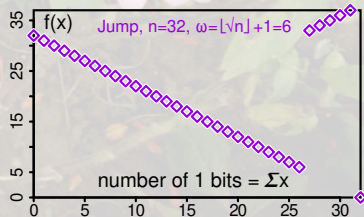
- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126} (two optima, a global and a local one, of same size, local search needs exponential runtime with 50% probability)



Black-Box Discrete Optimization



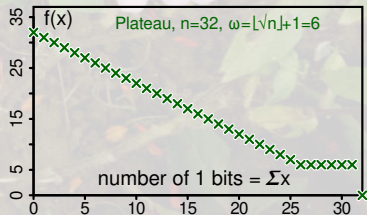
- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46} (like OneMax, but ω bits around optimum are deceptive)



Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ² (like Jump, but bits around optimum are neutral)



Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44} (many interactions between variables)

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷ (place N queens on a chessboard, many interactions between variables)

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷, W-Model^{132,140} (allows you to tune and combine the above features)

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷, W-Model^{132,140}
- **Baseline:** randomized local search (RLS) (unary operator flips 1 bit)

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷, W-Model^{132,140}
- **Baseline:** RLS, $(1 + 1)$ EA (unary operator flips each bit with probability $1/n$), $(1 + 1)$ EA

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷, W-Model^{132,140}
- **Baseline:** RLS, $(1 + 1)$ EA, the Standard Genetic Algorithm (SGA)^{6,27,53,92,94,128} (traditional population-based GA)

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷, W-Model^{132,140}
- **Baseline:** RLS, $(1 + 1)$ EA, the SGA^{6,27,53,92,94,128}, and Tabu Search (TS) (like RLS, but forbid flipping of bits recently flipped to avoid cycles)

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷, W-Model^{132,140}
- **Baseline:** RLS, $(1 + 1)$ EA, the SGA^{6,27,53,92,94,128}, and TS
- **SOTA:** Self-Adjusting $(1 + (\lambda, \lambda))$ GA^{17,32} (linear runtime on)

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷, W-Model^{132,140}
- **Baseline:** RLS, $(1 + 1)$ EA, the SGA^{6,27,53,92,94,128}, and TS
- **SOTA:** Self-Adjusting $(1 + (\lambda, \lambda))$ GA^{17,32}, **Greedy $(2 + 1)$ GA**^{16,17,119,120} (algorithm using binary operator and being twice as fast as any unary-operator only black-box method on)

Black-Box Discrete Optimization



- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷, W-Model^{132,140}
- **Baseline:** RLS, $(1 + 1)$ EA, the SGA^{6,27,53,92,94,128}, and TS
- **SOTA:** Self-Adjusting $(1 + (\lambda, \lambda))$ GA^{17,32}, Greedy $(2 + 1)$ GA^{16,17,119,120}, and the $(1 + 1)$ EA with FFA $((1 + 1)$ FEA)^{138,139,147} (the only algorithm achieving polynomial runtime on , , and).

Black-Box Discrete Optimization

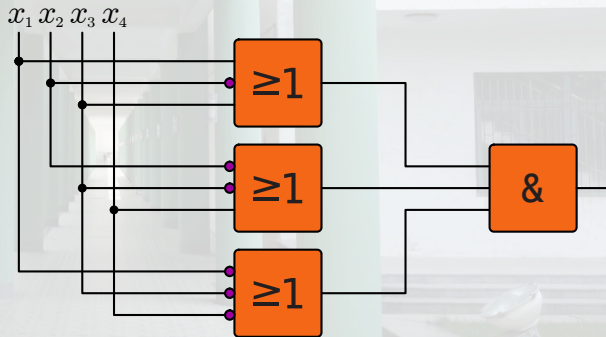


- **Problem:** Search space $\mathbb{X}$ corresponds to the set $\{0, 1\}^n$ of bit strings of length n and nothing is known about objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$.
- Benchmarks here often have the goal to investigate problematic aspects of discrete problems in isolation^{37,147}.
- **Benchmarks:** ^{40,96}, LinHarm^{37,39}, BinInt¹²³, ^{115,142}, ^{39,100}, ^{46,126}, ^{39,46}, ², ^{37,43,44}, ³⁷, W-Model^{132,140}
- **Baseline:** RLS, $(1 + 1)$ EA, the SGA^{6,27,53,92,94,128}, and TS
- **SOTA:** Self-Adjusting $(1 + (\lambda, \lambda))$ GA^{17,32}, Greedy $(2 + 1)$ GA^{16,17,119,120}, and the $(1 + 1)$ FEA^{138,139,147} (the only algorithm achieving polynomial runtime on , , and).



Maximum Satisfiability Problem

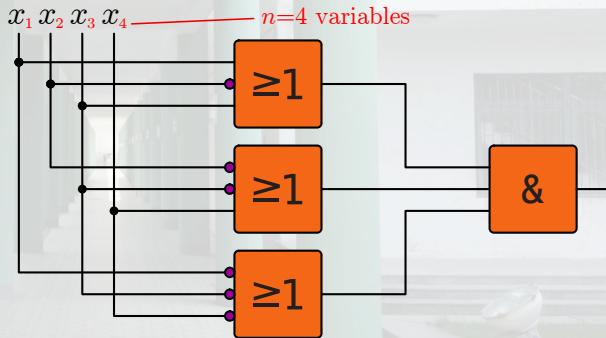
- The goal of the Maximum Satisfiability (MaxSAT)^{23,60} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.





Maximum Satisfiability Problem

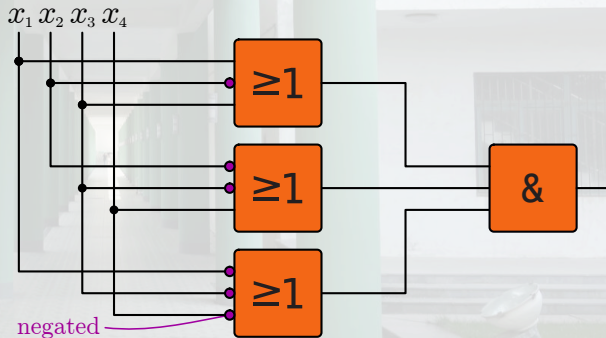
- The goal of the Maximum Satisfiability (MaxSAT)^{23,60} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.





Maximum Satisfiability Problem

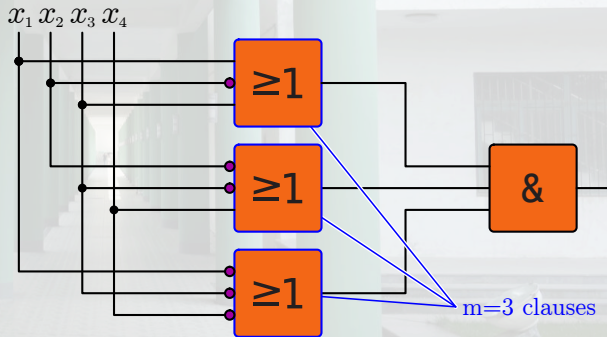
- The goal of the Maximum Satisfiability (MaxSAT)^{23,60} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or **negated** in m OR-clauses, whose results flow into one AND-clause.





Maximum Satisfiability Problem

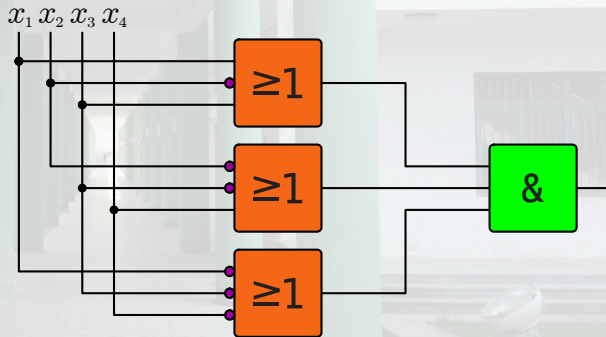
- The goal of the Maximum Satisfiability (MaxSAT)^{23,60} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.





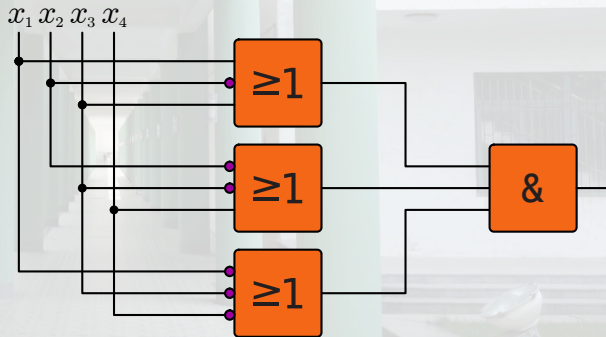
Maximum Satisfiability Problem

- The goal of the Maximum Satisfiability (MaxSAT)^{23,60} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.



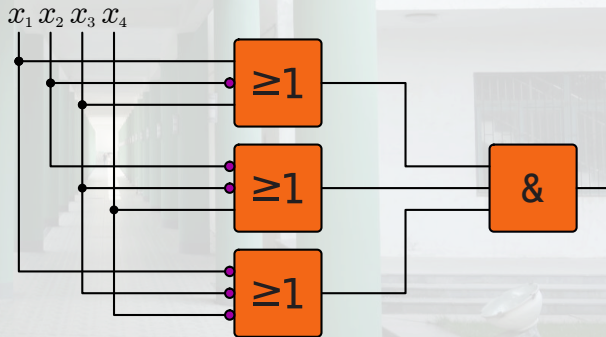
Maximum Satisfiability Problem

- The goal of the Maximum Satisfiability (MaxSAT)^{23,60} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.
- $\mathbb{X}$ is the set of all possible bit strings of length n .



Maximum Satisfiability Problem

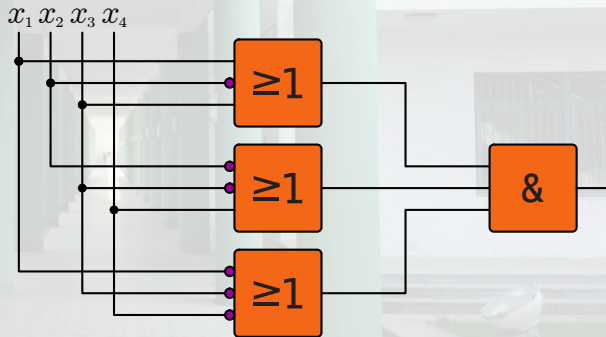
- The goal of the Maximum Satisfiability (MaxSAT)^{23,60} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.
- $\mathbb{X}$ is the set of all possible bit strings of length n .
- The objective function $f : \mathbb{X} \mapsto \mathbb{R}$ is the number of unsatisfied OR-clauses.





Maximum Satisfiability Problem

- The goal of the Maximum Satisfiability (MaxSAT)^{23,60} problem is to find a setting of n variables that makes a Boolean formula F become True. The variables appear directly or negated in m OR-clauses, whose results flow into one AND-clause.
- $\mathbb{X}$ is the set of all possible bit strings of length n .
- The objective function $f : \mathbb{X} \mapsto \mathbb{R}$ is the number of unsatisfied OR-clauses.
- If the problem can be solved, then the optimum $x^* \in \mathbb{X}$ has $f(x^*) = 0$, i.e., all clauses satisfied, i.e., $F(x^*) = \text{True}$.



Maximum Satisfiability Problem



- **Problem:** Find a bit string that makes as many OR-clauses True as possible.

Maximum Satisfiability Problem



- **Problem:** Find a bit string that makes as many OR-clauses True as possible.
- The so-called phase transition region with $\frac{m}{n} \approx 4.26$, the average instance hardness for stochastic local search algorithms is maximal ^{34,35,62}

Maximum Satisfiability Problem



- **Problem:** Find a bit string that makes as many OR-clauses True as possible.
- The so-called phase transition region with $\frac{m}{n} \approx 4.26$, the average instance hardness for stochastic local search algorithms is maximal ^{34,35,62}
- **Benchmarks:** SATLIB^{61,62} provides satisfiable benchmark instances from the phase transition region (i.e., the hardest type of instances) for different scales $n \in 20 \cup \{25i \mid i \in 2..10\}$.
- **Baselines:** WalkSAT ¹¹⁸

Maximum Satisfiability Problem



- **Problem:** Find a bit string that makes as many OR-clauses True as possible.
- The so-called phase transition region with $\frac{m}{n} \approx 4.26$, the average instance hardness for stochastic local search algorithms is maximal ^{34,35,62}
- **Benchmarks:** SATLIB^{61,62} provides satisfiable benchmark instances from the phase transition region (i.e., the hardest type of instances) for different scales $n \in 20 \cup \{25i \mid i \in 2..10\}$.
- **Baselines:** WalkSAT ¹¹⁸

Maximum Satisfiability Problem



- **Problem:** Find a bit string that makes as many OR-clauses True as possible.
- The so-called phase transition region with $\frac{m}{n} \approx 4.26$, the average instance hardness for stochastic local search algorithms is maximal ^{34,35,62}
- **Benchmarks:** SATLIB^{61,62} provides satisfiable benchmark instances from the phase transition region (i.e., the hardest type of instances) for different scales $n \in 20 \cup \{25i \mid i \in 2..10\}$.
- **Baselines:** WalkSAT ¹¹⁸ and the baselines for discrete optimization.
- **SOTA:**
 - For smaller-scale instances: branch and bound (B&B) algorithms like in⁸⁰.

Maximum Satisfiability Problem



- **Problem:** Find a bit string that makes as many OR-clauses True as possible.
- The so-called phase transition region with $\frac{m}{n} \approx 4.26$, the average instance hardness for stochastic local search algorithms is maximal ^{34,35,62}
- **Benchmarks:** SATLIB^{61,62} provides satisfiable benchmark instances from the phase transition region (i.e., the hardest type of instances) for different scales $n \in 20 \cup \{25i \mid i \in 2..10\}$.
- **Baselines:** WalkSAT ¹¹⁸ and the baselines for discrete optimization.
- **SOTA:**
 - For smaller-scale instances: branch and bound (B&B) algorithms like in ⁸⁰.
 - Encoding-based methods can deal with even much larger instances¹⁰⁵.

Maximum Satisfiability Problem



- **Problem:** Find a bit string that makes as many OR-clauses True as possible.
- The so-called phase transition region with $\frac{m}{n} \approx 4.26$, the average instance hardness for stochastic local search algorithms is maximal ^{34,35,62}
- **Benchmarks:** SATLIB^{61,62} provides satisfiable benchmark instances from the phase transition region (i.e., the hardest type of instances) for different scales $n \in 20 \cup \{25i \mid i \in 2..10\}$.
- **Baselines:** WalkSAT ¹¹⁸ and the baselines for discrete optimization.
- **SOTA:**
 - For smaller-scale instances: branch and bound (B&B) algorithms like in⁸⁰.
 - Encoding-based methods can deal with even much larger instances¹⁰⁵.
 - The SOTA for incomplete solvers⁵⁴ comprises local search algorithms, often based on ideas dating back to WalkSAT, such as AMLS⁸⁸ or NuWLS²².

Integer Set Division Problem



- Problem:

Integer Set Division Problem



- Problem:
- Benchmarks:

Integer Set Division Problem



- Problem:
- Benchmarks:
- Baselines:

Integer Set Division Problem



- Problem:
- Benchmarks:
- Baselines:
- SOTA:

Low Autocorrelation Binary Sequences

- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.



Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .
- **Baselines:** see discrete optimization

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .
- **Baselines:** see discrete optimization and for small n a GA⁹³

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .
- **Baselines:** see discrete optimization and for small n a GA⁹³ or exhaustive enumeration⁹¹

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .
- **Baselines:** see discrete optimization and for small n a GA⁹³ or exhaustive enumeration⁹¹ and the (outdated) online repository¹².

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .
- **Baselines:** see discrete optimization and for small n a GA⁹³ or exhaustive enumeration⁹¹ and the (outdated) online repository¹².
- **SOTA:**

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .
- **Baselines:** see discrete optimization and for small n a GA⁹³ or exhaustive enumeration⁹¹ and the (outdated) online repository¹².
- **SOTA:**
 - the algorithm by Dimitrov [29]

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .
- **Baselines:** see discrete optimization and for small n a GA⁹³ or exhaustive enumeration⁹¹ and the (outdated) online repository¹².
- **SOTA:**
 - the algorithm by Dimitrov [29],
 - the MA by Mow, Du, and Wu [95]

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .
- **Baselines:** see discrete optimization and for small n a GA⁹³ or exhaustive enumeration⁹¹ and the (outdated) online repository¹².
- **SOTA:**
 - the algorithm by Dimitrov [29],
 - the MA by Mow, Du, and Wu [95],
 - the xLastovka-family of methods by Bošković and Brest [11, 14]

Low Autocorrelation Binary Sequences



- **Problem:** Low-Autocorrelation Binary Sequences (LABS) are bit strings of length n whose autocorrelation is as low as possible.
- This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat.
- The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and can be defined as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$.
- Such sequences are needed for communication engineering^{13,95}, cryptography⁹⁵, as well as modulation pulses in radar and sonar ranging¹⁰².
- **Benchmarks:** The objective function for different values of n .
- **Baselines:** see discrete optimization and for small n a GA⁹³ or exhaustive enumeration⁹¹ and the (outdated) online repository¹².
- **SOTA:**
 - the algorithm by Dimitrov [29],
 - the MA by Mow, Du, and Wu [95],
 - the xLastovka-family of methods by Bošković and Brest [11, 14], and
 - the local search by Dimitrov, Baitcheva, and Nikolov [30].



(Multi-)Permutation Problems



Black-Box Permutation Problems



- **Problem:** The search space $\mathbb{X}$ corresponds to the set of all permutations of the first n natural numbers.

Black-Box Permutation Problems



- **Problem:** The search space $\mathbb{X}$ corresponds to the set of all permutations of the first n natural numbers.
- **Baselines:**
 - RLS with a unary operator that swaps two elements

Black-Box Permutation Problems



- **Problem:** The search space $\mathbb{X}$ corresponds to the set of all permutations of the first n natural numbers.
- **Baselines:**
 - RLS with a unary operator that swaps two elements,
 - RLS with a unary operator that swaps a binomially distributed number of elements (similar to the $(1 + 1)$ EA for permutations)

Black-Box Permutation Problems



- **Problem:** The search space $\mathbb{X}$ corresponds to the set of all permutations of the first n natural numbers.
- **Baselines:**
 - RLS with a unary operator that swaps two elements,
 - RLS with a unary operator that swaps a binomially distributed number of elements (similar to the $(1 + 1)$ EA for permutations),
 - SA with the same operators and suitable exponential temperature schedule setups

Black-Box Permutation Problems



- **Problem:** The search space $\mathbb{X}$ corresponds to the set of all permutations of the first n natural numbers.
- **Baselines:**
 - RLS with a unary operator that swaps two elements,
 - RLS with a unary operator that swaps a binomially distributed number of elements (similar to the $(1 + 1)$ EA for permutations),
 - SA with the same operators and suitable exponential temperature schedule setups,
 - TS with the same operator that forbids moving values to recently-used indices

Black-Box Permutation Problems



- **Problem:** The search space $\mathbb{X}$ corresponds to the set of all permutations of the first n natural numbers.
- **Baselines:**
 - RLS with a unary operator that swaps two elements,
 - RLS with a unary operator that swaps a binomially distributed number of elements (similar to the $(1 + 1)$ EA for permutations),
 - SA with the same operators and suitable exponential temperature schedule setups,
 - TS with the same operator that forbids moving values to recently-used indices, and a
 - $(\mu + \lambda)$ EA with these operators and maybe Alternating Position Crossover (AP)^{75,76}.

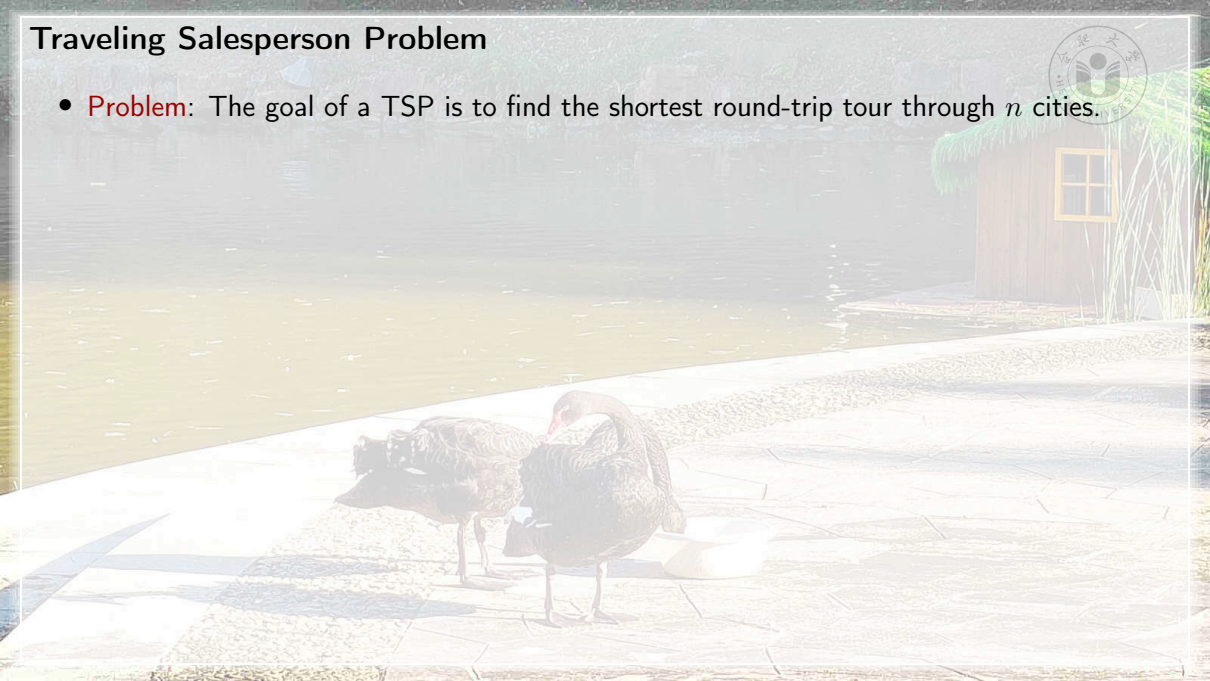
Black-Box Permutation Problems



- **Problem:** The search space $\mathbb{X}$ corresponds to the set of all permutations of the first n natural numbers.
- **Baselines:**
 - RLS with a unary operator that swaps two elements,
 - RLS with a unary operator that swaps a binomially distributed number of elements (similar to the $(1 + 1)$ EA for permutations),
 - SA with the same operators and suitable exponential temperature schedule setups,
 - TS with the same operator that forbids moving values to recently-used indices, and a
 - $(\mu + \lambda)$ EA with these operators and maybe Alternating Position Crossover (AP)^{75,76}.
- **SOTA:**

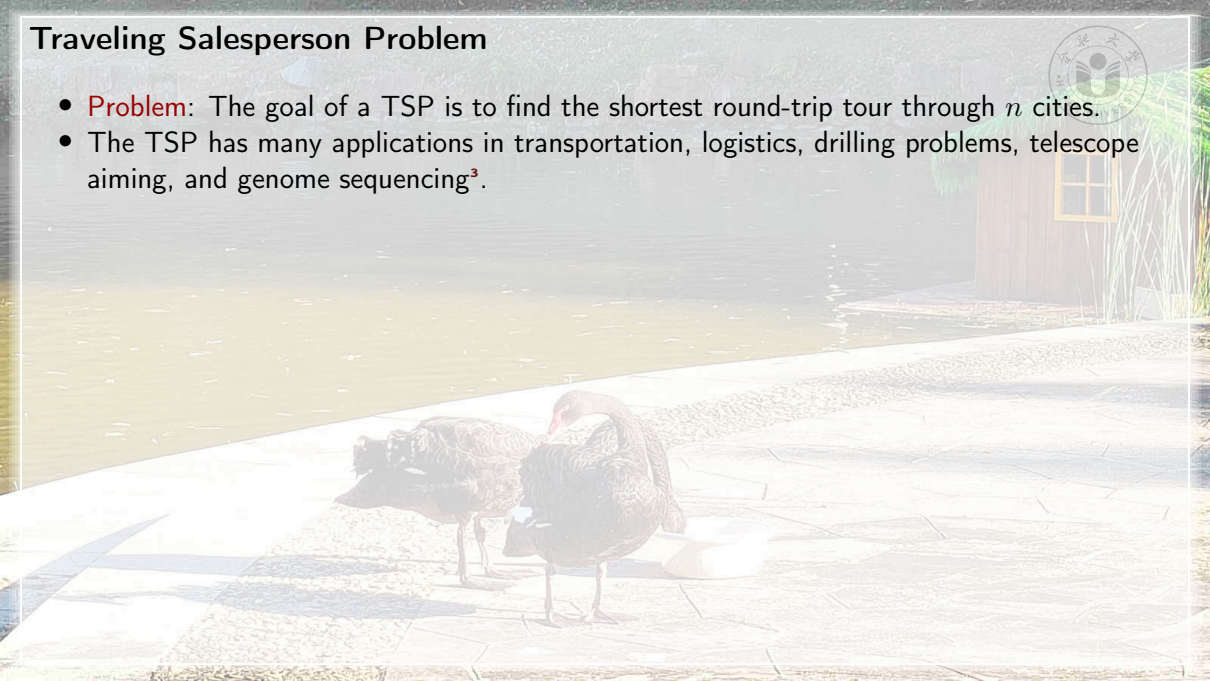
Traveling Salesperson Problem

- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.



Traveling Salesperson Problem

- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.



Traveling Salesperson Problem



- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114} with 110 symmetric instances with scales $n \in 14..85\,900$ and 19 asymmetric instances with scales $n \in 17..443$.



Traveling Salesperson Problem



- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114}, the
 - instances of the 8th DIMACS challenge⁶⁷⁻⁶⁹, with instances with up to 10 000 000 cities.



Traveling Salesperson Problem



- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114}, the
 - instances of the 8th DIMACS challenge⁶⁷⁻⁶⁹, the
 - *World TSP* repository²⁶ with many real-world geographical instances, including one with all 1 904 711 registered populated places of the Earth at some point in time in the 2000s



Traveling Salesperson Problem



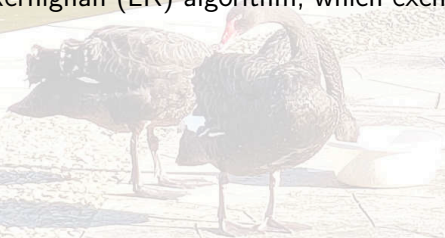
- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114}, the
 - instances of the 8th DIMACS challenge⁶⁷⁻⁶⁹, the
 - *World TSP* repository²⁶, and the
 - TSP Suite, which offers an integrated Java environment for benchmarking algorithms on the TSPLib instances¹³¹.



Traveling Salesperson Problem



- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114}, the
 - instances of the 8th DIMACS challenge⁶⁷⁻⁶⁹, the
 - *World TSP* repository²⁶, and the
 - TSP Suite¹³¹.
- **Baselines:** the Lin-Kernighan (LK) algorithm, which exchanges variable number of edges in a tour.⁸⁴



Traveling Salesperson Problem



- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114}, the
 - instances of the 8th DIMACS challenge⁶⁷⁻⁶⁹, the
 - *World TSP* repository²⁶, and the
 - TSP Suite¹³¹.
- **Baselines:** the LK algorithm⁸⁴
- **SOTA:**
 - the Lin-Kernighan Helsgaun (LKH) algorithm building on top of the LK algorithm. LKH algorithm^{58,59}

Traveling Salesperson Problem



- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114}, the
 - instances of the 8th DIMACS challenge⁶⁷⁻⁶⁹, the
 - *World TSP* repository²⁶, and the
 - TSP Suite¹³¹.
- **Baselines:** the LK algorithm⁸⁴
- **SOTA:**
 - the LKH algorithm^{58,59},
 - an EA with edge assembly crossover (EAX) as binary operator⁹⁷⁻⁹⁹
 - an MA with Generalized Partition Crossover (GPX) as binary operator¹⁴³

Traveling Salesperson Problem



- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114}, the
 - instances of the 8th DIMACS challenge⁶⁷⁻⁶⁹, the
 - *World TSP* repository²⁶, and the
 - TSP Suite¹³¹.
- **Baselines:** the LK algorithm⁸⁴
- **SOTA:**
 - the LKH algorithm^{58,59},
 - EAX⁹⁷⁻⁹⁹,
 - an MA with Generalized Partition Crossover (GPX) as binary operator¹⁴³

Traveling Salesperson Problem



- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114}, the
 - instances of the 8th DIMACS challenge⁶⁷⁻⁶⁹, the
 - *World TSP* repository²⁶, and the
 - TSP Suite¹³¹.
- **Baselines:** the LK algorithm⁸⁴
- **SOTA:**
 - the LKH algorithm^{58,59},
 - EAX⁹⁷⁻⁹⁹,
 - GPX¹⁴³, in particular
 - LKH run in an MA-fashion with GPX as binary operator¹⁴³

Traveling Salesperson Problem



- **Problem:** The goal of a TSP is to find the shortest round-trip tour through n cities.
- The TSP has many applications in transportation, logistics, drilling problems, telescope aiming, and genome sequencing³.
- **Benchmarks:**
 - The TSPLib^{113,114}, the
 - instances of the 8th DIMACS challenge^{67–69}, the
 - *World TSP* repository²⁶, and the
 - TSP Suite¹³¹.
- **Baselines:** the LK algorithm⁸⁴
- **SOTA:**
 - the LKH algorithm^{58,59},
 - EAX^{97–99},
 - GPX¹⁴³, in particular
 - LKH + GPX¹⁴³, and
 - Concorde, an exact solver for the TSP^{3,24,25}, which could even solve the largest instance of TSPLib (with 85 900 cities) to optimality!

Quadratic Assignment Problem



- Problem:

Quadratic Assignment Problem



- Problem:
- Benchmarks:

Quadratic Assignment Problem



- Problem:
- Benchmarks:
- Baselines:

Quadratic Assignment Problem

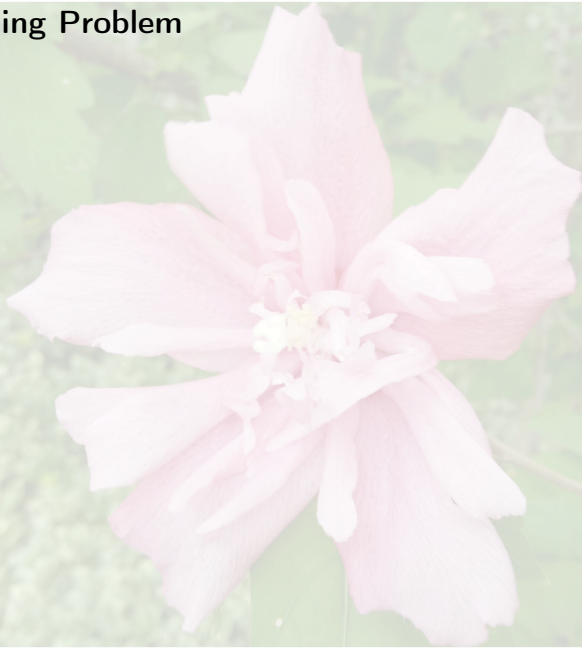


- Problem:
- Benchmarks:
- Baselines:
- SOTA:

Job Shop Scheduling Problem



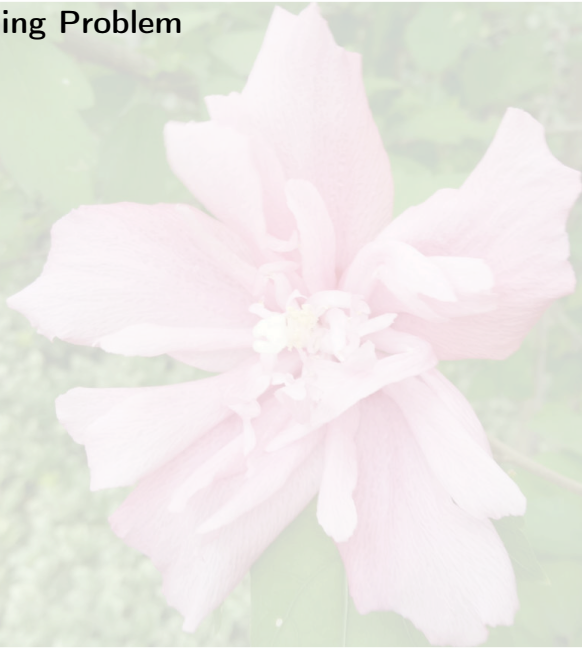
- Problem:



Job Shop Scheduling Problem



- Problem:
- Benchmarks:



Job Shop Scheduling Problem



- Problem:
- Benchmarks:
- Baselines:



Job Shop Scheduling Problem



- Problem:
- Benchmarks:
- Baselines:
- SOTA:



2D Bin Packing Problem



- Problem:



2D Bin Packing Problem



- Problem:
- Benchmarks:



2D Bin Packing Problem



- Problem:
- Benchmarks:
- Baselines:



2D Bin Packing Problem



- Problem:
- Benchmarks:
- Baselines:
- SOTA:



Traveling Tournament Problem



- Problem:

Traveling Tournament Problem



- Problem:
- Benchmarks:

Traveling Tournament Problem



- Problem:
- Benchmarks:
- Baselines:

Traveling Tournament Problem



- Problem:
- Benchmarks:
- Baselines:
- SOTA:

Vehicle Routing Problems



- Problem:

Vehicle Routing Problems



- Problem:
- Benchmarks:

Vehicle Routing Problems



- Problem:
- Benchmarks:
- Baselines:

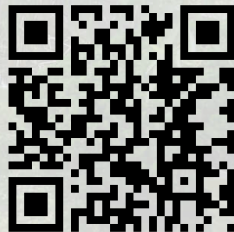
Vehicle Routing Problems



- Problem:
- Benchmarks:
- Baselines:
- SOTA:



Continuous Optimization



Black-Box Continuous Optimization



- Problem:

Black-Box Continuous Optimization



- Problem:
- Benchmarks:

Black-Box Continuous Optimization



- Problem:
- Benchmarks:
- Baselines:

Black-Box Continuous Optimization



- Problem:
- Benchmarks:
- Baselines:
- SOTA:



Multi-Objective Optimization / Noisy Optimization



Black-Box Multi-Objective Problems



- Problem:

Black-Box Multi-Objective Problems



- Problem:
- Benchmarks:

Black-Box Multi-Objective Problems



- Problem:
- Benchmarks:
- Baselines:

Black-Box Multi-Objective Problems



- Problem:
- Benchmarks:
- Baselines:
- SOTA:

Black-Box Noisy-Objective Problems



- Problem:

Black-Box Noisy-Objective Problems



- Problem:
- Benchmarks:

Black-Box Noisy-Objective Problems



- Problem:
- Benchmarks:
- Baselines:

Black-Box Noisy-Objective Problems



- Problem:
- Benchmarks:
- Baselines:
- SOTA:

Material Flow Control for Make-to-Stock with Reorder Points



- Problem:

Material Flow Control for Make-to-Stock with Reorder Points



- Problem:
- Benchmarks:

Material Flow Control for Make-to-Stock with Reorder Points



- Problem:
- Benchmarks:
- Baselines:

Material Flow Control for Make-to-Stock with Reorder Points



- Problem:
- Benchmarks:
- Baselines:
- SOTA:



Tree-based Search Spaces



Equation and Algorithm Synthesis



- Problem:



Equation and Algorithm Synthesis



- Problem:
- Benchmarks:

Equation and Algorithm Synthesis



- Problem:
- Benchmarks:
- Baselines:

Equation and Algorithm Synthesis



- Problem:
- Benchmarks:
- Baselines:
- SOTA:



Summary



Summary



- Today, I gave you a brief introduction into optimization.

Summary



- Today, I gave you a brief introduction into optimization.
- You learned roughly what it is.

Summary



- Today, I gave you a brief introduction into optimization.
- You learned roughly what it is.
- Then we looked at different examples for optimization problems.



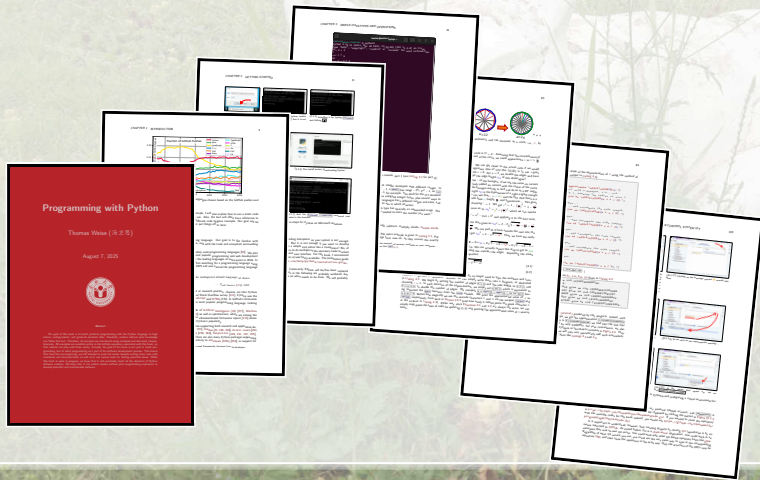
Advertisement



Programming with Python



We have a freely available course book on *Programming with Python* at <https://thomasweise.github.io/programmingWithPython>, with focus on practical software development using the Python ecosystem of tools¹³⁰.



Databases



We have a freely available course book on *Databases* at <https://thomasweise.github.io/databases>, with actual practical examples using a real database management system (DBMS)¹²⁷.



Metaheuristic Optimization in Python: moptipy



We offer moptipy¹³⁷, a mature open source Python package for metaheuristic optimization, which implements several algorithms, can run self-documenting experiments in parallel and in a distributed fashion, and offers statistical evaluation tools.

Table of Contents

- moptipy: Metaheuristic Optimization in Python
- 1. Introduction
- 2. Installation
- 3. How-To
- 3.1. How to Apply 1 Optimization Algorithm Once to 1 Problem Instance
- 3.2. How to Run a Series of Experiments
- 3.3. How to Solve an Optimization Problem
 - 3.3.1. Define a New Problem Type
 - 3.3.2. Define a New Algorithm
 - 3.3.3. Applying an Own Algorithm to an Own Problem
- 4. Implemented Algorithms, Search Spaces, and Problems
 - 4.1. Implemented Algorithms
 - 4.1.1. Single-

moptipy: Metaheuristic Optimization in Python

- Index
- Module Index
- Search Page

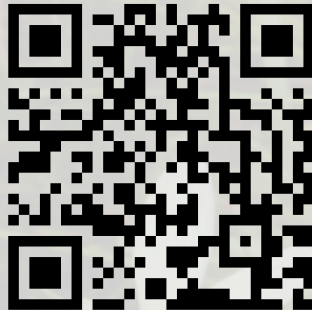
1. Introduction

moptipy is a library with implementations of metaheuristic optimization methods in Python 3.12 that also offers an environment for replicable experiments. The framework, `algorithms`, and the library design are accompanied by the book `Optimization Algorithms`. The library is structured with performance, ease-of-use, and generality in mind, but also based on an educational and research perspective. It is therefore hopefully suitable for practical industrial applications, scientific research, and for students who are just entering the field of metaheuristic optimization.

Metaheuristic optimization algorithms are methods for solving hard problems. moptipy provides an `API`, several `algorithm implementations`, as well as `experiment execution and evaluation` facilities for metaheuristics.

A metaheuristic algorithm can be a black-box method, which can solve problems without deeper knowledge about their nature. Such a black-box algorithm only requires methods to `create` and `modify` points in the `search space` and to `evaluate their quality`. With these operations, it will try to discover better solutions step-by-step. Black-box metaheuristics are very general and can be adapted to almost any optimization problem. They allow us also to do it.

Progress plots are implemented in the module `moptipy.evaluation.plot_progress`.





谢谢你们！
Thank you!
Vielen Dank!



References I



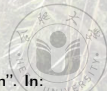
- [1] Hernán E. Aguirre and Keiki Takadama, eds. *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyoto, Japan. New York, NY, USA: Association for Computing Machinery (ACM), 2018. ISBN: **978-1-4503-5618-3**. doi:**10.1145/3205455**.
- [2] Denis Antipov and Benjamin Doerr. "Precise Runtime Analysis for Plateaus". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 2. Sept. 8–12, 2018, Coimbra, Portugal. Ed. by Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley. Vol. 11102 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, pp. 117–128. ISSN: **0302-9743**. ISBN: **978-3-319-99258-7**. doi:**10.1007/978-3-319-99259-4_10** (cit. on pp. **217–228, 361**).
- [3] David Lee Applegate, Robert E. Bixby, Vašek Chvátal, and William John Cook. *The Traveling Salesman Problem: A Computational Study*. 2nd ed. Vol. 17 of Princeton Series in Applied Mathematics. Princeton, NJ, USA: Princeton University Press, 2007. ISBN: **978-0-691-12993-8**. This book also discusses the Concorde TSP solver. (Cit. on pp. **121, 270–281, 356, 363**).
- [4] Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley, eds. *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 2. Sept. 8–12, 2018, Coimbra, Portugal. Vol. 11102 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018. ISSN: **0302-9743**. ISBN: **978-3-319-99258-7**. doi:**10.1007/978-3-319-99259-4**.
- [5] Paul Gustav Heinrich Bachmann. *Die Analytische Zahlentheorie / Dargestellt von Paul Bachmann*. Vol. Zweiter Theil of Zahlentheorie: Versuch einer Gesamtdarstellung dieser Wissenschaft in ihren Haupttheilen. Leipzig, Sachsen, Germany: B. G. Teubner, 1894. ISBN: **978-1-4181-6963-3**. URL: <http://gallica.bnf.fr/ark:/12148/bpt6k994750> (visited on 2023-12-13) (cit. on p. **365**).
- [6] Thomas Bäck, David B. Fogel, and Zbigniew "Zbyszek" Michalewicz, eds. *Handbook of Evolutionary Computation*. Bristol, England, UK: IOP Publishing Ltd and Oxford, Oxfordshire, England, UK: Oxford University Press, 1997. ISBN: **978-0-7503-0392-7** (cit. on pp. **223–228, 357, 362**).
- [7] Tim Berners-Lee. *Re: Qualifiers on Hypertext links...* Geneva, Switzerland: World Wide Web project, European Organization for Nuclear Research (CERN) and Newsgroups: alt.hypertext, Aug. 6, 1991. URL: <https://www.w3.org/People/Berners-Lee/1991/08/art-6484.txt> (visited on 2025-02-05) (cit. on p. **363**).

References II



- [8] Tim Berners-Lee, Larry Masinter, and Mark P. McCahill. *Uniform Resource Locators (URL)*. Request for Comments (RFC) 1738. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Dec. 1994. URL: <https://www.ietf.org/rfc/rfc1738.txt> (visited on 2025-02-05) (cit. on p. 363).
- [9] Jacek Błażewicz, Wolfgang Domschke, and Erwin Pesch. "The Job Shop Scheduling Problem: Conventional and New Solution Techniques". *European Journal of Operational Research* 93(1):1–33, Aug. 1996. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(95)00362-2 (cit. on p. 358).
- [10] Joshua Bloch. *Effective Java*. Reading, MA, USA: Addison-Wesley Professional, May 2008. ISBN: 978-0-321-35668-0 (cit. on p. 358).
- [11] Borko Bošković and Janez Brest. "Computational Search of Long Skew-Symmetric Binary Sequences with High Merit Factors". *MENDEL – Soft Computing Journal* 28(2):17–24, Dec. 2022. Brno, Czech Republic: Brno University of Technology (Vysoké učení technické v Brně). ISSN: 1803-3814. doi:10.13164/mendel.2022.2.017 (cit. on pp. 248–261, 359).
- [12] Borko Bošković, Franc Brglez, and Janez Brest. *A GitHub Archive for Solvers and Solutions of the labs Problem*. San Francisco, CA, USA: GitHub Inc, Jan. 2016. URL: https://github.com/borkob/git_labs (visited on 2026-09-20) (cit. on pp. 248–261).
- [13] Borko Bošković, Franc Brglez, and Janez Brest. "Low-Autocorrelation Binary Sequences: On Improved Merit Factors and Runtime Predictions to Achieve Them". *Applied Soft Computing (ASOC)* 56:262–285, July 2017. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/J.ASOC.2017.02.024 (cit. on pp. 248–261, 359).
- [14] Janez Brest and Borko Bošković. "A Heuristic Algorithm for a Low Autocorrelation Binary Sequence Problem with Odd Length and High Merit Factor". *IEEE Access* 6:4127–4134, Jan. 2018. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 2169-3536. doi:10.1109/ACCESS.2018.2789916 (cit. on pp. 248–261, 359).
- [15] Rainer E. Burkard, Eranda Čela, Panos Miltiades Pardalos, and Leonidas S. Pitsoulis. "The Quadratic Assignment Problem". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1713–1809. ISBN: 978-1-4613-7987-4. doi:10.1007/978-1-4613-0303-9_27 (cit. on p. 361).

References III



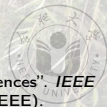
- [16] Eduardo Carvalho Pinto and Carola Doerr. "A Simple Proof for the Usefulness of Crossover in Black-Box Optimization". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 2. Sept. 8–12, 2018, Coimbra, Portugal. Ed. by Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley. Vol. 11102 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, pp. 29–41. ISSN: **0302-9743**. ISBN: **978-3-319-99258-7**. doi:[10.1007/978-3-319-99259-4_3](https://doi.org/10.1007/978-3-319-99259-4_3) (cit. on pp. **226–228**, **358**).
- [17] Eduardo Carvalho Pinto and Carola Doerr. *Towards a More Practice-Aware Runtime Analysis of Evolutionary Algorithms*. arXiv.org: Computing Research Repository (CoRR) abs/1812.00493. Ithaca, NY, USA: Cornell University Library, Dec. 3, 2018. doi:[10.48550/arXiv.1812.00493](https://doi.org/10.48550/arXiv.1812.00493). URL: <https://arxiv.org/abs/1812.00493> (visited on 2025-08-08). arXiv:1812.00493v1 [cs.NE] 3 Dec 2018 (cit. on pp. **167–177**, **209–228**, **355**, **358**, **362**).
- [18] Vladimír Černý. "Thermodynamical Approach to the Traveling Salesman Problem: An Efficient Simulation Algorithm". *Journal of Optimization Theory and Applications* 45(1):41–51, Jan. 1985. New York, NY, USA: Springer Science+Business Media, LLC. ISSN: **0022-3239**. doi:[10.1007/BF00940812](https://doi.org/10.1007/BF00940812) (cit. on p. **361**).
- [19] Bo Chen, Chris N. Potts, and Gerhard J. Woeginger. "A Review of Machine Scheduling: Complexity, Algorithms and Approximability". In: *Handbook of Combinatorial Optimization*. Ed. by Panos Miliotis Pardalos, Ding-Zhu Du, and Ronald Lewis Graham. 1st ed. Boston, MA, USA: Springer, 1998, pp. 1493–1641. ISBN: **978-1-4613-7987-4**. doi:[10.1007/978-1-4613-0303-9_25](https://doi.org/10.1007/978-1-4613-0303-9_25). See also pages 21–169 in volume 3/3 by Norwell, MA, USA: Kluwer Academic Publishers. (Cit. on pp. **358**, **365**).
- [20] Jiayang Chen (陈嘉阳), Zhize Wu (吴志泽), Sarah Louise Thomson, and Thomas Weise (汤卫恩). "Frequency Fitness Assignment: Optimization Without Bias for Good Solution Outperforms Randomized Local Search on the Quadratic Assignment Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 27–37. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0012888600003837](https://doi.org/10.5220/0012888600003837) (cit. on pp. **357**, **360**, **361**).
- [21] Philippe Chrétienne, Edward G. Coffman, Jan Karel Lenstra, and Zhen Liu, eds. *Scheduling Theory and Its Applications*. Chichester, West Sussex, England, UK: John Wiley and Sons Ltd., Sept. 1995. ISBN: **978-0-471-94059-3** (cit. on p. **358**).

References IV



- [22] Yi Chu, Shaowei Cai, and Chuan Luo. "NuWLS: Improving Local Search for (Weighted) Partial MaxSAT by New Weighting Techniques". In: *37th AAAI Conference on Artificial Intelligence (AAAI'2023)*. Feb. 7–14, 2023, Washington, D.C., USA. Ed. by Brian Williams, Yiling Chen, and Jennifer Nevi. Washington, D.C., USA: AAAI Press, 2023, pp. 3915–3923. ISSN: **2159-5399**. ISBN: **978-1-57735-880-0**. doi:[10.1609/AAAI.V37I4.25505](https://doi.org/10.1609/AAAI.V37I4.25505) (cit. on pp. **237–243**).
- [23] Stephen Arthur Cook. "The Complexity of Theorem-Proving Procedures". In: *Third Annual ACM Symposium on Theory of Computing (STOC'1971)*. May 3–5, 1971, Shaker Heights, OH, USA. Ed. by Michael A. Harrison, Ranan B. Banerji, and Jeffrey D. Ullman. New York, NY, USA: Association for Computing Machinery (ACM), 1971, pp. 151–158. ISBN: **978-1-4503-7464-4**. doi:[10.1145/800157.805047](https://doi.org/10.1145/800157.805047) (cit. on pp. **229–236, 360, 365**).
- [24] William John Cook. *Concorde TSP Solver*. Waterloo, ON, Canada: University of Waterloo, 1997. URL: <https://math.uwaterloo.ca/tsp/concorde> (visited on 2026-09-26) (cit. on pp. **270–281, 356**).
- [25] William John Cook. *In Pursuit of the Traveling Salesman: Mathematics at the Limits of Computation*. Princeton, NJ, USA: Princeton University Press, 2012–Nov. 9, 2014. ISBN: **978-0-691-16352-9** (cit. on pp. **270–281, 356, 359**).
- [26] William John Cook. *World TSP*. Waterloo, ON, Canada: University of Waterloo, 2013–Oct. 25, 2025. URL: <http://www.math.uwaterloo.ca/tsp/world> (visited on 2026-01-03) (cit. on pp. **270–281, 359**).
- [27] Kenneth Alan De Jong. *Evolutionary Computation: A Unified Approach*. Vol. 4 of Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, 2006. ISBN: **978-0-262-04194-2**. URL: <https://www.researchgate.net/publication/220740669> (visited on 2025-08-08) (cit. on pp. **223–228, 357, 362**).
- [28] Paul Deitel, Harvey Deitel, and Abbey Deitel. *Internet & World Wide Web: How to Program*. 5th ed. Hoboken, NJ, USA: Pearson Education, Inc., Nov. 2011. ISBN: **978-0-13-299045-5** (cit. on p. **363**).
- [29] Miroslav Dimitrov. *New Classes of Binary Sequences with High Merit Factor*. arXiv.org: Computing Research Repository (CoRR). Ithaca, NY, USA: Cornell University Library, June 24, 2022. doi:[10.48550/arXiv.2206.12070](https://doi.org/10.48550/arXiv.2206.12070). URL: <https://arxiv.org/pdf/2206.12070> (visited on 2026-09-21). arXiv:2206.12070v1 [cs.IT] 24 Jun 2022 (cit. on pp. **248–261, 359**).

References V



- [30] Miroslav Dimitrov, Tsonka Baitcheva, and Nikolay Nikolov. "Efficient Generation of Low Autocorrelation Binary Sequences". *IEEE Signal Processing Letters* 27:341–345, 2020. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1558-2361. doi:10.1109/LSP.2020.2972127 (cit. on pp. 248–261, 359).
- [31] Slobodan Dmitrović. *Modern C for Absolute Beginners: A Friendly Introduction to the C Programming Language*. New York, NY, USA: Apress Media, LLC, Mar. 2024. ISBN: 979-8-8688-0224-9 (cit. on p. 356).
- [32] Benjamin Doerr and Carola Doerr. "Optimal Static and Self-Adjusting Parameter Choices for the $(1 + (\lambda, \lambda))$ Genetic Algorithm". *Algorithmica* 80(5):1658–1709, May 2018. London, England, UK: Springer Nature Limited. ISSN: 0178-4617. doi:10.1007/s00453-017-0354-9 (cit. on pp. 209–228, 362).
- [33] Benjamin Doerr, Carola Doerr, and Franziska Ebel. "From Black-Box Complexity to Designing New Genetic Algorithms". *Theoretical Computer Science* 567:87–104, Feb. 2015. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2014.11.028 (cit. on p. 360).
- [34] Benjamin Doerr, Frank Neumann, and Andrew M. Sutton. "Improved Runtime Bounds for the $(1+1)$ EA on Random 3-CNF Formulas Based on Fitness-Distance Correlation". In: *17th Genetic and Evolutionary Computation Conference (GECCO'2015)*. July 11–15, 2015, Madrid, Spain. Ed. by Sara Silva and Anna Isabel Esparcia-Alcázar. New York, NY, USA: Association for Computing Machinery (ACM), 2015, pp. 1415–1422. ISBN: 978-1-4503-3472-3. doi:10.1145/2739480.2754659 (cit. on pp. 237–243).
- [35] Benjamin Doerr, Frank Neumann, and Andrew M. Sutton. "Time Complexity Analysis of Evolutionary Algorithms on Random Satisfiable k -CNF Formulas". *Algorithmica* 78(2):561–586, 2017. London, England, UK: Springer Nature Limited. ISSN: 0178-4617. doi:10.1007/s00453-016-0190-3 (cit. on pp. 237–243).
- [36] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang, and Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". In: *21st Genetic and Evolutionary Computation Conference Companion (GECCO'2019)*. July 13–17, 2019, Prague, Czech Republic. Ed. by Manuel López-Ibáñez, Anne Auger, and Thomas Stützle. New York, NY, USA: Association for Computing Machinery (ACM), 2019, pp. 1798–1806. ISBN: 978-1-4503-6748-6. doi:10.1145/3319619.3326810. See also ^{37,38} (cit. on pp. 338, 339).
- [37] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang, and Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". *Applied Soft Computing (ASOC)* 88:106027, Mar. 2020. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 1568-4946. doi:10.1016/j.asoc.2019.106027. See also ^{36,38} (cit. on pp. 209–228, 338, 339, 355, 358, 360).

References VI



- [38] Carola Doerr, Furong Ye, Naama Horesh, Hao Wang, and Ofer M. ShirThomas Bäck. "Benchmarking Discrete Optimization Heuristics with IOHprofiler". (abs/1912.09237), Jan. 31, 2021. doi:10.48550/arXiv.1912.09237. URL: <https://arxiv.org/abs/1912.09237> (visited on 2024-06-27). arXiv:1912.09237v2 [cs.NE] 31 Jan 2021, see also ^{36,37} (cit. on p. 338).
- [39] Stefan Droste, Thomas Jansen, and Ingo Wegener. "On the Analysis of the $(1 + 1)$ Evolutionary Algorithm". *Theoretical Computer Science* 276(1-2):51–81, Apr. 2002. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/S0304-3975(01)00182-7 (cit. on pp. 167–177, 211–228, 355, 359, 362).
- [40] Stefan Droste, Thomas Jansen, and Ingo Wegener. "Upper and Lower Bounds for Randomized Search Heuristics in Black-Box Optimization". *Theoretical Computer Science* 39(4):525–544, July 2006. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1007/S00224-004-1177-2 (cit. on pp. 209–228, 360).
- [41] Harald Dyckhoff and Ute Finke. *Cutting and Packing in Production and Distribution: A Typology and Bibliography*. Contributions to Management Science. Heidelberg, Baden-Württemberg, Germany: Physica-Verlag, 1992. ISSN: 1431-1941. ISBN: 978-3-642-63487-1. doi:10.1007/978-3-642-58165-6 (cit. on pp. 13–18).
- [42] Luca Ferrari and Enrico Pirozzi. *Learn PostgreSQL*. 2nd ed. Birmingham, England, UK: Packt Publishing Ltd, Oct. 2023. ISBN: 978-1-83763-564-1 (cit. on p. 361).
- [43] Simon Fischer. "A Polynomial Upper Bound for a Mutation-Based Algorithm on the Two-Dimensional Ising Model". In: *6th Genetic and Evolutionary Computation Conference (GECCO'2004), Part I*. June 26–30, 2004, Seattle, WA, USA. Ed. by Kalyanmoy Deb, Riccardo Poli, Wolfgang Banzhaf, Hans-Georg Beyer, Edmund K. Burke, Paul J. Darwen, Dipankar Dasgupta, Dario Floreano, James A. Foster, Mark Harman, Owen Holland, Pier Luca Lanzi, Lee Spector, Andrea Tettamanzi, Dirk Thierens, and Andrew M. Tyrrell. Vol. 3102 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2004, pp. 1100–1112. ISSN: 0302-9743. ISBN: 978-3-540-22344-3. doi:10.1007/978-3-540-24854-5_108 (cit. on pp. 218–228, 358).
- [44] Simon Fischer and Ingo Wegener. "The One-Dimensional Ising Model: Mutation versus Recombination". *Theoretical Computer Science* 344(2-3):208–225, Nov. 2005. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0304-3975. doi:10.1016/J.TCS.2005.04.002 (cit. on pp. 218–228, 358).

References VII



- [45] Tobias Friedrich, Pietro Simone Oliveto, Dirk Sudholt, and Carsten Witt. "Analysis of Diversity-Preserving Mechanisms for Global Exploration". *Evolutionary Computation* 17(4):455–476, Win. 2009. Cambridge, MA, USA: MIT Press. ISSN: **1063-6560**. doi:[10.1162/evco.2009.17.4.17401](https://doi.org/10.1162/evco.2009.17.4.17401) (cit. on p. **363**).
- [46] Tobias Friedrich, Francesco Quinzan, and Markus Wagner. "Escaping Large Deceptive Basins of Attraction with Heavy-Tailed Mutation Operators". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyoto, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018, pp. 293–300. ISBN: **978-1-4503-5618-3**. doi:[10.1145/3205455.3205515](https://doi.org/10.1145/3205455.3205515) (cit. on pp. **215–228, 359, 363**).
- [47] José E. Gallardo, Carlos Cotta, and Antonio J. Fernández. "A Memetic Algorithm for the Low Autocorrelation Binary Sequence Problem". In: *9th Genetic and Evolutionary Computation Conference (GECCO'2007)*. July 7–11, 2007, London, England, UK. Ed. by Hod Lipson. New York, NY, USA: Association for Computing Machinery (ACM), 2007, pp. 1226–1233. ISBN: **978-1-59593-697-4**. doi:[10.1145/1276958.1277195](https://doi.org/10.1145/1276958.1277195) (cit. on p. **359**).
- [48] Michael R. Garey and David Stifler Johnson. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman and Company, 1979. ISBN: **978-0-7167-1045-5** (cit. on p. **363**).
- [49] Fred Glover. "Tabu Search – Part I". *ORSA Journal on Computing* 1(3):190–206, Sum. 1989. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0899-1499**. doi:[10.1287/ijoc.1.3.190](https://doi.org/10.1287/ijoc.1.3.190) (cit. on p. **362**).
- [50] Fred Glover. "Tabu Search – Part II". *ORSA Journal on Computing* 2(1):4–32, Win. 1990. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0899-1499**. doi:[10.1287/ijoc.2.1.4](https://doi.org/10.1287/ijoc.2.1.4) (cit. on p. **362**).
- [51] Fred Glover and Gary A. Kochenberger, eds. *Handbook of Metaheuristics*. Vol. 57 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2003. ISSN: **0884-8289**. ISBN: **978-0-306-48056-0**. doi:[10.1007/b101874](https://doi.org/10.1007/b101874) (cit. on p. **360**).
- [52] Fred Glover, Éric D. Taillard, and Dominique de Werra. "A User's Guide to Tabu Search". *Annals of Operations Research* 41(1):3–28, Mar. 1993. London, England, UK: Springer Nature Limited. ISSN: **0254-5330**. doi:[10.1007/BF02078647](https://doi.org/10.1007/BF02078647). Special issue on TS. (Cit. on p. **362**).

References VIII



- [53] David Edward Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Redwood City, CA, USA: Addison Wesley Longman Publishing Co., Inc., 1989. ISBN: 978-0-201-15767-3 (cit. on pp. 223–228, 357, 362).
- [54] Carla P. Gomes, Henry A. Kautz, Ashish Sabharwal, and Bart Selman. “Satisfiability Solvers”. In: *Handbook of Knowledge Representation*. Ed. by Frank van Harmelen, Vladimir Lifschitz, and Bruce Porter. Vol. 3 of Foundations of Artificial Intelligence. Amsterdam, The Netherlands: Elsevier B.V., 2008. Chap. 2, pp. 89–134. ISSN: 1574-6526. URL: <https://www.cs.cornell.edu/gomes/papers/SATSolvers-KR-Handbook.pdf> (visited on 2026-09-20) (cit. on pp. 237–243).
- [55] Michael T. Goodrich. *A Gentle Introduction to NP-Completeness*. Irvine, CA, USA: University of California, Irvine, Apr. 2022. URL: <https://ics.uci.edu/~goodrich/teach/cs165/notes/NPComplete.pdf> (visited on 2025-08-01) (cit. on pp. 364, 365).
- [56] Gregory Z. Gutin and Abraham P. Punnen, eds. *The Traveling Salesman Problem and its Variations*. Vol. 12 of Combinatorial Optimization (COOP). New York, NY, USA: Springer New York, May 2002. ISSN: 1388-3011. doi:10.1007/b101971 (cit. on p. 363).
- [57] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. “A Formal Basis for the Heuristic Determination of Minimum Cost Paths”. *IEEE Transactions on Systems Science and Cybernetics* 4:100–107, July 31, 1968. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0536-1567. doi:10.1109/TSSC.1968.300136 (cit. on pp. 43–108, 356).
- [58] Keld Helsgaun. “General k -opt Submoves for the Lin–Kernighan TSP Heuristic”. *Mathematical Programming Computation* 1(2-3):119–163, July 2009. London, England, UK: Springer Nature Limited. ISSN: 1867-2949. doi:10.1007/s12532-009-0004-6. URL: <https://www2.seas.gwu.edu/~simhaweb/champalg/tsp/papers/Helsgaun2.pdf> (visited on 2026-09-26) (cit. on pp. 270–281, 359).
- [59] Keld Helsgaun. “An Effective Implementation of the Lin-Kernighan Traveling Salesman Heuristic”. 126(1):106–130, Oct. 2000. doi:10.1016/S0377-2217(99)00284-2. URL: http://akira.ruc.dk/~keld/research/LKH/LKH-2.0/DOC/LKH_REPORT.pdf (visited on 2026-09-26) (cit. on pp. 270–281, 359).
- [60] Holger H. Hoos and Thomas Stützle. *Stochastic Local Search: Foundations & Applications*. The Morgan Kaufmann Series in Artificial Intelligence. Amsterdam, The Netherlands: Elsevier B.V., 2005. ISBN: 978-1-55860-872-6. doi:10.1016/B978-1-55860-872-6.X5016-1 (cit. on pp. 229–236, 360, 362).

References IX



- [61] Holger H. Hoos and Thomas Stützle. *SATLIB – The Satisfiability Library*. 1.4.4. Vancouver, BC, Canada: The University of British Columbia (UBC), May 11, 2001. URL: <https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html> (visited on 2026-09-20). See paper ⁶² (cit. on pp. 237–243, 342, 362).
- [62] Holger H. Hoos and Thomas Stützle. "SATLIB: An Online Resource for Research on SAT". In: *SAT2000 – Highlights of Satisfiability Research in the Year 2000*. Ed. by Ian Philip Gent, Toby Walsh, and Hans van Maaren. Vol. 63 of *Frontiers in Artificial Intelligence and Applications*. Amsterdam, The Netherlands: IOS Press BV, Jan. 2000, pp. 283–292. ISSN: 0922-6389. ISBN: 978-1-58603-061-2. See the website ⁶¹ (cit. on pp. 237–243, 342, 362).
- [63] John Hunt. *A Beginners Guide to Python 3 Programming*. 2nd ed. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2023. ISSN: 1863-7310. ISBN: 978-3-031-35121-1. doi:10.1007/978-3-031-35122-8 (cit. on p. 361).
- [64] *Information Technology – Universal Coded Character Set (UCS)*. International Standard ISO/IEC 10646:2020. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Dec. 2020 (cit. on p. 354).
- [65] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". In: *15th Annual Workshop on Microprogramming (MICRO 15)*. Oct. 5–7, 1982. Ed. by Joseph Allen Fisher, William J. Tracz, and Bill Hopkins. Palo Alto, CA, USA: Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE) and New York, NY, USA: Association for Computing Machinery (ACM), Oct. 1982, pp. 143–148. doi:10.5555/800036.800944. See ⁶⁶ (cit. on p. 342).
- [66] Dean Jacobs, Jan Prins, Peter Siegel, and Kenneth Wilson. "Monte Carlo Techniques in Code Optimization". *ACM SIGMICRO Newsletter* 13(4):143–148, Dec. 1982. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 1050-916X. doi:10.1145/1014194.800944. See ⁶⁵ (cit. on pp. 342, 361).
- [67] David Stifler Johnson, Gregory Z. Gutin, Lyle Andrew McGeoch, Anders Yeo, Weixiong Zhang, and Alexei Zverovitch. *Experimental Analysis of Heuristics for the ATSP*. Ed. by Gregory Z. Gutin and Abraham P. Punnen. New York, NY, USA, May 2002. doi:10.1007/0-306-48213-4_10. URL: <https://archive.dimacs.rutgers.edu/Challenges/TSP/papers/atpschap.pdf> (visited on 2026-09-25) (cit. on pp. 270–281).
- [68] David Stifler Johnson and Lyle Andrew McGeoch. *Experimental Analysis of Heuristics for the STSP*. Ed. by Gregory Z. Gutin and Abraham P. Punnen. New York, NY, USA, May 2002. doi:10.1007/0-306-48213-4_9. URL: <https://archive.dimacs.rutgers.edu/Challenges/TSP/papers/stpschap.pdf> (visited on 2026-09-25) (cit. on pp. 270–281).

References X



- [69] David Stifler Johnson, Lyle Andrew McGeoch, Fred Glover, and Cesar Rego. *8th DIMACS Implementation Challenge: The Traveling Salesman Problem*. June 1, 2000–Nov. 19, 2013. URL: <https://archive.dimacs.rutgers.edu/Challenges/TSP> (visited on 2026-09-25) (cit. on pp. 270–281).
- [70] Scott Kirkpatrick, C. Daniel Gelatt, Jr., and Mario P. Vecchi. "Optimization by Simulated Annealing". *Science Magazine* 220(4598):671–680, May 13, 1983. Washington, D.C., USA: American Association for the Advancement of Science (AAAS). ISSN: 0036-8075. doi:10.1126/science.220.4598.671. URL: <https://www.researchgate.net/publication/6026283> (visited on 2025-08-08) (cit. on pp. 178–190, 361).
- [71] Donald Ervin Knuth. "Big Omicron and Big Omega and Big Theta". *ACM SIGACT News* 8(2):18–24, Apr.–June 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0163-5700. doi:10.1145/1008328.1008329 (cit. on p. 365).
- [72] Donald Ervin Knuth. *Fundamental Algorithms*. 3rd ed. Vol. 1 of The Art of Computer Programming. Reading, MA, USA: Addison-Wesley Professional, 1997. ISBN: 978-0-201-89683-1 (cit. on p. 365).
- [73] Tjalling Charles Koopmans and Martin Beckmann. "Assignment Problems and the Location of Economic Activities". *Econometrica* 25(1):53–76, 1957. New Haven, CT, USA: The Econometric Society and Chichester, West Sussex, England, UK: John Wiley and Sons Ltd. ISSN: 0012-9682. doi:10.2307/1907742 (cit. on p. 361).
- [74] Edmund Landau. *Handbuch der Lehre von der Verteilung der Primzahlen*. Leipzig, Sachsen, Germany: B. G. Teubner, 1909. ISBN: 978-0-8218-2650-8 (cit. on p. 365).
- [75] Pedro Larrañaga, Cindy M. H. Kuijpers, Roberto H. Murga, Iñaki Inza, and S. Dizdarevic. "Genetic Algorithms for the Travelling Salesman Problem: A Review of Representations and Operators". *Artificial Intelligence Review* 13(2):129–170, Apr. 1999. London, England, UK: Springer Nature Limited. ISSN: 0269-2821. doi:10.1023/A:1006529012972 (cit. on pp. 263–269, 356).
- [76] Pedro Larrañaga, Cindy M. H. Kuijpers, Mikel Poza, and Roberto H. Murga. "Decomposing Bayesian Networks: Triangulation of the Moral Graph with Genetic Algorithms". *Statistics and Computing* 7(1):19–34, Mar. 1997. London, England, UK: Springer Nature Limited. ISSN: 0960-3174. doi:10.1023/A:1018553211613. This paper introduces AP. (Cit. on pp. 263–269, 356).

References XI



- [77] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. "Sequencing and Scheduling: Algorithms and Complexity". In: *Production Planning and Inventory*. Ed. by Stephen C. Graves, Alexander Hendrik George Rinnooy Kan, and Paul H. Zipkin. Vol. IV of Handbooks of Operations Research and Management Science. Amsterdam, The Netherlands: Elsevier B.V., 1993. Chap. 9, pp. 445–522. ISSN: 0927-0507. ISBN: 978-0-444-87472-6. doi:10.1016/S0927-0507(05)80189-6. URL: <http://alexandria.tue.nl/repository/books/339776.pdf> (visited on 2023-12-06) (cit. on pp. 358, 365).
- [78] Eugene Leighton Lawler, Jan Karel Lenstra, Alexander Hendrik George Rinnooy Kan, and David B. Shmoys. *The Traveling Salesman Problem: A Guided Tour of Combinatorial Optimization*. Estimation, Simulation, and Control – Wiley-Interscience Series in Discrete Mathematics and Optimization. Chichester, West Sussex, England, UK: Wiley Interscience, Sept. 1985. ISSN: 0277-2698. ISBN: 978-0-471-90413-7 (cit. on pp. 121, 363).
- [79] Kent D. Lee and Steve Hubbard. *Data Structures and Algorithms with Python*. Undergraduate Topics in Computer Science (UTICS). Cham, Switzerland: Springer, 2015. ISSN: 1863-7310. ISBN: 978-3-319-13071-2. doi:10.1007/978-3-319-13072-9 (cit. on p. 361).
- [80] Chu Min Li, Felip Manyà, Nouredine Ould Mohamedou, and Jordi Planes. "Resolution-based Lower Bounds in MaxSAT". *Constraints* 15(4):456–484, Oct. 2010. London, England, UK: Springer Nature Limited. ISSN: 1383-7133. doi:10.1007/S10601-010-9097-9 (cit. on pp. 237–243).
- [81] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, Sarah Louise Thomson, and Thomas Weise (汤卫思). "Addressing the Traveling Salesperson Problem with Frequency Fitness Assignment and Hybrid Algorithms". *Soft Computing* 28(17-18):9495–9508, July 2024. London, England, UK: Springer Nature Limited. ISSN: 1432-7643. doi:10.1007/S00500-024-09718-8 (cit. on pp. 357, 360, 363).
- [82] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Jörg Lässig, Daan van den Berg, and Thomas Weise (汤卫思). "Solving the Traveling Salesperson Problem using Frequency Fitness Assignment". In: *IEEE Symposium Series on Computational Intelligence (SSCI'2022)*. Dec. 4–7, 2022, Singapore. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE), 2022. ISBN: 978-1-6654-8769-6. doi:10.1109/SSCI51031.2022.10022296 (cit. on pp. 357, 360).

References XII



- [83] Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Matthias Thüerer, Markus Wagner, and Thomas Weise (汤卫恩). "Generating Small Instances with Interesting Features for the Traveling Salesperson Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 173–180. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0012888800003837](https://doi.org/10.5220/0012888800003837) (cit. on p. 360).
- [84] Shen Lin and Brian W. Kernighan. "An Effective Heuristic Algorithm for the Traveling-Salesman Problem". *Operations Research* 21(2):498–516, Mar.–Apr. 1973. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0030-364X**. doi:[10.1287/opre.21.2.498](https://doi.org/10.1287/opre.21.2.498). URL: <https://www.cs.princeton.edu/~bwk/btl.mirror/tsp.pdf> (visited on 2026-09-26) (cit. on pp. 270–281, 359).
- [85] John D. C. Little, Katta G. Murty, Dura W. Sweeny, and Caroline Karel. *An Algorithm for the Traveling Salesman Problem*. Sloan Working Papers 07-63. Massachusetts Institute of Technology (MIT): Massachusetts Institute of Technology (MIT), Sloan School of Management, Mar. 1963. URL: <http://dspace.mit.edu/bitstream/handle/1721.1/46828/algorithmfortrav00litt.pdf> (visited on 2026-01-03) (cit. on p. 356).
- [86] Eliane Maria Loiola, Nair Maria Maia de Abreu, Paulo Oswaldo Boaventura-Netto, Peter M. Hahn, and Tania Querido. "A Survey for the Quadratic Assignment Problem". *European Journal of Operational Research* 176(2):657–690, 2007. Amsterdam, The Netherlands: Elsevier B.V. ISSN: **0377-2217**. doi:[10.1016/j.ejor.2005.09.032](https://doi.org/10.1016/j.ejor.2005.09.032) (cit. on p. 361).
- [87] Marc Loy, Patrick Niemeyer, and Daniel Leuck. *Learning Java*. 5th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2020. ISBN: **978-1-4920-5627-0** (cit. on p. 358).
- [88] Zhipeng Lü and Jin-Kao Hao. "Adaptive Memory-Based Local Search for MAX-SAT". *Applied Soft Computing (ASOC)* 12(8):2063–2071, Aug. 2012. Amsterdam, The Netherlands: Elsevier B.V. ISSN: **1568-4946**. doi:[10.1016/J.ASOC.2012.01.013](https://doi.org/10.1016/J.ASOC.2012.01.013) (cit. on pp. 237–243).
- [89] Mark Lutz. *Learning Python*. 6th ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2025. ISBN: **978-1-0981-7130-8** (cit. on p. 361).
- [90] Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe, eds. *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0000195000003837](https://doi.org/10.5220/0000195000003837).

References XIII



- [91] Stephan Mertens. "Exhaustive Search for Low-Autocorrelation Binary Sequences". *Journal of Physics A: Mathematical and General* 29(18):L473, Sept. 1996. Bristol, England, UK: IOP Publishing Ltd. ISSN: 0305-4470. doi:10.1088/0305-4470/29/18/005 (cit. on pp. 248–261, 359).
- [92] Zbigniew "Zbyszek" Michalewicz. *Genetic Algorithms + Data Structures = Evolution Programs*. Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 1996. ISBN: 978-3-540-58090-4. doi:10.1007/978-3-662-03315-9 (cit. on pp. 223–228, 357, 362).
- [93] Burkhard Militzer, Michele Zamparelli, and Dieter Beule. "Evolutionary Search for Low Autocorrelated Binary Sequences". *IEEE Transactions on Evolutionary Computation* 2(1):34–39, Apr. 1998. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/4235.728212 (cit. on pp. 248–261, 359).
- [94] Melanie Mitchell. *An Introduction to Genetic Algorithms*. Complex Adaptive Systems. Cambridge, MA, USA: MIT Press, Feb. 1998. ISBN: 978-0-262-13316-6. URL: <http://boente.eti.br/fuzzy/ebook-fuzzy-mitchell.pdf> (visited on 2025-08-08) (cit. on pp. 223–228, 357, 362).
- [95] Wai Ho Mow, Ke-Lin Du, and Wei Hsiang Wu. "New Evolutionary Search for Long Low Autocorrelation Binary Sequences". *IEEE Transactions on Aerospace and Electronic Systems* 51(1):290–303, Jan. 2015. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 0018-9251. doi:10.1109/TAES.2014.130518 (cit. on pp. 248–261, 359).
- [96] Heinz Mühlenbein. "How Genetic Algorithms Really Work: Mutation and Hillclimbing". In: *Parallel Problem Solving from Nature 2 (PPSN-II)*. Sept. 28–30, 1992, Brussels, Belgium. Ed. by Reinhard Männer and Bernard Manderick. Amsterdam, The Netherlands: Elsevier B.V., 1992, pp. 15–26. URL: <https://www.researchgate.net/publication/284805798> (visited on 2025-08-15) (cit. on pp. 209–228, 360).
- [97] Yuichi Nagata and Shigenobu Kobayashi. "A Powerful Genetic Algorithm using Edge Assembly Crossover for the Traveling Salesman Problem". *INFORMS Journal on Computing* 25(2):346–363, Spr. 2013. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 1091-9856. doi:10.1287/ijoc.1120.0506 (cit. on pp. 270–281, 357).
- [98] Yuichi Nagata and Shigenobu Kobayashi. "Edge Assembly Crossover: A High-Power evolutionary algorithm (EA) Genetic Algorithm for the Traveling Salesman Problem". In: *7th International Conference on Genetic Algorithms (ICGA'1997)*. July 19–23, 1997, East Lansing, MI, USA. Ed. by Thomas Bäck. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1997, pp. 450–457. ISBN: 978-1-55860-487-2 (cit. on pp. 270–281, 357).

References XIV



- [99] Yuichi Nagata and David Soler. "A New Genetic Algorithm for the Asymmetric Traveling Salesman Problem". *Expert Systems with Applications* 39(10):8947–8953, Aug. 2012. Amsterdam, The Netherlands: Elsevier B.V. ISSN: **0957-4174**. doi:[10.1016/j.eswa.2012.02.029](https://doi.org/10.1016/j.eswa.2012.02.029) (cit. on pp. **270–281, 357**).
- [100] Siegfried Nijssen and Thomas Bäck. "An Analysis of the Behavior of Simplified Evolutionary Algorithms on Trap Functions". *IEEE Transactions on Evolutionary Computation* 7(1):11–22, Feb. 2003. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: **1089-778X**. doi:[10.1109/TEVC.2002.806169](https://doi.org/10.1109/TEVC.2002.806169) (cit. on pp. **214–228, 362**).
- [101] Regina O. Obe and Leo S. Hsu. *PostgreSQL: Up and Running*. 3rd ed. Sebastopol, CA, USA: O'Reilly Media, Inc., Oct. 2017. ISBN: **978-1-4919-6336-4** (cit. on p. **361**).
- [102] Tom Packebusch and Stephan Mertens. "Low Autocorrelation Binary Sequences". *Journal of Physics A: Mathematical and Theoretical* 49(16):165001, Mar. 2016. Bristol, England, UK: IOP Publishing Ltd. ISSN: **1751-8121**. doi:[10.1088/1751-8113/49/16/165001](https://doi.org/10.1088/1751-8113/49/16/165001) (cit. on pp. **248–261, 359**).
- [103] Panos Miltiades Pardalos, Ding-Zhu Du, and Ronald Lewis Graham, eds. *Handbook of Combinatorial Optimization*. 1st ed. Boston, MA, USA: Springer, 1998. ISBN: **978-1-4613-7987-4**.
- [104] Amit Patel. *Amit's A* Pages*. Stanford, CA, USA: Stanford University, 1997–2025. URL: <https://theory.stanford.edu/~amitp/GameProgramming> (visited on 2026-01-02) (cit. on pp. **43–108, 356**).
- [105] Tobias Paxian, Sven Reimer, and Bernd Becker. "Dynamic Polynomial Watchdog Encoding for Solving Weighted MaxSAT". In: *21st International Conference on Theory and Applications of Satisfiability Testing (SAT'2018), Held as Part of the Federated Logic Conference (FloC'2018)*. July 9–12, 2018, Oxford, Oxfordshire, England, UK. Ed. by Olaf Beyersdorff and Christoph M. Wintersteiger. Vol. 10929 of Theoretical Computer Science and General Issues (LNTCS), sub-series of Lecture Notes in Computer Science (LNCS). 2018, pp. 37–53. ISSN: **2512-2010**. doi:[10.1007/978-3-319-94144-8_3](https://doi.org/10.1007/978-3-319-94144-8_3) (cit. on pp. **237–243**).
- [106] Martin Pincus. "Letter to the Editor – A Monte Carlo Method for the Approximate Solution of Certain Types of Constrained Optimization Problems". *Operations Research* 18(6):1225–1228, Nov.–Dec. 1970. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: **0030-364X**. doi:[10.1287/opre.18.6.1225](https://doi.org/10.1287/opre.18.6.1225) (cit. on p. **361**).

References XV



- [107] Alexander Podlich, Thomas Weise (汤卫恩), Manfred Menze, and Christian Gorldt. "Intelligente Wechselbrückensteuerung für die Logistik von Morgen". *Electronic Communications of the EASST (ECEASST)* 17(Communication in Distributed Systems), Feb. 27, 2009. The Netherlands: European Association of Software Science and Technology (EASST). ISSN: 1863-2122. doi:10.14279/tuj.eceasst.17.205. In the proceedings of *Workshops der Wissenschaftlichen Konferenz Kommunikation in Verteilten Systemen 2009* (WowKiVS 2009), 3 6, 2009, Kassel, Hessen, Germany (cit. on pp. 13–18).
- [108] *PostgreSQL Essentials: Leveling Up Your Data Work*. Sebastopol, CA, USA: O'Reilly Media, Inc., Mar. 2024 (cit. on p. 361).
- [109] S. D. Prestwich. *Improved Branch-and-Bound for Low Autocorrelation Binary Sequences*. arXiv.org: Computing Research Repository (CoRR) abs/1305.6187. Ithaca, NY, USA: Cornell University Library, Apr. 3, 2024. doi:10.48550/arXiv.1305.6187. URL: <https://arxiv.org/abs/1305.6187> (visited on 2013-07-23). arXiv:1305.6187v2 [cs.AI] 23 Jul 2013 (cit. on p. 359).
- [110] *Programming Languages – C, Working Document of SC22/WG14*. International Standard ISO/31EC9899:2017 C17 Ballot N2176. Geneva, Switzerland: International Organization for Standardization (ISO) and International Electrotechnical Commission (IEC), Nov. 2017. URL: <https://files.lhmouse.com/standards/ISO%20C%20N2176.pdf> (visited on 2024-06-29) (cit. on p. 356).
- [111] Sanatan Rai and George Vairaktarakis. "NP-Complete Problems and Proof Methodology". In: *Encyclopedia of Optimization*. Ed. by Christodoulos A. Floudas and Panos Miliotakis Pardalos. 2nd ed. Boston, MA, USA: Springer, Sept. 2008, pp. 2675–2682. ISBN: 978-0-387-74758-3. doi:10.1007/978-0-387-74759-0_462 (cit. on p. 364).
- [112] Soraya B. Rana and L. Darrell Whitley. "Genetic Algorithm Behavior in the MAXSAT Domain". In: *5th International Conference on Parallel Problem Solving from Nature (PPSN V)*. Sept. 27–30, 1998, Amsterdam, The Netherlands. Ed. by Ágoston Endre Eiben, Thomas Bäck, Marc Schoenauer, and Hans-Paul Schwefel. Lecture Notes in Computer Science (LNCS). 1998, pp. 785–794. ISSN: 0302-9743. ISBN: 978-3-540-65078-2. doi:10.1007/BFb0056920 (cit. on p. 360).
- [113] Gerhard Reinelt. "TSPLIB – A Traveling Salesman Problem Library". *ORSA Journal on Computing* 3(4):376–384, Aut. 1991. Catonsville, MD, USA: The Institute for Operations Research and the Management Sciences (INFORMS). ISSN: 0899-1499. doi:10.1287/ijoc.3.4.376 (cit. on pp. 270–281, 363).

References XVI



- [114] Gerhard Reinelt. *TSPLIB95*. Tech. rep. Heidelberg, Baden-Württemberg, Germany: Universität Heidelberg, Institut für Angewandte Mathematik, 1995–Apr. 28, 2016. URL: <https://github.com/shredderzwj/TSPLIB> (visited on 2026-09-25). The original Uniform Resource Locator (URL) of the repository has become defunct in 2026. Therefore, we provide the repository listed as main URL which contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite> of our own project *TSPSuite*, which contains all the TSPLib instances as well as Java code to load them. (Cit. on pp. 270–281, 363).
- [115] Günter Rudolph. *Convergence Properties of Evolutionary Algorithms*. Vol. 35 of Forschungsergebnisse zur Informatik. Hamburg, Germany: Verlag Dr. Kovač, 1997. ISSN: 1435-6260. ISBN: 978-3-86064-554-3 (cit. on pp. 213–228, 359).
- [116] Sartaj Sahni and Teofilo Francisco Gonzalez Arce. “P-complete Approximation Problems”. *Journal of the ACM (JACM)* 23(3):555–565, July 1976. New York, NY, USA: Association for Computing Machinery (ACM). ISSN: 0004-5411. doi:10.1145/321958.321975. URL: <https://sites.cs.ucsb.edu/~teo/papers/JACM-PCOM.pdf> (visited on 2026-02-21) (cit. on p. 361).
- [117] Guntram Scheithauer. *Introduction to Cutting and Packing Optimization: Problems, Modeling Approaches, Solution Methods*. Vol. 263 of International Series in Operations Research & Management Science (ISOR). Cham, Switzerland: Springer, 2018. ISSN: 0884-8289. ISBN: 978-3-319-64402-8. doi:10.1007/978-3-319-64403-5 (cit. on pp. 13–18).
- [118] Bart Selman, Henry A. Kautz, and Bram Cohen. “Noise Strategies for Improving Local Search”. In: *12th National Conference on Artificial Intelligence*. Vol. 1. July 31–Aug. 4, 1994, Seattle, WA, USA. Ed. by Barbara Hayes-Roth and Richard E. Korf. Washington, D.C., USA: AAAI Press and Cambridge, MA, USA: MIT Press, 1994, pp. 337–343. ISBN: 978-0-262-61102-2. URL: <http://www.aaai.org/Library/AAAI/1994/aaai94-051.php> (cit. on pp. 237–243).
- [119] Dirk Sudholt. “Crossover Speeds up Building-Block Assembly”. In: *14th Genetic and Evolutionary Computation Conference (GECCO’2012)*. July 7–11, 2012, Philadelphia, PA, USA. Ed. by Terence Soule and Jason Hall Moore. New York, NY, USA: Association for Computing Machinery (ACM), 2012, pp. 689–702. ISBN: 978-1-4503-1177-9. doi:10.1145/2330163.2330260. This paper is on the Greedy (2 + 1) Genetic Algorithm (Greedy (2 + 1) GA), see also ¹²⁰ (cit. on pp. 226–228, 349, 358).
- [120] Dirk Sudholt. “How Crossover Speeds up Building Block Assembly in Genetic Algorithms”. *Evolutionary Computation* 25(2):237–274, 2017. Cambridge, MA, USA: MIT Press. ISSN: 1063-6560. doi:10.1162/EVCO_a_00171. This paper is on the Greedy (2 + 1) GA, see also ¹¹⁹ (cit. on pp. 226–228, 349, 358).

References XVII



- [121] Éric D. Taillard. "Benchmarks for Basic Scheduling Problems". *European Journal of Operational Research* 64(2):278–285, Jan. 1993. Amsterdam, The Netherlands: Elsevier B.V. ISSN: 0377-2217. doi:10.1016/0377-2217(93)90182-M. URL: <https://arodes.hes-so.ch/record/8224/files/Preprint.pdf> (visited on 2026-02-20) (cit. on p. 358).
- [122] Alkin Tezuysal and Ibrar Ahmed. *Database Design and Modeling with PostgreSQL and MySQL*. Birmingham, England, UK: Packt Publishing Ltd, July 2024. ISBN: 978-1-80323-347-5 (cit. on p. 361).
- [123] Dirk Thierens, David Edward Goldberg, and Angela Guimarães Pereira. "Domino Convergence, Drift, and the Temporal-Salience Structure of Problems". In: pp. 535–540. doi:10.1109/ICEC.1998.700085 (cit. on pp. 212–228, 355).
- [124] Sarah Louise Thomson, Gabriela Ochoa, Daan van den Berg, Tianyu Liang (梁天宇), and Thomas Weise (汤卫恩). "Entropy, Search Trajectories, and Explainability for Frequency Fitness Assignment". In: *Parallel Problem Solving from Nature (PPSN XVIII)*. Vol. 1. Sept. 14–18, 2024, Hagenberg, Mühlkreis, Austria. Ed. by Michael Affenzeller, Stephan M. Winkler, Anna V. Kononova, Heike Trautmann, Tea Tušar, Penousal Machado, and Thomas Bäck. Vol. 15148 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer. ISSN: 0302-9743. ISBN: 978-3-031-70054-5. doi:10.1007/978-3-031-70055-2_23 (cit. on pp. 357, 360).
- [125] Renato Tinós, Keld Helsgaun, and L. Darrell Whitley. "Efficient Recombination in the Lin-Kernighan-Helsgaun Traveling Salesman Heuristic". In: *15th International Conference on Parallel Problem Solving from Nature (PPSN XV)*. Vol. 1. Sept. 8–12, 2018, Coimbra, Portugal. Ed. by Anne Auger, Carlos M. Fonseca, Nuno Lourenço, Penousal Machado, Luís Paquete, and L. Darrell Whitley. Vol. 11101 of Lecture Notes in Computer Science (LNCS). Cham, Switzerland: Springer, 2018, pp. 95–107. ISSN: 0302-9743. ISBN: 978-3-319-99252-5. doi:10.1007/978-3-319-99253-2_8 (cit. on p. 357).
- [126] Clarissa Van Hoyweghen, David Edward Goldberg, and Bart Naudts. "From Twomax To The Ising Model: Easy And Hard Symmetrical Problems". In: *4th Genetic and Evolutionary Computation Conference (GECCO'2002)*. July 9–13, 2002, New York, NY, USA. Ed. by William B. Langdon, Erick Cantú-Paz, Keith E. Mathias, Rajkumar Roy, David Davis, Riccardo Poli, Karthik Balakrishnan, Vasant G. Honavar, Günter Rudolph, Joachim Wegener, Larry Bull, Mitchell A. Potter, Alan C. Schultz, Julian F. Miller, Edmund K. Burke, and Natasa Jonoska. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 2002, pp. 626–633. ISBN: 978-1-55860-878-8 (cit. on pp. 215–228, 363).

References XVIII



- [127] Thomas Weise (汤卫思). *Databases*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2025. URL: <https://thomasweise.github.io/databases> (visited on 2025-01-05) (cit. on pp. 331, 357).
- [128] Thomas Weise (汤卫思). *Global Optimization Algorithms – Theory and Application*. self-published, 2009. URL: <https://www.researchgate.net/publication/200622167> (visited on 2025-07-25) (cit. on pp. 223–228, 357, 360, 362).
- [129] Thomas Weise (汤卫思). *Programming with C*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2026. URL: <https://thomasweise.github.io/programmingWithC> (visited on 2026-05-10) (cit. on p. 356).
- [130] Thomas Weise (汤卫思). *Programming with Python*. Hefei, Anhui, China (中国安徽省合肥市): Hefei University (合肥大学), School of Artificial Intelligence and Big Data (人工智能与大数据学院), 2024–2026. URL: <https://thomasweise.github.io/programmingWithPython> (visited on 2025-01-05) (cit. on pp. 330, 361).
- [131] Thomas Weise (汤卫思), Raymond Chiong, Jörg Lässig, Ke Tang (唐珂), Shigeyoshi Tsutsui, Wenxiang Chen (陈文祥), Zbigniew “Zbyszek” Michalewicz, and Xin Yao (姚新). “Benchmarking Optimization Algorithms: An Open Source Framework for the Traveling Salesman Problem”. *IEEE Computational Intelligence Magazine (CIM)* 9(3):40–52, Aug. 2014. Piscataway, NJ, USA: Institute of Electrical and Electronics Engineers (IEEE). ISSN: 1556-603X. doi:10.1109/MCI.2014.2326101 (cit. on pp. 270–281, 356, 363).
- [132] Thomas Weise (汤卫思), Stefan Niemczyk, Hendrik Skubch, Roland Reichle, and Kurt Geihs. “A Tunable Model for Multi-Objective, Epistatic, Rugged, and Neutral Fitness Landscapes”. In: *10th Genetic and Evolutionary Computation Conference (GECCO’2008)*. July 12–16, 2008, Atlanta, GA, USA. New York, NY, USA: Association for Computing Machinery (ACM), 2008, pp. 795–802. ISBN: 978-1-60558-130-9. doi:10.1145/1389095.1389252 (cit. on pp. 220–228, 363).
- [133] Thomas Weise (汤卫思), Alexander Podlich, and Christian Gorltdt. “Solving Real-World Vehicle Routing Problems with Evolutionary Algorithms”. In: *Natural Intelligence for Scheduling, Planning and Packing Problems*. Ed. by Raymond Chiong and Sandeep Dhakal. Vol. 250 of Studies in Computational Intelligence (SCI). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, Sept. 2009. Chap. 2, pp. 29–53. ISSN: 1860-949X. ISBN: 978-3-642-04038-2. doi:10.1007/978-3-642-04039-9_2 (cit. on pp. 13–18).

References XIX



- [134] Thomas Weise (汤卫思), Alexander Podlich, Manfred Menze, and Christian Gorltd. "Optimierte Güterverkehrsplanung mit Evolutionären Algorithmen". *Industrie Management – Zeitschrift für industrielle Geschäftsprozesse* 10(3):37–40, June 2009. Berlin, Germany: GITO mbH Verlag (cit. on pp. 13–18).
- [135] Thomas Weise (汤卫思), Alexander Podlich, Kai Reinhard, Christian Gorltd, and Kurt Geihs. "Evolutionary Freight Transportation Planning". In: *Applications of Evolutionary Computing (EvoWorkshops 2009): Proceedings of EvoCOMNET, EvoENVIRONMENT, EvoFIN, EvoGAMES, EvoHOT, EvoIASP, EvoINTERACTION, EvoMUSART, EvoNUM, EvoSTOC, EvoTRANSLOG*. Apr. 15–17, 2009, Tübingen, Baden-Württemberg, Germany. Ed. by Mario Giacobini, Anthony Brabazon, Stefano Cagnoni, Gianni A. Caro, Anikó Ekárt, Anna Isabel Esparcia-Alcázar, Muddassar Farooq, Andreas Fink, and Penousal Machado. Vol. 5484 of Theoretical Computer Science and General Issues (LNTCS), sub-series of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, Apr. 2009, pp. 768–777. ISSN: 2512-2010. doi:10.1007/978-3-642-01129-0_87 (cit. on pp. 13–18).
- [136] Thomas Weise (汤卫思), Mingxu Wan (万明绪), Pu Wang (王璞), Ke Tang (唐珂), Alexandre Devert, and Xin Yao (姚新). "Frequency Fitness Assignment". *IEEE Transactions on Evolutionary Computation* 18(2):226–243, Apr. 2014. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2013.2251885 (cit. on p. 357).
- [137] Thomas Weise (汤卫思) and Zhize Wu (吴志泽). "Replicable Self-Documenting Experiments with Arbitrary Search Spaces and Algorithms". In: *25th Conference on Genetic and Evolutionary Computation (GECCO'2023), Companion Volume*. July 15–19, 2023, Lisbon, Portugal. Ed. by Sara Silva and Luís Paquete. New York, NY, USA: Association for Computing Machinery (ACM), 2023, pp. 1891–1899. ISBN: 979-8-4007-0120-7. doi:10.1145/3583133.3596306 (cit. on pp. 332, 360).
- [138] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), and Yan Chen (陈岩). "Frequency Fitness Assignment: Making Optimization Algorithms Invariant under Bijective Transformations of the Objective Function Value". *IEEE Transactions on Evolutionary Computation* 25(2):307–319, Apr. 2021. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2020.3032090 (cit. on pp. 227, 228, 355, 357, 359, 360, 362, 363).
- [139] Thomas Weise (汤卫思), Zhize Wu (吴志泽), Xinlu Li (李新路), Yan Chen (陈岩), and Jörg Lässig. "Frequency Fitness Assignment: Optimization without Bias for Good Solutions can be Efficient". *IEEE Transactions on Evolutionary Computation* 27(4):980–992, 2023. Los Alamitos, CA, USA: IEEE Computer Society. ISSN: 1089-778X. doi:10.1109/TEVC.2022.3191698 (cit. on pp. 227, 228, 355, 357, 359, 362, 363).

References XX



- [140] Thomas Weise (汤卫恩) and Zijun Wu (吴自军). "Difficult Features of Combinatorial Optimization Problems and the Tunable W-Model Benchmark Problem for Simulating Them". In: *20th Genetic and Evolutionary Computation Conference (GECCO'2018)*. July 15–19, 2018, Kyoto, Japan. Ed. by Hernán E. Aguirre and Keiki Takadama. New York, NY, USA: Association for Computing Machinery (ACM), 2018. ISBN: **978-1-4503-5618-3**. doi:[10.1145/3205651.3208240](https://doi.org/10.1145/3205651.3208240) (cit. on pp. **220–228, 363**).
- [141] Thomas Weise (汤卫恩), Yuezhong Wu (吴越钟), Raymond Chiong, Ke Tang (唐珂), and Jörg Lässig. "Global versus Local Search: The Impact of Population Sizes on Evolutionary Algorithm Performance". *Journal of Global Optimization* 66(3):511–534, Feb. 2016. London, England, UK: Springer Nature Limited. ISSN: **0925-5001**. doi:[10.1007/s10898-016-0417-5](https://doi.org/10.1007/s10898-016-0417-5) (cit. on pp. **356, 363**).
- [142] L. Darrell Whitley. "The GENITOR Algorithm and Selection Pressure: Why Rank-Based Allocation of Reproductive Trials is Best". In: *3rd International Conference on Genetic Algorithms (ICGA'1989)*. June 1989, Fairfax, VA, USA: George Mason University. Ed. by J. David Schaffer. Burlington, MA, USA/San Mateo, CA, USA: Morgan Kaufmann Publishers, 1989, pp. 116–123. ISBN: **978-1-55860-066-9**. URL: <https://www.researchgate.net/publication/2527551> (visited on 2025-08-08) (cit. on pp. **213–228, 359, 362**).
- [143] L. Darrell Whitley, Doug Hains, and Adele Howe. "A Hybrid Genetic Algorithm for the Traveling Salesman Problem using Generalized Partition Crossover". In: *11th International Conference on Parallel Problem Solving from Nature (PPSN XI)*. Vol. 1. Sept. 11–15, 2010, Kraków, Poland. Ed. by Robert Schaefer, Carlos Cotta, Joanna Kolodziej, and Günter Rudolph. Vol. 6238 of Lecture Notes in Computer Science (LNCS). Berlin/Heidelberg, Germany: Springer-Verlag GmbH Germany, 2010, pp. 566–575. ISSN: **0302-9743**. ISBN: **978-3-642-15843-8**. doi:[10.1007/978-3-642-15844-5_57](https://doi.org/10.1007/978-3-642-15844-5_57). URL: <https://www.cs.colostate.edu/~howe/cs640/papers/GpxREV.pdf> (visited on 2026-09-26) (cit. on pp. **270–281, 357**).
- [144] CAO Xiang (曹翔), Zhize Wu (吴志泽), Daan van den Berg, and Thomas Weise (汤卫恩). "Randomized Local Search vs. NSGA-II vs. Frequency Fitness Assignment on The Traveling Tournament Problem". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 38–49. ISSN: **2184-3236**. ISBN: **978-989-758-721-4**. doi:[10.5220/0012891500003837](https://doi.org/10.5220/0012891500003837) (cit. on pp. **356, 360**).
- [145] Kinza Yasar and Craig S. Mullins. *Definition: Database Management System (DBMS)*. Newton, MA, USA: TechTarget, Inc., June 2024. URL: <https://www.techtarget.com/searchdatamanagement/definition/database-management-system> (visited on 2025-01-11) (cit. on p. **357**).

References XXI



- [146] François Yergeau. *UTF-8, A Transformation Format of ISO 10646*. Request for Comments (RFC) 3629. Wilmington, DE, USA: Internet Engineering Task Force (IETF), Nov. 2003. URL: <https://www.ietf.org/rfc/rfc3629.txt> (visited on 2025-02-05). See Unicode and ⁶⁴ (cit. on p. 363).
- [147] Jiazheng Zeng (曾嘉政), Thomas Weise (汤卫思), Zhize Wu (吴志泽), and Markus Wagner. "When Frequency Fitness Assignment Fails: Trapped States in Frequency-Guided Local Search". In: *28th Genetic and Evolutionary Computation Conference (GECCO'2026)*. July 13–17, 2026, San José, Costa Rica. Ed. by Leonardo Trujillo and Ting Hu. New York, NY, USA: Association for Computing Machinery (ACM), 2026, pp. 233–242. ISBN: 979-8-4007-2487-9. doi:10.1145/3795095.3805098 (cit. on pp. 209–228, 355, 357–363).
- [148] Rui Zhao (赵睿), Tianyu Liang (梁天宇), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer, and Thomas Weise (汤卫思). "Randomized Local Search on the 2D Rectangular Bin Packing Problem with Item Rotation". In: *26th Genetic and Evolutionary Computation Conference (GECCO'2024)*. July 14–18, 2024, Melbourne, VIC, Australia. Ed. by Xiaodong Li and Julia Handl. New York, NY, USA: Association for Computing Machinery (ACM), 2024, pp. 235–238. ISBN: 979-8-4007-0494-9. doi:10.1145/3638530.3654139 (cit. on pp. 13–18, 118–120, 360).
- [149] Rui Zhao (赵睿), Zhize Wu (吴志泽), Daan van den Berg, Matthias Thüerer, Tianyu Liang (梁天宇), Ming Tan (檀明), and Thomas Weise (汤卫思). "Randomized Local Search for Two-Dimensional Bin Packing and a Negative Result for Frequency Fitness Assignment". In: *16th International Joint Conference on Computational Intelligence (IJCCI'24)*. Nov. 20–22, 2024, Porto, Portugal. Ed. by Francesco Marcelloni, Kurosh Madani, Niki van Stein, and Joaquim Filipe. Porto, Portugal: SciTePress: Science and Technology Publications, Lda, 2024, pp. 15–26. ISSN: 2184-3236. ISBN: 978-989-758-721-4. doi:10.5220/0012888500003837 (cit. on pp. 13–18, 118–120, 360).

Glossary I



- (1 + 1) EA** The (1 + 1) EA is a local search algorithm that retains the best solution x_c discovered so far during the search^{17,39}. In each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then the (1 + 1) EA uses a unary search operator that flips each bit independently with probability m/n , where usually $m = 1$. This operator is the main difference to randomized local search (RLS). The (1 + 1) EA is a special case of the $(\mu + \lambda)$ EA where $\mu = \lambda = 1$.
- (1 + 1) FEA** The (1 + 1) EA with Frequency Fitness Assignment (FFA) plugged in^{138,139,147}. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips each bit with probability $1/n$, where n is the dimension of the search space, distinguishing it from the unary operator in RLS with FFA (FRLS) which flips exactly one bit. On other domains, such as permutations, the term is used synonymously with FRLS.
- $(\mu + \lambda)$ EA** The $(\mu + \lambda)$ EA is an evolutionary algorithm (EA) where, in each generation, λ offspring solutions are generated from the current population of μ parent solutions. The offspring and parent populations are merged, yielding $\mu + \lambda$ solutions, from which then the best μ solutions are retained to form the parent population of the next generation. If the search space is the bit strings of length n , then this algorithm usually applies a mutation operator flipping each bit independently with probability $1/n$.
- BinInt** The *Binary Integer Problem* is a benchmark problem for bit-string based black-box search^{123,147}. It is very similar to OneMax: It tries to maximize the number of 1 bits in the bit string of a chosen dimension n . Different from OneMax, where each bit has the same weight in the objective function, the bits of BinInt have a exponentially increasing weight: The bit string is treated as a binary-encoded integer whose value should be maximized. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = 2^n - 1 - \sum_{i=1}^n 2^{i-1} \cdot x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately.
- LinHarm** The Linear Harmonic Function is a benchmark problem for bit-string based black-box search^{37,39,147}. It is very similar to OneMax: It tries to maximize the number of 1 bits in the bit string of a chosen dimension n . Different from OneMax, where each bit has the same weight in the objective function, the bits of LinHarm have a linear increasing weight. The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = \frac{1}{2}n(n+1) - \sum_{i=1}^n i \cdot x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The (1 + 1) EA can solve this problem with an expected runtime in $\Theta(n \ln n)$ ^{37,39}.

Glossary II



TSP Suite is a open source benchmarking framework for the TSP written in Java¹³¹. It uses the instances of the TSPLib and offers both automated experiment execution as well as several metrics to measure runtime. It is available at <https://thomasweise.github.io/tspSuite> and has been used in many research works, such as ¹⁴¹. It can be viewed as a predecessor of moptipy.

A* Algorithm The A* Algorithm is an a greedy best-first-first search for finding the shortest path between two locations ^{57,104}. The algorithm iteratively constructs the shortest path. It maintains a list of current candidate paths. For choosing the next candidate path whose end node should be expanded, it computes a value $f(p)$ of these paths p . $f(p)$ is the sum of the cost $g(p)$ so far incurred by the path as well as a heuristic $h(p)$ predicting the cost from the current end of the path to the goal. $h(p)$ must never overestimate that cost.

AP *Alternating Position Crossover* is a binary operator for EAs using a permutation-based representation ^{75,76}. It simply creates an offspring by selecting alternately the next element of the first parent and the next element of the second parent, omitting the elements already present in the offspring. For example, if parent 1 is (1, 2, 3, 4, 5, 6, 7, 8) and parent 2 is (3, 7, 5, 1, 6, 8, 2, 4), AP yields (1, 3, 2, 7, 5, 4, 6, 8) and exchanging the two parents would yield (3, 7, 5, 1, 6, 8, 2, 4) ⁷⁵. A generalized version of this operator will randomly choose a parent in each step, thus increasing the diversity of the results ¹⁴⁴.

B&B *Branch and bound* is a family of optimization methods that combines systematic exploration of the search space with both and objective function f subject to minimization and a lower bound function ⁸⁵. The idea is that we divide the search space into two sets. We then pick the set that seems more interesting and divide it again. We do this recursively until we have arrived at a set of size 1, which is our first solution y . We can compute its objective value $z = f(y)$. Now we continue with the sets that we have created and not yet explored. However, we can discard all sets for which the lower bound of the solution quality is $\geq z$, because they can never give us a better solution than y . If we have divided another set down to size 1 and it yields a better solution than y , of cause we update these values. Eventually, we have either discarded or fully expanded all sets – and then the solution we got must be optimal.

C is a programming language, which is very successful in system programming situations ^{31,110,129}.

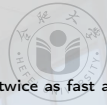
Concorde is an exact solver for the TSP^{3,24,25}, which could even solve the largest instance of TSPLib (with 85 900 cities) to optimality.

Glossary III



- DB** A *database* is an organized collection of structured information or data, typically stored electronically in a computer system. Databases are discussed in our book *Databases* ¹²⁷.
- DBMS** A *database management system* is the software layer located between the user or application and the database (DB). The DBMS allows the user/application to create, read, write, update, delete, and otherwise manipulate the data in the DB ¹⁴⁵.
- EA** An *evolutionary algorithm* is a metaheuristic optimization method that maintains a population of candidate solutions, which undergo selection (where better solutions are chosen with higher probability) and reproduction (where mutation and recombination create a new candidate solution from one or two existing ones, respectively) ^{6,128}.
- EAX** *Edge assembly crossover* is a binary operator to be used inside a EA for the TSP ⁹⁷⁻⁹⁹. It belongs to the SOTA on that field.
- FFA** *Frequency Fitness Assignment* (中文: 频率适应度分配) is a algorithm plugin for optimization methods applied to discrete or combinatorial problems with not-too-many different possible objective values ^{136,138,139}. It replaces the objective values in all comparisons with their absolute encounter frequency so far during the search. FFA has successfully been applied to the MaxSAT ^{138,139}, the TSP ^{81,82}, and the Quadratic Assignment Problem (QAP) ^{20,124}, amongst others. Its properties have been intensely researched on discrete benchmark functions such as OneMax, LeadingOnes, Trap, TwoMax, Jump, and Plateau ^{138,139,147}. The most common realizations of FFA so far are the $(1 + 1)$ FEA and FRLS.
- GA** A *Genetic Algorithm* is an EA usually for bit-string based search spaces ^{6,27,53,92,94,128} (although many researchers use the terms EA and GA synonymously). It is a population-based metaheuristic applying a unary and a binary search operator as well as a selection scheme. The original GA is the Standard Genetic Algorithm (SGA), which today is considered outdated. Modern variants are the Greedy $(2 + 1)$ GA and the Self-Adjusting $(1 + (\lambda, \lambda))$ Genetic Algorithm (Self-Adjusting $(1 + (\lambda, \lambda))$ GA) as well as bit-string based $(\mu + \lambda)$ EAs.
- GPX** *Generalized Partition Crossover* (GPX) is a binary operator for EAs for the TSP ¹⁴³. It is *respectful*, because the generated tour inherits all edges common to both parents. It is *transmits alleles*, because it only uses edges that occur in at least one parent. It belongs to the SOTA on the TSP, especially if used in conjunction with LKH ¹²⁵.

Glossary IV



- Greedy (2 + 1) GA** The Greedy (2 + 1) GA ^{16,17,119,120} is an EA for bit-string based search spaces, i.e., a GA. It was shown to be twice as fast as any mutation-only EA on OneMax ^{119,120}.
- Ising Model** The Ising models are benchmark problems for bit-string based black-box search ^{37,43,44,147}. Here, a neighborhood is defined around each bit. The bit needs to have the same value as all bits in the neighborhood structure T . Thus, we try to achieve that all bits have the same value. The objective function $f(x)$ of the minimization variant of this problem family can be defined as $f(x, T) = \sum_{(i,j) \in T} |x_i - x_j|$, where T contains the index pairs of neighboring bits and is defined differently for the Ising model instances Ising 1D and Ising 2D. The problems are highly epistatic, as all variables interact with some other variables.
- Ising 1D** The 1D is a benchmark problem for bit-string based black-box search ^{37,44,147}. The bits are arranged in a one-dimensional ring and we try to minimize the number of bits that have different values than their neighbors. Hence, the neighborhood structure T is defined as $T_{1D} = \{(i + 1, (i + 1 \bmod n) + 1) \mid i \in [0..n - 1]\}$. The problems are highly epistatic, as all variables interact with some other variables. The (1 + 1) EA has an expected runtime in $\mathcal{O}(n^3)$ on 1-Dimensional Ising Model (Ising 1D) ⁴⁴.
- Ising 2D** The 2D is a benchmark problem for bit-string based black-box search ^{37,43,147}. The bits are arranged in a two-dimensional torus and we try to minimize the number of bits that have different values than their neighbors. Hence, the neighborhood structure $T = T_{2D}$ is defined as the set of all $\{(\alpha + \beta N, \gamma + \delta N)\}$ pairs with $\alpha, \beta, \gamma, \delta \in [0..N - 1]$ and either $(\gamma = (\alpha + 1) \bmod N \text{ and } \delta = \beta) \text{ or } (\gamma = \alpha \text{ and } \delta = (\beta + 1) \bmod N)$ with $n = N^2$. The problems are highly epistatic, as all variables interact with some other variables. On a similar class of problems, the (1 + 1) EA has an exponential expected runtime ⁴³.
- Java** is another very successful programming language, with roots in the C family of languages ^{10,87}.
- JSSP** The *Job Shop Scheduling Problem* ^{9,21,77} is one of the most prominent and well-studied scheduling tasks. In a JSSP instance, there are k machines and m jobs ¹²¹. Each job must be processed once by each machine in a job-specific sequence and has a job-specific processing time on each machine. The goal is to find an assignment of jobs to machines that results in an overall shortest makespan, i.e., the schedule which can complete all the jobs in the shortest time. The JSSP is $\mathcal{NP}$ -complete ^{19,77}.

Glossary V



Jump The Jump problem is a benchmark problem for bit-string based black-box search^{39,46,147}. Like the OneMax, it tries to maximize the number of 1 bits in a bit string of chosen length n . However, the solutions representing the last ω bits around the optimum to flip are deceptive, meaning that they are actually worse and getting worse than surrounding solutions. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and $\omega + \|x\|_1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the deceptive basin around the optimum that an algorithm needs to cross. An $(1+1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^\omega + n \ln n)$ ³⁹. In experiments, the $(1+1)$ FEA achieves polynomial runtime on Jump, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{138,139,147}.

LABS *Low-Autocorrelation Binary Sequences* are bit strings of length n whose autocorrelation is as low as possible^{11,13,14,29,30,47,91,93,95,102,109}. This means that sub-sequences of any length $1 \leq k < n$ should ideally never repeat. The objective function $f : \{0, 1\}^n \mapsto \mathbb{N}_0$ represents the energy of a bit string x and is subject to minimization. We can define it as $f(x) = \sum_{k=1}^{n-1} \left(\sum_{i=0}^{n-1-k} s_i \cdot s_{i+k} \right)^2$ with $s_j = 2x_j - 1$. Such sequences are needed for communication engineering^{13,14,95}, cryptography^{29,95}, as well as modulation pulses in radar and sonar ranging¹⁰².

LeadingOnes The LeadingOnes problem is a benchmark problem for bit-string based black-box search^{115,142,147}. It tries to maximize the number of *leading* 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n \prod_{j=1}^i x_j$. Different from OneMax, the contribution to the objective value of the bit at index i thus depends on the state of all the bits *before* this index. The problem is unimodal, i.e., has only a single global optimum and no local optimum. Different from OneMax, it is not separable, i.e., we can optimize each variable separately. The expected runtime of $(1+1)$ EA on LeadingOnes is in $\Theta(n^2)$ ³⁹. The average runtime $(1+1)$ FEA on LeadingOnes seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^3)$ ¹³⁸.

LK The *Lin-Kernighan* (LK) algorithm⁸⁴ is a SLS method for the TSP. The LK algorithm performs iterative k -opt moves, i.e., in each step, it constructs the two sets X and Y of k edges such that removing the edges in X from the current tour T and then adding the edges from Y to T results in a better tour. The LK algorithm is one of the most classical approaches to the TSP.

LKH The *Lin-Kernighan-Helsgaun* algorithm^{25,58,59} is a SLS method for the TSP based on the Lin-Kernighan (LK) algorithm⁸⁴. LKH-1 was proposed in⁵⁹ and LKH-2 in⁵⁸. It holds the record in the *World TSP*²⁶ as of 2025.

Glossary VI



- MA** A *Memetic Algorithm* is an EA where each new solution is refined by a local search before it enters the population ¹²⁸.
- MaxSAT** The goal of satisfiability problems is to find an assignment for n Boolean variables that make a given Boolean formula $F : \{0, 1\}^n \mapsto \{0, 1\}$ become true. In the *Maximum Satisfiability* problem (MaxSAT) ⁶⁰, F is given in conjunctive normal form, i.e., the variables appear either directly or negated in m “or” clauses, which are all combined into one “and.” The objective function $f(x)$, subject to minimization, computes the number of clauses which are false under the variable setting x . If $f(x) = 0$, then all clauses of F are true, which solves the problem. The MaxSAT problem is $\mathcal{NP}$ -complete ²³ and exhibits low epistasis but deceptiveness ¹¹², making it hard for local searches.
- metaheuristic** A *metaheuristic* is an optimization algorithm which performs a cycle of trial and error during which candidate solutions are sampled from the solution space $\mathbb{Y}$ and evaluated using an objective function $f : \mathbb{Y} \mapsto \mathbb{R}$ ^{51,128}. The results of the objective function are used to decide which solution is promising. The neighborhoods around promising solutions or similarly-structured solutions are then sampled in the next cycle. Prominent metaheuristics are SLSes, SA, TS, and EAs.
- moptipy** is the *Metaheuristic Optimization in Python* library ¹³⁷. It has been used in several different research works, including ^{20,81–83,124,144,148,149}. Algorithms implemented by moptipy include EAs, (1 + 1) EA, RLS, SA, and the (1 + 1) FEA, amongst others. Learn more at <https://thomasweise.github.io/moptipy> and <https://thomasweise.github.io/moptipyapps>.
- N-Queens** The N-Queens Problem is a benchmark problem for bit-string based black-box search ^{37,147}. The goal is to arrange N queens on an $N \times N$ chessboard so that none can attack the other. The search space has thus dimension $n = N^2$, with each bit standing for a field and encoding whether or not a queen is placed there. The objective function $f(x)$ of the minimization variant is $N - (\sum_{i=1}^n x_i) - N \sum_{\xi} \max \{0, Q_{\xi}(x) - 1\}$ where $Q_{\xi}(x)$ is the number of queens in the horizontal, vertical, or diagonale ξ . The problem is quite epistatic.
- OneMax** OneMax is the simplest possible benchmark problem for bit-string based black-box search ^{40,96,147}. The goal is to maximize the number of 1 bits in the bit string of a chosen dimension n . The objective function $f(x)$ of the minimization variant of this problem can be defined as $f(x) = n - \sum_{i=1}^n x_i$. The problem is unimodal, i.e., has only a single global optimum and no local optimum. It is fully separable, i.e., we can optimize each variable separately. The expected runtime of (1 + 1) EA on OneMax is in $\Theta(n \ln n)$ ^{33,96}. The average runtime (1 + 1) FEA on OneMax seems to be slower by factor proportional to n , i.e., seems to be in $\mathcal{O}(n^2 \ln n)$ ¹³⁸.

Glossary VII



- Plateau** The Plateau problem is a benchmark problem for bit-string based black-box search^{2,147}. It is structured exactly like Jump, but instead of being deceptive, the last ω bits to flip around the optimum are neutral. The objective function $f(x)$ of the minimization variant is $n - \|x\|_1$ if $\|x\|_1 = n$ or $\|x\|_1 \leq n - \omega$ hold, and ω otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. This means that, the larger ω , the larger the neutral plane around the optimum that an algorithm needs to cross. An $(1 + 1)$ EA has an expected runtime of $\Theta(n^\omega)$ on plateaus². The average runtime of the $(1 + 1)$ FEA on Plateaus is slower by a factor seemingly proportional in n .
- PostgreSQL** An open source object-relational DBMS^{42,101,108,122}. See <https://postgresql.org> for more information.
- Python** The Python programming language^{63,79,89,130}, i.e., what you will learn about in our book¹³⁰. Learn more at <https://python.org>.
- QAP** The *Quadratic Assignment Problem* is an optimization problem where the goal is to assign a set of n facilities to a set of n locations^{15,20,73,86}. Such an assignment can be represented as a permutation x of the first n natural numbers, where x_i specifies the location where facility i should be placed. For each QAP, a distance matrix D is given, where D_{pq} specifies the distance from location p to location q , as well as a flow matrix F , where F_{ij} is the amount of material flowing from facility i to facility j . The objective function $f(x)$ then rates a permutation x as $f(x) = \sum_{i=1}^n \sum_{j=1}^n D_{x_i x_j} F_{ij}$. The QAP is $\mathcal{NP}$ -complete¹¹⁶.
- RLS** Randomized local search retains the best solution x_c discovered so far during the search and, in each step, it applies a unary search operator to this best-so-far solution x_c and derives a new solution x_n . If the new solution x_n is *better or equally good* when compared with x_c , i.e., not worse, then it replaces it, i.e., is stored as the new x_c . If the search space are bit strings of length n , then RLS uses a unary search operator that flips exactly one bit. This operator is the main difference to $(1 + 1)$ EA.
- FRLS** The RLS with FFA plugged in¹⁴⁷. On the discrete bit-string domains, this algorithm has a very specific structure: Its unary search operator flips exactly one bit, distinguishing it from the unary operator in the $(1 + 1)$ FEA, which flips each bit with probability $1/n$. On other domains, such as permutations, we use the term synonymously with $(1 + 1)$ FEA.
- SA** Simulated Annealing is a SLS that sometimes accepts a worse solution^{18,66,70,106}. The probability to do so decreases over time and with the difference in objective values, i.e., is the lower the worse the new solution is.

Glossary VIII



SATLIB is a library of benchmark instances for the Maximum Satisfiability (MaxSAT) available at <https://www.cs.ubc.ca/~hoos/SATLIB/index-ubc.html>^{61,62}.

Self-Adjusting $(1 + (\lambda, \lambda))$ GA The Self-Adjusting $(1 + (\lambda, \lambda))$ GA is a theory-inspired EA for bit string-based search spaces¹⁷. It has linear runtime on the OneMax problem³².

SGA The Standard Genetic Algorithm^{6,27,53,92,94,128} was the first EA. It maintains a population of solutions and applies mutation and crossover to generate offspring solutions. It uses fitness proportionate selection to choose which solutions should “survive” into the next generation, which today is considered a very bad design choice, see, e.g.,¹⁴².

SLS *Stochastic local search* or *local search* for short, is one of the simplest metaheuristic search paradigms⁶⁰: A single “current” candidate solution is maintained and new solutions are sampled from its neighborhood. If these solutions are deemed interesting based on their objective value, they replace the “current”. The cycle continues. Basic examples are RLS and $(1 + 1)$ EA. More complex examples that add mechanisms to escape local optima on top of the SLS paradigm are TS and SA.

SOTA State-of-the-Art, i.e., the best-performing methods for a field or problem type

Trap The Trap problem is a benchmark problem for bit-string based black-box search^{39,100,147}. It is structured exactly like OneMax, however the worst and the best solution are swapped. This makes the problem highly deceptive, as following the “gradient” in the search space leads the search away from the global optimum. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = 0$ and $n - \|x\|_1 - 1$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1 + 1)$ EA will converge to the resulting local optimum and thus have an expected runtime of $\Theta(n^n)$ ³⁹. In experiments, the $(1 + 1)$ FEA achieves polynomial runtime on Trap, seemingly in $\mathcal{O}(n^2 \ln n)$ ^{138,139,147}.

TS *Tabu Search* is one of the most successful SLSes^{49,50,52}. In order to avoid returning to previous solutions, it declares certain solutions or search moves as “tabu.” For example, if a bit in a discrete problem has been flipped by the search operator, it may be forbidden to flip the bit again for some time, thus preventing the algorithm from getting trapped in a cycle. This rule may be overridden if a new best-so-far solution is discovered that outperforms all previous solutions, in which case it would be accepted (this is the so-called aspiration criterion).

Glossary IX



TSP In an instance of the *Traveling Salesperson Problem*, also known as *Traveling Salesman Problem*, a set of n cities or locations as well as the distances between them are defined^{3,56,78,81,131,141}. The goal is to find the shortest round-trip tour that starts at one city, visits all the other cities one time each, and returns to the origin. The TSP is one of the most well-known $\mathcal{NP}$ -hard combinatorial optimization problems^{48,56}.

TSPLib is a library of benchmark instances for the Traveling Salesperson Problem (TSP)^{113,114}. Its original URL was available until 2025 and then became defunct. The repository <https://github.com/shredderzvj/TSPLIB> contains the data. Additionally, you may also check <https://github.com/thomasWeise/tspSuite>, which contains all the TSPLib instances as well as Java code to load them. The TSPLib contains 110 instances of symmetric TSPs with scales n ranging from 14 to 85 900, but 93 instances have less than 2000 nodes. There also are 19 asymmetric TSPs with scales $n \in 17..443$.

TwoMax The TwoMax problem is a benchmark problem for bit-string based black-box search^{46,126,147}. It has a global and a local optimum, both of about the same size, at opposite ends of the search space. For bit strings of length n , the objective function $f(x)$ of the minimization variant is 0 if $\|x\|_1 = n$ and $1 + n - \max\{\|x\|_1, n - \|x\|_1\}$ otherwise, where $\|x\|_1 = \sum_{i=1}^n x_i$. An $(1 + 1)$ EA has a 50/50 chance to converge to the local or global optimum and thus has an expected runtime in $\Theta(n^n)$ ^{45,46}. In experiments, the $(1 + 1)$ FEA achieves polynomial runtime seemingly in $\mathcal{O}(n^2 \ln n)$ on TwoMax^{138,139,147}.

URI A *Uniform Resource Identifier* is an identifier for an abstract or physical resource in the internet¹⁴⁶. It can be a URL, a name, or both. URIs are supersets of URLs. The connection strings of the PostgreSQL DBMS are examples for URIs.

URL A *Uniform Resource Locator* identifies a resource in the World Wide Web (WWW) and a way to obtain it by describing a network access mechanism. The most notable example of URLs is the text you write into web browsers to visit websites⁸. URLs are subsets of Uniform Resource Identifiers (URIs).

W-Model The W-Model is a tunable benchmark function for bit-string based black-box search^{132,140}. It allows you to adjust the amount of ruggedness, deceptiveness, epistasis, and uniform neutrality in the search space.

WWW World Wide Web^{7,28}

Glossary X



$\circ^*$ the superscript $*$ symbol denotes that $\circ$ is a global optimum, i.e., there is no better solution than $\circ$.

$i..j$ with $i, j \in \mathbb{Z}$ and $i \leq j$ is the set that contains all integer numbers in the inclusive range from i to j . For example, $5..9$ is equivalent to $\{5, 6, 7, 8, 9\}$

$f(y)$ The *objective function* $f : \mathbb{Y} \mapsto \mathbb{R}$ can compute a cost or rating for each candidate solution y in the solution space $\mathbb{Y}$. The smaller this value, the better the solution, i.e., in the context of this work, objective functions are subject to minimization. Objective functions are not necessarily continuous and it is also quite common that they have integer or natural numbers as results, e.g., we often have $f : \mathbb{Y} \mapsto \mathbb{N}_0$ instead of $f : \mathbb{Y} \mapsto \mathbb{R}$.

$\gamma(x)$ The decoding function $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ translates point from the search space $\mathbb{X}$ to one point y in the solution space $\mathbb{Y}$. It is a very common case that $\mathbb{X} = \mathbb{Y}$ and that the decoding function γ is the identity mapping.

n We usually use n to denote the scale, i.e., the size of the problem to solve. For example, in a TSP, this may be the number of cities, in a discrete benchmark problem such as OneMax or Trap, this may be the number of bits over which the problem is defined, in a QAP, this may be the number of facilities and locations, and in a MaxSAT problem, this may be the number of variables that are input to the Boolean formula that we want to become true. There are also problems where the scale is not defined straightforwardly like this. For example, in a Job Shop Scheduling Problem (JSSP), we have a number of machines and a number of jobs, which together define the size of the problem. But even then, the scale n relates to the size of the solution space $\mathbb{Y}$, and, hence, also of the search space $\mathbb{X}$, which usually grow exponentially with n . And this is already the main issue: The larger n , the (exponentially) larger the number of possible solutions and thus, the larger the runtime of algorithms..

$\mathbb{N}_0$ the set of the natural numbers *including* 0, i.e., 0, 1, 2, 3, and so on. It holds that $\mathbb{N}_0 \subset \mathbb{Z}$.

$\mathcal{NP}$ is the class of computational problems that can be solved in polynomial time by a non-deterministic machine and can be verified in polynomial time by a deterministic machine (such as a normal computer) ⁵⁵.

$\mathcal{NP}$ -complete A decision problem is $\mathcal{NP}$ -complete if it is in $\mathcal{NP}$ and all problems in $\mathcal{NP}$ are reducible to it in polynomial time ^{55,111}. A problem is $\mathcal{NP}$ -complete if it is $\mathcal{NP}$ -hard and if it is in $\mathcal{NP}$.

Glossary XI



$\mathcal{NP}$ -hard Algorithms that guarantee to find the correct solutions of $\mathcal{NP}$ -hard problems ^{19,23,77} have a worst-case runtime that grows exponentially with the problem scale. A problem is $\mathcal{NP}$ -hard if all problems in $\mathcal{NP}$ are reducible to it in polynomial time ⁵⁵.

$\Omega(g(x))$ If $f(x) = \Omega(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $f(x) \geq c * g(x) \geq 0 \forall x \geq x_0$ ^{71,72}. In other words, $\Omega(g(x))$ describes a lower bound for function growth.

$\mathcal{O}(g(x))$ If $f(x) = \mathcal{O}(g(x))$, then there exist positive numbers $x_0 \in \mathbb{R}^+$ and $c \in \mathbb{R}^+$ such that $0 \leq f(x) \leq c * g(x) \forall x \geq x_0$ ^{5,71,72,74}. In other words, $\mathcal{O}(g(x))$ describes an upper bound for function growth.

$\Theta(g(x))$ If $f(x) = \Theta(g(x))$, then $f(x) = \mathcal{O}(g(x))$ and $f(x) = \Omega(g(x))$ ^{71,72}. In other words, $\Theta(g(x))$ describes an exact order of function growth.

$\mathbb{R}$ the set of the real numbers.

$\mathbb{R}^+$ the set of the positive real numbers, i.e., $\mathbb{R}^+ = \{x \in \mathbb{R} : x > 0\}$.

$\mathbb{X}$ The search space $\mathbb{X}$ is the space through which an optimization algorithm navigates in order to find good solutions. The search operators of metaheuristics are used to sample points $x \in \mathbb{X}$, for example. On many benchmark problems, the objective functions f directly evaluate the elements $x \in \mathbb{X}$, i.e., are defined as $f : \mathbb{X} \mapsto \mathbb{R}$. However, very often these elements x are not the same data structures that the user is interested in and/or that the objective functions evaluate: It is common that the search space $\mathbb{X}$ is a simple and plain data structure that can be tackled with standard operations or mathematical tools, such as real vectors $\mathbb{R}^n$, permutations, or bit strings, whereas the solution space $\mathbb{Y}$ can be something more complicated, like a Gantt chart. Then, the user only cares about the solutions $y \in \mathbb{Y}$ and not about the search space $\mathbb{X}$. A decoding $\gamma : \mathbb{X} \mapsto \mathbb{Y}$ is employed to translate the $x \in \mathbb{X}$ to $y = \gamma(x)$ with $y \in \mathbb{Y}$.

x a point in the search space $\mathbb{X}$.

Glossary XII



- $\mathbb{Y}$ the set of candidate solutions of an optimization problem is called the *solution space* $\mathbb{Y}$. The solution space corresponds to the data structure holding the candidate solutions y . These are the elements that are passed to the user after the optimization process is completed. They are also the parameters of the objective function(s) $f : \mathbb{Y} \mapsto \mathbb{R}$.
- y a candidate solution $y \in \mathbb{Y}$ is one possible solution to an optimization problem. After the optimization process is completed, one or multiple such solutions are passed to the user.
- $\mathbb{Z}$ the set of the integers numbers including positive and negative numbers and 0, i.e., $\dots, -3, -2, -1, 0, 1, 2, 3, \dots$, and so on. It holds that $\mathbb{Z} \subset \mathbb{R}$.